

Befach Task Manager (SmartTask AI) — Complete Project Documentation

Befach Task Manager (SmartTask AI) — Complete Project Documentation

Table of Contents

Part I — Project Overview & Infrastructure

1. Project Overview
2. Deployment Architecture
3. Android App Configuration
4. Documentation Guides (docs/)
5. Deploy Guides (deploy/)
6. Environment Variables

Part II — Backend (server/)

7. Server Core Modules
8. API Routes (server/src/routes/)
9. AI Modules (server/src/ai/)

Part III — Frontend: App Shell & Pages (client/src/)

10. Build & Shell Configuration
11. App Shell & Core Modules
12. Pages (client/src/pages/)

Part IV — Frontend: Components, Voice Pipeline, Libraries & Styles

13. Shared Components (client/src/components/)
14. Voice Pipeline (client/src/voice/)
15. Utility Libraries (client/src/lib/)
16. styles.css — Design System Summary

Befach Task Manager (SmartTask AI) — Complete Project Documentation

Generated: 10 July 2026 · Covers every source file, component, function, route, database table, and configuration in the project. This document describes the code as-is; nothing in the codebase was modified to produce it.

Table of Contents

Part I — Project Overview & Infrastructure

1. Project Overview
2. Deployment Architecture (Render + Vercel / Oracle Cloud)

3. Android App Configuration (Capacitor)
4. Documentation Guides (docs/)
5. Deploy Guides (deploy/)
6. Environment Variables (full reference)

Part II — Backend (server/) 7. Server Core Modules — index.js, db.js (full schema), auth.js, util.js, mailer.js, push.js, scheduler.js, digest.js, seed.js, restore.js, cliq.js, ragBackfill.js, ws/chatHub.js, ws/liveTranscribe.js 8. API Routes — assistant, auth, chat, dashboards, digest, invites, meetings, notifications, platform, tasks, usage, users 9. AI Modules — assistant, assistantChat, claude, dates, embeddings, extractor, openai, openrouter, ragIndex, ragRetrieve, rules, transcribe, usage, voiceCommand, voiceSearch, voiceTask

Part III — Frontend: App Shell & Pages (client/src/) 10. Build & Shell Configuration — package.json, vite.config.ts, capacitor.config.ts, index.html 11. App Shell & Core Modules — main.tsx, App.tsx (routing table), api.ts, auth.tsx, googleAuth.ts, push.ts, back.ts, report.ts, ui.tsx 12. Pages — AcceptInvite, Admin, Assistant, Chats, Dashboard, ForgotPassword, Login, MeetingDetail, Meetings, Platform, PrivacyPolicy, ResetPassword, Signup, Tasks, VerifyEmail

Part IV — Frontend: Components, Voice, Libraries & Styles 13. Shared Components — ConfirmHost, NotificationBell, ParticipantPicker, PasswordStrength, ProfileModal, TaskBoard, TaskDrawer, ToastHost, UserManagement, VoiceAssistant 14. Voice Pipeline — recorder.ts, tts.ts, useVoiceAssistant.ts, wakeword.ts 15. Utility Libraries — confirm.ts, passwordStrength.ts, pcmStream.ts, toast.ts, useEscape.ts, wallpaper.ts 16. styles.css — Design System Summary

Part I — Project Overview & Infrastructure

1. Project Overview

SmartTask AI (branded in the mobile app as **Befach Task Manager**) is an AI-powered task-management web application that converts **multilingual meeting conversations** — Telugu, Hindi, English, and code-mixed speech in both Latin and native scripts — into structured, trackable, assignable tasks automatically.

Core capabilities

- **Meeting intelligence** — paste a transcript and receive a speaker-wise breakdown, executive summary, decisions, risks, blockers, follow-ups, and extracted tasks.
- **Task extraction** — each task carries a title, description, assignee, assigned-by, due date, priority, ownership confidence, and the original spoken quote.
- **Ownership detection** — recognizes vocative address ("Munidhar, ..."), self-commitment ("I'll ...", "nenu chestanu"), or flags **Needs Confirmation** when unclear.
- **Natural-language deadlines** — resolves phrases like "by Friday", "repu", "kal", "end of week", "next Monday", "before deployment" into real dates.

- **Priority detection** — infers Critical / High / Medium / Low from urgency cues across all three languages.
- **Full task lifecycle** — To Do → In Progress → Blocked → In Review → Done → Reopened, plus comments, subtasks, dependencies, progress tracking, and a manager **approval workflow**.
- **Role-based dashboards** — Employee (my work) and Manager (team workload, project progress, overdue, org metrics, users, and audit log; the manager acts as the org admin).
- **AI assistant** — answers natural-language questions such as "show overdue tasks", "who is responsible for deployment", "daily status report", "workload imbalance".
- **Voice assistant** — hands-free "hey BTM" command interface (see setup summary below).
- **Security** — JWT authentication, role-based access control (RBAC), and an audit log of every mutation.

Tech stack

Layer	Choice
Frontend	React 18 + TypeScript + Vite
Backend	Node.js + Express
Database	SQLite via better-sqlite3 (zero setup)
Auth	JWT + bcrypt
AI engine	Claude API for multilingual extraction, with an offline rule-based fallback so the app runs with zero API keys

Architecture

```

client/  React SPA (Vite dev server on :5173, proxies /api → :4000)
server/  Express API (:4000)
  src/ai/      extractor.js (orchestrator) · claude.js · rules.js (offline)
· dates.js · assistant.js
  src/routes/  auth · users · meetings · tasks · dashboards · assistant
  src/db.js    SQLite schema      src/seed.js  demo org + 2 multilingual
meetings
  data/        smarttask.db (auto-created & seeded on first run)

```

The AI layer is **pluggable**: with ANTHROPIC_API_KEY set, transcripts are analyzed by Claude; without it, the deterministic rule-based engine (rules.js + dates.js) runs entirely offline. If Claude errors, the app automatically falls back to the rule-based engine.

Requirements: Node 18+ (uses global fetch). Demo accounts (password password123): priya@demo.io (Manager/Admin) and munidhar@demo.io (Employee). The database seeds with two multilingual meetings that produce ~11 tasks.

2. Deployment Architecture

The project supports two deployment paths. Both serve the same backend to both the web frontend and the Android APK.

Path A — Render (backend) + Vercel (frontend)

Defined by `render.yaml` (Render Blueprint) and `client/vercel.json`.

- **Backend on Render** — a single web service named `smarttask-api`, Node runtime, rooted in `server/`. It runs on the **Starter plan (~\$7/mo)** specifically because a **persistent disk** is required — the free tier has no disk, and the SQLite database plus uploaded files must survive restarts and redeployments.
 - **Region:** `singapore` (closest to India).
 - **Build command:** `npm install && cd ../client && npm install --include=dev && npm run build` — compiles the `better-sqlite3` native module and builds the web client. `--include=dev` is required so the client build has access to `vite/tsc` (`devDependencies`).
 - **Start command:** `npm start (node src/index.js)`, which also serves `../client/dist` — so one service hosts both the API and the website.
 - **Health check:** `/api/health`.
 - **Auto-deploy:** enabled — redeploys on every push to the connected branch.
 - **Persistent disk:** named `data`, 1 GB, mounted at `/opt/render/project/src/server/data`. The app writes `smarttask.db`, `chat_uploads/`, and `avatars/` here.
- **Frontend on Vercel** — free Hobby tier. Framework preset **Vite**, build command `npm run build`, output directory `dist`. A catch-all rewrite (`/(.*) → /index.html`) supports SPA client-side routing. Root directory is set to `client`, and `VITE_API_BASE` is set to the Render service URL.
- **Android APK** — `client/.env.production` points at the Render URL. Because Render serves over `https://`, WebSockets (live transcription, chat) automatically upgrade to secure `wss://` — no cleartext needed.

The deploy guide notes the repo lives at `github.com/Munidhar05/Task-Manager1` on branch `reddy-changes`; both Render and Vercel auto-deploy on push, but the APK must be rebuilt manually since it ships a static copy of the frontend.

Path B — Oracle Cloud (self-hosted, free)

Defined by `deploy/DEPLOY-ORACLE.md` and an install script `deploy/oracle-setup.sh`.

- Hosts the backend 24/7 for free on an **Oracle Always Free VM** (Ampere Arm VM.Standard.A1.Flex, 2 OCPU / 12 GB, Ubuntu 22.04/24.04), so the company APK works over mobile data with the PC off. SQLite handles 50+ users without a database change.
- A single curl-piped setup script installs Node, pulls the code, builds it, opens the firewall, and installs a systemd service (`smarttask`) for 24/7 uptime.
- Networking requires opening **TCP port 4000** (and later **443**) in the Oracle Security List ingress rules.
- Secrets go in `/opt/smarttask/server/.env`; the service is controlled via `systemctl restart smarttask`.

- **HTTPS** (strongly recommended) is added via **Caddy** as a reverse proxy in front of port 4000, with automatic Let's Encrypt certificates for a domain such as `tasks.befach.com`.
 - The APK is pointed at the server by setting `VITE_API_BASE` (either `http://<IP>:4000` or `https://tasks.befach.com`) and rebuilding. The database is a single file at `/opt/smarttask/server/data/smarttask.db` for easy backup.
-

3. Android App Configuration

The native Android shell is a **Capacitor** wrapper around the Vite-built web app.

Capacitor config (`client/capacitor.config.ts`)

- **App ID:** `io.smarttask.app`
- **App name:** Befach Task Manager
- **Web directory:** `dist` (Capacitor copies the Vite build into the native shell)
- **Server scheme:** `androidScheme: 'http'` with `cleartext: true` — serves the app over `http://localhost` (not the default `https`) so it can call a plain-http backend without the `WebView` blocking it as mixed content.
- **`android.allowMixedContent: true`** — belt-and-suspenders to allow http requests from the `WebView`.
- Commented-out blocks document two optional native integrations: `@capacitor-community/text-to-speech` for voice-assistant spoken replies, and `@codetrix-studio/capacitor-google-auth` (`plugins.GoogleAuth`) for native Google Sign-In, whose `serverClientId` should be set to the web OAuth client id.

Gradle build (`client/android/app/build.gradle`)

- **namespace / applicationId:** `io.smarttask.app`
- **versionCode:** `2` — **versionName:** `2.0`
- **Release signing** reads credentials from a git-ignored `android/keystore.properties`. When that file is absent (CI or fresh clone), release builds **fall back to debug signing** so they still assemble. `minifyEnabled` is false.
- The Google Services plugin (`com.google.gms.google-services`) is applied only if a `google-services.json` file is present; otherwise it is skipped and push notifications won't work.

SDK versions (`client/android/variables.gradle`)

- **minSdkVersion:** `24` — **compileSdkVersion:** `36` — **targetSdkVersion:** `36`
- Cordova Android 14.0.1; AndroidX AppCompat 1.7.1, Core 1.17.0, Activity 1.11.0, Fragment 1.8.9; core-splashscreen 1.2.0; JUnit 4.13.2 / `androidx.test`.

Permissions (`AndroidManifest.xml`)

Permission	Purpose
INTERNET	Network access to the backend API

Permission	Purpose
RECORD_AUDIO	Live meeting recording / browser captions (getUserMedia + Web Speech)
MODIFY_AUDIO_SETTINGS	Audio capture configuration
POST_NOTIFICATIONS	Push notifications (Android 13+ runtime permission)

The manifest also declares the single MainActivity (launcher, singleTask launch mode, exported="true") and an androidx.core.content.FileProvider (authority \${applicationId}.fileprovider, non-exported, with grantable URI permissions) backed by @xml/file_paths.

Capacitor plugins

(client/android/app/src/main/assets/capacitor.plugins.json)

Package	Class	Role
@capacitor-community/keep-awake	KeepAwakePlugin	Keep the screen awake (e.g. during recording)
@capacitor-community/text-to-speech	TextToSpeechPlugin	Native TTS for voice-assistant spoken replies
@capacitor/app	AppPlugin	App lifecycle / state events
@capacitor/push-notifications	PushNotificationsPlugin	Push notification handling

MainActivity.java

Located at

client/android/app/src/main/java/io/smarttask/app/MainActivity.java:

```
package io.smarttask.app;
```

```
import com.getcapacitor.BridgeActivity;
```

```
public class MainActivity extends BridgeActivity {}
```

This is the standard Capacitor entry point. It declares one class, MainActivity, which extends Capacitor's BridgeActivity and has an **empty body** — **it defines no methods of its own**. All behavior (creating the WebView, initializing the Capacitor JS↔native bridge, loading the web bundle from dist, and wiring up the registered plugins) is inherited from BridgeActivity. There are no overrides or custom methods to document.

4. Documentation Guides (docs/)

GOOGLE_SIGNIN_SETUP.md — Google Sign-In

The "Continue with Google" flow is already wired up and is **login-only**: Google authenticates people who **already have an account** (created via signup or invite); unknown Google emails are rejected and no organizations are auto-created. Flow: the client obtains a Google **ID token** (web button or native Android account picker), posts it to `POST /api/auth/google`, and the server verifies the token, matches the user by **email**, links their `google_id`, and returns the normal JWT session. The guide covers:

- **Part 1 (Web)**: create a **Web application** OAuth Client ID in Google Cloud Console, configure the OAuth consent screen, add authorized JavaScript origins (`http://localhost:5173`, `http://localhost`, and the production origin like `https://app.befach.com`), leave redirect URIs empty (token flow), then set `GOOGLE_CLIENT_ID` on the server and `VITE_GOOGLE_CLIENT_ID` on the client. The same Client ID renders the web button and verifies tokens.
- **Part 2 (Android native)**: the web GIS button is unreliable in the Android WebView, so a native plugin (`@codetrix-studio/capacitor-google-auth`) is used. Steps: install the plugin and `npx cap sync android`; create an **Android** OAuth Client ID keyed to package name `io.smarttask.app` plus the signing **SHA-1** (debug and release); add that Android id to the server's `GOOGLE_CLIENT_IDS_EXTRA` so the token's aud claim is accepted; and set the plugin's `serverClientId` (in `capacitor.config.ts`) to the **web** client id so the server can verify the returned token.
- Includes an env-var reference table and troubleshooting for the four common error codes (button not showing, 501 not configured, 401 unverifiable aud, 404 no account found).

VOICE_ASSISTANT_SETUP.md — Voice Assistant ("hey BTM")

Documents a hands-free voice assistant available as a floating mic button on every page, usable by managers, admins, and employees. It is **conversational** (asks follow-up questions and confirms before mutating), routes every action through the normal task APIs (so permissions/notifications are unchanged), and supports English/Hindi/Telugu/code-mixed.

- **Out of the box**: nothing to configure as long as the server has an AI engine (`OPENROUTER_API_KEY`, or Anthropic/OpenAI). Tap-the-mic works on web and Android; spoken replies use the browser's speech synthesis.
- **Optional 1 — "hey BTM" wake word**: uses **openWakeWord** (free, open-source, on-device via `onnxruntime-web`). Requires three ONNX models in `client/public/wakeword/`: two shared models (`melspectrogram.onnx`, `embedding_model.onnx`, already committed) and a custom `hey_btm.onnx` the user trains via `openWakeWord`'s automatic Colab notebook (target phrase "hey btm"). Enabled with `VITE_WAKEWORD_ENABLED=true`. Runs on-device only while the app is open; sensitivity tuned via `VITE_WAKEWORD_THRESHOLD` (default 0.5). For offline use, WASM can be self-hosted via `VITE_ORT_WASM_PATH=/ort/`. Documents the model tensor-shape contract.
- **Optional 2 — native Android TTS**: install `@capacitor-community/text-to-speech` and sync; no config needed (code auto-routes to native TTS on Android, `speechSynthesis` on web).

- **Server side:** the brain is `POST /api/assistant/command`, which turns each utterance into one resolved action (create/status/assign/priority/due), a navigation target, an answer, or a clarifying question; usage is metered under the `voice_command` feature. Includes permission notes (`RECORD_AUDIO` already granted) and troubleshooting. (Note: a stray troubleshooting line references a porcupine folder and `VITE_PORCUPINE_ACCESS_KEY`, apparently leftover from an earlier wake-word implementation.)

client/public/wakeword/README.md — Wake-word model files

A focused companion to the voice-assistant guide. Lists the three required ONNX models in that folder: `melspectrogram.onnx` and `embedding_model.onnx` (both included, Apache-2.0, ~2.4 MB total, pulled from openWakeWord v0.5.1 release assets) and `hey_btm.onnx` (the user must train it). Gives the remaining step — train via the openWakeWord automatic Colab notebook with target phrase "hey btm", export ONNX, rename, and drop in the folder — then set `VITE_WAKEWORD_ENABLED=true` and rebuild. Until then the assistant works via tap-the-mic.

5. Deploy Guides (deploy/)

DEPLOY-ORACLE.md

A step-by-step guide to hosting the backend free and 24/7 on an Oracle Cloud Always Free VM. **Part A** creates the server (sign up, launch an Ampere Arm VM.Standard.A1.Flex Ubuntu instance with a public IP, open TCP port 4000 in the Security List). **Part B** connects via SSH from Windows PowerShell using the generated key. **Part C** runs a one-line curl-piped `oracle-setup.sh` that installs Node, pulls code, builds, opens the firewall, and installs a systemd service; the operator then creates `/opt/smarttask/server/.env`, restarts the service, and verifies `/api/health`. **Part D** points the APK at the server (`VITE_API_BASE`) and rebuilds it. **Part E** (strongly recommended) adds free HTTPS via a DNS A-record, port 443, and Caddy auto-HTTPS reverse-proxying to `localhost:4000`. Closes with update steps (`git pull + systemctl restart`) and handy `journalctl/systemctl` commands, noting the DB is a single backup-able file.

DEPLOY-RENDER-VERCEL.md

A guide to hosting the backend on **Render** (paid Starter ~\$7/mo, persistent disk) and the frontend on **Vercel** (free). **Part A** ensures the repo (with `render.yaml + vercel.json`) is pushed to GitHub. **Part B** deploys the backend on Render via **New → Blueprint**, applying the `smarttask-api` service, pasting secret env vars (AI, transcription, email) into the Environment tab, waiting for Live, and verifying `/api/health`; first login uses the seeded `priya@demo.io` account. **Part C** deploys the frontend on Vercel by importing the repo with root directory `client`, Vite preset, and `VITE_API_BASE` set to the Render URL; suggests "Add to Home screen" for an app-like feel. **Part D** rebuilds the Android APK against Render (WebSockets auto-upgrade to `wss://`). An optional section covers migrating local data, and the guide closes with auto-deploy-on-push behavior, `logs/health/backup` tips, and a cost summary.

6. Environment Variables

From `render.yaml`

Variable	Source / handling	Purpose
NODE_VERSION	fixed value 20.19.0	Satisfies Vite 8's engine requirement so vite build doesn't crash; keeps the Node 20 ABI so better-sqlite3's prebuilt binary applies
JWT_SECRET	generateValue: true	JWT signing key; Render generates a strong stable value once
OPENROUTER_API_KEY	secret (sync:false)	OpenRouter AI provider key (primary AI engine)
OPENROUTER_MODEL	secret	OpenRouter model selection
ANTHROPIC_API_KEY	secret	Anthropic Claude API key (alternate AI engine)
ANTHROPIC_MODEL	secret	Claude model id (e.g. claude-opus-4-8)
OPENAI_API_KEY	secret	OpenAI API key (alternate AI engine)
OPENAI_MODEL	secret	OpenAI model selection
TRANSCRIPTION_PROVIDER	secret	Selects the audio-transcription backend
OPENAI_TRANSCRIBE_MODEL	secret	Model used for OpenAI-based transcription
SARVAM_API_KEY	secret	Sarvam transcription provider key
SARVAM_LANGUAGE	secret	Language setting for Sarvam transcription
GROQ_API_KEY	secret	Groq transcription/inference key
CLIQ_WEBHOOK_URL	secret (optional)	Zoho Cliq webhook for notifications
DIGEST_HOUR	secret (optional)	Hour of day to send the digest/report
SMTP_HOST	secret	Outbound email server host
SMTP_PORT	secret	Outbound email server port
SMTP_USER	secret	SMTP authentication username
SMTP_PASS	secret	SMTP authentication password
MAIL_FROM	secret	"From" address on outgoing email
APP_URL	secret	Public app origin used to build links in emails (verify / reset / invite); must be the service URL, not localhost
PLATFORM_ADMIN_EMAILS	secret	Comma-separated emails granted PLATFORM (super) admin, overseeing all organizations; synced on login

From README.md (server .env)

Variable	Purpose
ANTHROPIC_API_KEY	Enables real Claude AI extraction; when set, the engine pill switches to "claude"
ANTHROPIC_MODEL	Claude model id, documented as <code>claude-opus-4-8</code>
TRANSCRIPTION_PROVIDER	Pluggable extension point for audio transcription (paste text transcripts until wired)

Referenced in the docs guides (not in render.yaml)

Variable	Where	Purpose
GOOGLE_CLIENT_ID	server/.env	Web OAuth client id; verifies Google ID tokens server-side
GOOGLE_CLIENT_IDS_EXTRA	server/.env	Comma-separated Android/iOS client ids accepted in the token aud claim
VITE_GOOGLE_CLIENT_ID	client/.env*	Web client id used to render the Google button
VITE_API_BASE	client/.env.production	Backend base URL the web app / APK talks to
VITE_WAKEWORD_ENABLED	client/.env*	Enables the "hey BTM" wake word
VITE_WAKEWORD_THRESHOLD	client/.env*	Wake-word sensitivity (default 0.5)
VITE_ORT_WASM_PATH	client/.env*	Local path for onnxruntime-web WASM files (offline use)

(The voice-assistant troubleshooting section also mentions `VITE_PORCUPINE_ACCESS_KEY`, an apparent leftover from a prior Porcupine-based wake-word implementation; the active implementation uses `openWakeWord`.)

Part II — Backend (server/)

7. Server Core Modules

server/package.json

This is the npm manifest for the SmartTask AI backend, an Express + SQLite service described as a "meeting-to-task backend." It declares the project as an ES module (`"type": "module"`), keeps it private, and defines four scripts: `start` runs the server with

node src/index.js, dev runs it with node --watch for auto-restart on file changes, seed executes the seed script directly, and rag:index runs the RAG backfill utility. Its runtime dependencies are bcryptjs (password hashing), better-sqlite3 (the synchronous SQLite driver), cors (cross-origin middleware), dotenv (loading .env), express (the web framework), google-auth-library (Google sign-in verification), jsonwebtoken (JWT auth and FCM OAuth signing), multer (multipart/file uploads), nanoid (short id generation), nodemailer (SMTP email), ws (WebSockets for live transcription and chat), and xlsx (spreadsheet import/export, pulled from the SheetJS CDN tarball).

src/index.js

This is the application entry point that wires the whole server together: it loads environment variables, restores/initializes the database, mounts every REST route, serves the built web client, and starts background jobs and WebSocket endpoints. The import order is deliberate and load-bearing — ./restore.js is imported before ./db.js so a committed database snapshot can be copied onto the persistent disk before SQLite opens the file. It reads a wide range of environment variables indirectly (through the modules it imports) and directly reads OPENROUTER_API_KEY, OPENROUTER_MODEL, ANTHROPIC_API_KEY, OPENAI_API_KEY, TRANSCRIPTION_PROVIDER, RAG_SYNC_MINUTES, and PORT.

Startup sequence: After imports, it calls initSchema() to create/migrate all tables, then ensureSeed() to populate an empty database with demo data. It then constructs the Express app.

Middleware: cors() enables cross-origin requests for all routes; express.json({ limit: '5mb' }) parses JSON request bodies up to 5 MB. A custom middleware mounted on /api sets Cache-Control: no-store, no-cache, must-revalidate and Vary: Authorization on every API response so one user's per-account data (e.g. notifications) can never be served to another from a cache. A final error-handling middleware (four-argument signature) logs any thrown error with an [error] prefix and responds with HTTP 500 and a JSON { error } body.

Health route: GET /api/health returns a JSON object reporting ok: true, which AI engine is active (OpenRouter, Claude, OpenAI, or offline rule-based, chosen by which API key is set), the configured transcription provider, and a rag object indicating whether embeddings are enabled, the embedding model name, and the count of rows in the embeddings table.

Route mounting: Twelve route modules are mounted under /api prefixes: auth → /api/auth, invites → /api/invites, users → /api/users, meetings → /api/meetings, tasks → /api/tasks, dashboards → /api/dashboards, assistant → /api/assistant, notifications → /api/notifications, digest → /api/digest, chat → /api/chat, platform → /api/platform, and usage → /api/usage.

Static web client: It resolves ../../client/dist relative to this file. If index.html exists there, it serves that directory as static files and adds an SPA fallback: a regex route matching any non-/api/ GET request returns index.html, so client-side (BrowserRouter) routes survive a page refresh while unknown /api/* paths still return a proper JSON 404. If the build is absent it logs an API-only message and skips static serving.

Server startup: It listens on PORT (default 4000), logging the URL, active AI engine, RAG status with indexed count, and the number of users in the database. Inside the listen callback it calls startScheduler() to begin the daily-digest timer.

Background jobs: If embeddings are configured (`hasEmbeddings()`), it runs a one-time `syncAll()` on boot to index anything created while embeddings were off, logging embedded/pruned counts and swallowing errors. It then sets up a recurring `setInterval` that re-runs `syncAll()` every `RAG_SYNC_MINUTES` (default 5) minutes to keep the vector index current and prune orphans; the timer is `unref()`-ed so it never keeps the process alive on its own. Two WebSocket servers are attached to the same HTTP server: `attachLiveTranscribe(server)` for live meeting transcription and `attachChatHub(server)` for real-time chat. Finally, a shutdown handler closes the server and exits on `SIGINT/SIGTERM`, and a one-shot `SIGUSR2` handler closes and re-signals for clean `nodemon/--watch` restarts, preventing `EADDRINUSE` from an orphaned process squatting on the port.

src/db.js

This module opens the SQLite database with `better-sqlite3`, enables WAL journaling and foreign keys, exports the shared db connection, and defines the schema plus a small migration framework. It computes the database path as `../data/smarttask.db` relative to the file, creates the data directory if missing, and enables `journal_mode = WAL` and `foreign_keys = ON` pragmas. It reads no environment variables.

Database tables:

- **organizations** — `id` (PK), `name`, `created_at`; migration-added `is_personal` (INTEGER, 0 = company, 1 = solo/personal account), `allowed_domains` (TEXT, comma-separated allowed email domains, NULL = unrestricted), `usage_access` (INTEGER, lets an org's own admins view usage).
- **departments** — `id` (PK), `org_id` (FK organizations), `name`.
- **users** — `id` (PK), `org_id` (FK), `department_id` (FK departments), `name`, `email` (unique), `password_hash`, `role` (CHECK in admin/manager/employee), `phone` (default ""), `aliases` (comma-separated spoken-name aliases for AI matching), `preferred_language` (default 'en'), `avatar_color` (default '#6366f1'), `created_at`; migration-added `platform_admin` (INTEGER, super-admin flag), `email_verified` (INTEGER), `google_id` (TEXT, linked Google "sub"), `last_seen` (TEXT, updated on last socket disconnect), `avatar_file` (uploaded profile photo).
- **projects** — `id` (PK), `org_id` (FK), `name`, `department_id`, `created_at`.
- **meetings** — `id` (PK), `org_id` (FK), `title`, `meeting_date`, `uploaded_by`, `source_type` (transcript | audio), `audio_filename`, `raw_transcript`, `detected_languages` (JSON array), `status` (uploaded | processing | processed | failed), `summary_json`, `engine` (claude | rule-based), `created_at`; migration-added `description`.
- **transcript_segments** — `id` (PK), `meeting_id` (FK, cascade delete), `seq` (INTEGER order), `speaker`, `text`, `language`.
- **tasks** — `id` (PK), `org_id` (FK), `title`, `description`, `assignee_id` (FK users), `assignee_name_raw` (spoken name before matching), `assigned_by_id`, `assigned_by_name_raw`, `due_date`, `due_date_raw` (original natural-language phrase), `priority` (CHECK Critical/High/Medium/Low, default Medium), `status` (To Do | In Progress | Blocked | In Review | Done | Reopened), `project_id`, `department_id`, `meeting_id` (origin meeting), `ownership_confidence` (high | low | needs_confirmation), `parent_task_id` (FK tasks, for subtasks, cascade), `progress` (0–100), `approval_status` (none | pending | approved | rejected), `source_quote` (exact transcript line), `assigned_at`, `submitted_at`, `completed_at`, `visible_to_manager` (INTEGER, 0 = private employee draft), `created_at`, `updated_at`.
- **task_dependencies** — composite PK (`task_id`, `depends_on_task_id`), both FKs to tasks with cascade delete.

- **task_comments** — id (PK), task_id (FK cascade), user_id (FK users), body, created_at.
- **attachments** — id (PK), task_id (FK cascade), filename, uploaded_by, created_at.
- **audit_logs** — id (PK), org_id, actor_id, action, entity_type, entity_id, detail, created_at.
- **usage_events** — id (PK), org_id, user_id, provider (sarvam | openrouter | anthropic | openai | groq), feature (transcription | assistant | voice_search | voice_task | meeting_analysis), model, input_tokens, output_tokens, total_tokens, cost_usd (REAL), created_at; indexed by (org_id, created_at).
- **notifications** — id (PK), org_id, user_id (recipient, FK cascade), type (task_submitted | task_approved | task_reopened | task_assigned), message, task_id, read (INTEGER default 0), created_at; indexed by (user_id, read).
- **device_tokens** — id (PK), user_id (FK cascade), token (unique FCM token), platform (default 'android'), created_at; indexed by user_id.
- **app_meta** — key (PK), value; a generic key/value store used for run-once migration markers and scheduler state.
- **embeddings** — RAG vector store: id (PK), org_id, source_type (task | meeting | segment | chat), source_id (source row id), ref_user_id (task assignee, employee-scope filter), ref_convo_id (chat conversation membership filter), chunk_text (the embedded text), content_hash (skip re-embed if unchanged), dim (vector dimensions), vector (Float32 BLOB), model, updated_at; UNIQUE (source_type, source_id), indexed by (org_id, source_type).
- **chat_messages** — id (PK), org_id, conversation_id, sender_id (FK cascade), recipient_id (legacy, nullable), body, file_name, file_stored, file_type, file_size, deleted_for_all (INTEGER), reply_to, edited_at, read, created_at; migration-added forwarded (INTEGER); indexed by (sender_id, recipient_id, created_at) and (conversation_id, created_at).
- **chat_message_hidden** — "delete for me": composite PK (message_id, user_id), both FKs cascade; hides a message from one user's view only.
- **chat_conversations** — id (PK), org_id, type (direct | group), name (group name, null for direct), avatar_color, created_by, created_at, updated_at; migration-added avatar_file (uploaded group photo).
- **chat_participants** — composite PK (conversation_id, user_id), both FKs cascade; role (admin | member), last_read_at (drives unread counts/read receipts), joined_at; migration-added muted and pinned (per-user flags); indexed by user_id.
- **chat_reactions** — composite PK (message_id, user_id), emoji, created_at; one reaction row per user per message.
- **chat_stars** — composite PK (message_id, user_id), created_at; per-user starred/bookmarked messages.
- **conversations** — AI Assistant chat history: id (PK), org_id, user_id (FK cascade), title (default 'New chat'), messages (JSON array of {role, text, tasks?}, default '[]'), created_at, updated_at; indexed by (user_id, updated_at).
- **meeting_participants** — composite PK (meeting_id, user_id), both FKs cascade; the attendees eligible to be suggested as task owners.
- **suggested_tasks** — AI review queue: id (PK), meeting_id (FK cascade), org_id, title, description, suggested_assignee_id (FK users), suggested_assignee_raw (spoken name), assignee_reasoning, confidence (0–100, default 50), priority (default Medium), due_date, due_date_raw, source_quote, status (pending | approved | rejected | merged), merged_into, created_task_id, created_at, updated_at; indexed by (meeting_id, status).
- **invites** — id (PK), org_id (FK), email, role, department_id, token (unique), invited_by, status (pending | accepted | revoked), created_at, accepted_at, expires_at; indexed by (org_id, status).

- **password_resets** — token (PK), user_id (FK cascade), expires_at, used (INTEGER), created_at; single-use expiring reset tokens.
- **email_verifications** — token (PK), user_id (FK cascade), expires_at, used (INTEGER), created_at; single-use expiring email-confirmation tokens.

Functions:

- **initSchema()** — no parameters, returns nothing. Executes one large CREATE TABLE IF NOT EXISTS batch defining every table and index above, then runs a series of lightweight migrations. It calls ensureColumn(...) for each column added after the original schema (e.g. organizations.is_personal, users.platform_admin, chat file columns), creates the idx_chat_convo index after its column exists, and calls runOnce(...) for four multi-step migrations: rebuilding chat_messages to make recipient_id nullable for group support, backfilling legacy 1:1 messages into conversations, removing legacy standalone admin accounts, and renaming/ensuring the IT/Marketing/Sales/Management department set. This is the single function that brings any database (new or old) to the current schema.
- **runOnce(key, fn)** — parameters: a string key and a callback fn; returns nothing. It checks app_meta for the key; if present it returns immediately, otherwise it runs fn() and records the key with value '1' so that migration body never runs again. This gives idempotent, once-only migrations tracked in the database itself.
- **ensureColumn(table, col, def)** — parameters: table name, column name, and SQL column definition; returns nothing. It reads PRAGMA table_info(table) and, if the column isn't already present, issues ALTER TABLE ... ADD COLUMN. This works around SQLite's lack of ADD COLUMN IF NOT EXISTS, letting older databases gain new columns safely.
- Inline within initSchema, the **chat_messages_rebuild_v2** runOnce body detects a non-nullable recipient_id with a FK, turns off foreign keys, and in a transaction creates a new table, copies all rows, drops the old table, renames the new one, and recreates indexes — allowing group messages that have no single recipient. The **chat_conversations_v1** body groups legacy messages by sorted sender/recipient pair, creates a direct conversation and two participant rows per pair, and stamps each message's conversation_id. The **remove_standalone_admin_v1** body nulls tasks and deletes comments referencing any admin user then deletes those users. The **departments_...v1** body renames Engineering/QA/DevOps to IT/Marketing/Sales and ensures all four departments exist per org.

src/auth.js

This module centralizes authentication and authorization: password hashing, JWT signing/verification, and Express middleware for requiring a login, a specific role, or platform-admin status. It reads JWT_SECRET (defaulting to a dev placeholder), JWT_EXPIRES_IN (default '30d'), and PLATFORM_ADMIN_EMAILS.

- **hashPassword(pw)** — parameter: plaintext password; returns a bcrypt hash string. It calls bcrypt.hashSync with a cost factor of 10 to produce a salted hash suitable for storage in users.password_hash.
- **verifyPassword(pw, hash)** — parameters: plaintext password and stored hash; returns a boolean. It calls bcrypt.compareSync to check whether the password matches the hash, used during login.
- **signToken(user)** — parameter: a user row; returns a signed JWT string. It embeds sub (user id), role, org_id, and name as claims, signs them with the secret, and applies the configured expiry. This token is what clients send back on subsequent requests.

- **verifyToken(token)** — parameter: a raw JWT string (e.g. from a WebSocket query param); returns the matching user row or null. It verifies the token signature/expiry and looks up the user by the sub claim; any failure (missing, invalid, expired, or unknown user) yields null. It exists because WebSocket handshakes can't carry Authorization headers.
- **authRequired(req, res, next)** — Express middleware. It reads the Authorization: Bearer header, responds 401 if absent, otherwise verifies the token and loads the user. On success it attaches req.user and calls next(); on an invalid/expired token or unknown user it responds 401 with a JSON error.
- **requireRole(...roles)** — parameter: one or more allowed role strings; returns an Express middleware. The returned middleware responds 403 unless req.user exists and its role is in the allowed set, otherwise calls next(). Usage is e.g. requireRole('admin', 'manager').
- **requirePlatformAdmin(req, res, next)** — Express middleware. It responds 403 unless req.user.platform_admin is truthy, otherwise calls next(). This is the single deliberate exception to per-org isolation, gating cross-org super-admin routes.
- **platformAdminEmails()** — no parameters; returns an array of lowercased, trimmed email strings from the PLATFORM_ADMIN_EMAILS env var (comma-separated, empties filtered out). Because it's operator-controlled via env, super-admin status can't be self-granted inside the app.

src/util.js

This is a grab-bag of shared helpers: id/token generation, date math, URL building, password/domain validation, audit logging, and the notification-plus-push helper. It reads APP_URL (default http://localhost:5173) and imports db and sendPushToUser.

- **id(prefix = '')** — parameter: optional prefix; returns a string id. It uses a 12-character nanoid over lowercase alphanumerics and prepends prefix_ when a prefix is given (e.g. id('usr') → usr_ab12...).
- **now()** — no parameters; returns the current time as an ISO 8601 string, used for all created_at/updated_at stamps.
- **genToken()** — no parameters; returns a 43-character URL-safe base64url string from 32 random bytes. It's used for higher-entropy invite, password-reset, and email-verification links, stronger than an id().
- **appUrl()** — no parameters; returns the front-end base URL from APP_URL (or the local default) with any trailing slash stripped, so email links can be built consistently.
- **inDays(days)** — parameter: number of days; returns an ISO timestamp that many days in the future, used for token expiry.
- **dueDateForPriority(priority, from = new Date())** — parameters: a priority string and an optional base date; returns a YYYY-MM-DD string in the server's local timezone. It looks up a per-priority lead time (Critical = 0 days, High = 1, Medium = 3, Low = 5, defaulting to Medium), adds it to the base date, and formats using local date parts so "same day" is correct for IST evenings. This auto-fills a task's due date when none was specified.
- **isCommonPassword(pw)** — parameter: a candidate password; returns a boolean. It lowercases the input and checks membership in a hardcoded Set of common/breached passwords, providing server-side defense-in-depth against weak passwords that the client also blocks.
- **orgAllowedDomains(orgId)** — parameter: an org id; returns an array of lowercased allowed email domains. It reads the org's allowed_domains column and splits it on commas (empty = no restriction), used to enforce each org's own domain policy on user creation, imports, and invites.

- **emailDomainAllowed(orgId, email)** — parameters: org id and email; returns a boolean. If the org configured no domains it returns true; otherwise it extracts the domain after @ and returns whether it's in the allowed list.
- **audit(orgId, actorId, action, entityType, entityId, detail = '')** — parameters as named; returns nothing. It inserts one row into `audit_logs` with a generated id and current timestamp, serializing the `detail` to JSON when it isn't already a string. This records who did what to which entity.
- **notify(orgId, userId, type, message, taskId = null)** — parameters as named; returns nothing. It inserts an unread in-app notifications row for the recipient, then fires a fire-and-forget native push via `sendPushToUser` using a human title looked up from a `PUSH_TITLES` map (falling back to 'SmartTask'), swallowing any push error so a slow/failed send never blocks the request. It no-ops if `userId` is falsy.
- **notifyManagers(orgId, type, message, taskId = null, excludeId = null)** — parameters as named; returns nothing. It selects every manager/admin in the org and calls `notify(...)` for each, skipping the optional `excludeId` (typically the actor). This broadcasts events like task submissions to all supervisors.

src/mailer.js

This module abstracts outbound email so the rest of the app can call one function regardless of whether SMTP credentials exist. When SMTP is configured it sends real mail via nodemailer; otherwise it runs in "preview" mode and prints the email to the console so flows remain testable without credentials. It reads `SMTP_HOST`, `SMTP_PORT` (default 587, secure only when 465), `SMTP_USER`, `SMTP_PASS`, and `MAIL_FROM`. Module-level state holds a lazily-created transporter and a mode string.

- **init()** — no parameters; returns nothing. It lazily builds the nodemailer transporter the first time it's needed: if `SMTP_HOST`, `SMTP_USER`, and `SMTP_PASS` are all present it creates an SMTP transport (choosing TLS based on port) and sets mode to 'smtp'; otherwise it leaves the app in preview mode. It returns early if already initialized.
- **mailerMode()** — no parameters; returns the current mode string ('smtp' or 'preview'). It calls `init()` first so callers (like the digest) can branch on whether real email will be sent.
- **sendMail({ to, subject, text, html })** — parameter: a destructured mail object; returns a promise resolving to { `sent`, `mode` }. It ensures `init`, then in SMTP mode sends via the transporter using `MAIL_FROM` (falling back to `SMTP_USER`) and returns { `sent`: true, `mode`: 'smtp' }; in preview mode it logs the recipient, subject, and text between decorative separators and returns { `sent`: false, `mode`: 'preview' }.

src/push.js

This module implements native push notifications through Firebase Cloud Messaging's HTTP v1 API without the heavy `firebase-admin` SDK — it mints a short-lived OAuth token from a service account using `jsonwebtoken` and posts via plain `fetch`. If credentials are missing or invalid, every function degrades to a safe no-op so the app runs fine without push. It reads the `FIREBASE_SERVICE_ACCOUNT` env var (the full service-account JSON). Module state caches the parsed service account and the OAuth token.

- **getServiceAccount()** — no parameters; returns the parsed service-account object or null. On first call it parses `FIREBASE_SERVICE_ACCOUNT`, validating that `client_email`, `private_key`, and `project_id` are present, logs whether push is on or off, and caches the result (including a null on failure) so parsing happens only once.

- **getAccessToken()** — no parameters; returns a promise resolving to an access-token string or null. If a cached token is still valid (more than 60 s from expiry) it reuses it; otherwise it builds and RS256-signs a JWT assertion scoped to `firebase.messaging`, POSTs it to Google's OAuth token endpoint, and caches the returned `access_token` with its expiry. It tolerates double-escaped newlines in the private key and throws on a non-OK OAuth response.
- **saveDeviceToken(userId, token, platform = 'android')** — parameters as named; returns nothing. It first deletes any existing row for that token (so a token migrating between users/devices always re-binds to the latest user and pushes never leak to a previous account), then inserts a fresh `device_tokens` row with a time-plus-random id. It no-ops if `userId` or `token` is missing.
- **removeDeviceToken(token)** — parameter: a token; returns nothing. It deletes the matching `device_tokens` row (used on logout/unregister), no-op if token is falsy.
- **sendPushToUser(userId, { title, body, data })** — parameters: a user id and a destructured payload; returns a promise. It fetches the service account and the user's device tokens (returning early if none), obtains an access token, stringifies all data values (FCM requires string values), and POSTs one FCM v1 message per token in parallel with high-priority Android notification options. On a 404/NOT_FOUND/UNREGISTERED/INVALID_ARGUMENT response it prunes the dead token from the database; other failures are logged. It never throws, matching its fire-and-forget contract.

src/scheduler.js

This is a minimal in-process daily scheduler with no external cron dependency: it ticks every minute and, at a configured hour, runs the daily digest exactly once per day, guarded by a marker in `app_meta` so a restart can't double-send. It reads `DIGEST_HOUR` (default 8, in local server time).

- **getMeta(key)** — parameter: a key; returns the stored string value or null. It reads a single `app_meta` row, used to check when the digest was last sent.
- **setMeta(key, value)** — parameters: key and value; returns nothing. It upserts an `app_meta` row via `INSERT ... ON CONFLICT DO UPDATE`, used to record the last-sent date.
- **tick()** — no parameters; async, returns nothing. Called every minute, it checks whether the current local hour equals `SEND_HOUR` and the stored `digest_last_sent` is not today's date; if so it first stamps today's date (so a crash mid-send won't loop), then awaits `sendDailyDigests()`, logging any error.
- **startScheduler()** — no parameters; returns nothing. It logs the scheduled hour and starts a `setInterval` calling `tick()` every 60 seconds. This is invoked once from `index.js` at server startup.

src/digest.js

This module builds and sends the daily task digest: its primary channel is a per-organization Zoho Cliq team summary, and it additionally sends per-person email when SMTP is configured, both falling back to console preview. It imports `db`, the mailer helpers, and the Cliq helpers. It reads no environment variables directly (delegating to the mailer and Cliq modules). It defines a module-local OPEN SQL fragment listing the open task statuses and a `today()` helper for the current date string.

- **today()** — no parameters; returns today's date as a YYYY-MM-DD string, used throughout for overdue comparisons and headings.

- **fmtTask(t)** — parameter: a task row; returns a single formatted bullet line. It renders priority, title, and status, and appends a due-date suffix marked with an overdue warning when the due date is before today.
- **buildTeamSummary(orgId)** — parameter: an org id; returns a multi-line string. It selects all employees and managers in the org, and for each lists their open, manager-visible, top-level tasks (up to 8, with an "...and N more" line), tallying total open and overdue counts. It heads the message with the date and finishes with either a celebratory "No open tasks" line or a team-total summary — this is the single standup message posted to Cliq.
- **buildDigest(user)** — parameter: a user row; returns { subject, text } or null. It selects that user's open top-level tasks; for employees with zero tasks it returns null (skip emailing them), otherwise it composes a personalized greeting, a count, one line per task, and a signature. This is used only for the per-person SMTP emails.
- **sendDailyDigests()** — no parameters; async, returns a summary object. It gathers the distinct org ids across all users and posts a Cliq team summary for each, then, only if the mailer is in SMTP mode, emails every user with a non-empty address whose buildDigest is non-null. It logs and returns a summary of Cliq mode, message count, email count, and email mode. This is what the scheduler invokes daily and what the digest route can trigger manually.

src/seed.js

This module populates a brand-new (empty) database with a demo organization, departments, users, projects, and two processed demo meetings, so a fresh install has realistic data to explore. It imports db, id/time helpers, hashPassword, the offline analyzeTranscript rules engine, and persistMeeting from the meetings route. It reads no environment variables; the demo password is the hardcoded 'password123'. Two multilingual demo transcript constants (MEETING_1, MEETING_2, mixing English, Hindi, and Telugu) drive the seeded meetings.

- **daysAgo(n)** — parameter: number of days; returns a YYYY-MM-DD string that many days in the past, used to date the demo meetings.
- **ensureSeed()** — no parameters; returns nothing. It returns immediately if the users table is non-empty. Otherwise it creates an org ("Befach Technologies"), four departments (IT, Marketing, Sales, Management), five users (one manager who is also the org admin, plus four employees with aliases, languages, and avatar colors) all sharing the demo password hash, and two projects. It then defines an inner seedMeeting closure that runs a transcript through analyzeTranscript and persists it via persistMeeting with autoApprove: true (so AI suggestions become real assigned tasks immediately), processes both demo meetings with all users as participants, tags the first four created tasks onto projects with varied statuses/progress to make dashboards look alive, and logs the seed result plus demo login credentials.

src/restore.js

This is a one-time data-restore module with no exported functions — its work runs as an import side effect, which is why index.js imports it before db.js. On the Render persistent disk the live database started empty (only ever seeded), so real org data created locally wasn't present; this module copies a committed snapshot (server/restore/smarttask.db, which ships with the git checkout outside the disk mount) onto the disk the first time this version boots. It reads no environment variables. It ensures the data directory exists, and if the snapshot file exists and a .imported-v1

marker on the disk does not, it deletes the live database along with its `-wal` and `-shm` sidecar files (so SQLite can't replay a stale journal onto the fresh copy and corrupt it), copies the snapshot into place, and writes the marker with the current timestamp so later redeployments never clobber data created afterward. The whole thing is wrapped in a `try/catch` that only warns on failure, so a restore problem never blocks startup — the app still boots on whatever data exists. (Bumping the marker suffix would force a re-import.)

src/cliq.js

This small module posts messages to a Zoho Cliq channel through an Incoming Webhook URL, or prints them to the console in preview mode when no webhook is configured. It reads `CLIQ_WEBHOOK_URL`.

- **cliqEnabled()** — no parameters; returns a boolean indicating whether `CLIQ_WEBHOOK_URL` is set, letting callers report the real channel mode versus preview.
- **postToCliq(text)** — parameter: the message text; async, returns `{ sent, mode }`. If no webhook URL is set it logs the message between preview separators and returns `{ sent: false, mode: 'preview' }`; otherwise it POSTs a JSON `{ text }` body (the shape Zoho Cliq expects), throws with the status and truncated body on a non-OK response, and returns `{ sent: true, mode: 'cliq' }` on success.

src/ragBackfill.js

This is a standalone CLI script (run via `npm run rag:index`) that embeds every existing task, meeting, transcript segment, and chat message into the embeddings table, whether for an initial index or a repeatable catch-up. It has no exported functions — it executes top-level. It loads `.env`, calls `initSchema()` to ensure tables exist, and checks `hasEmbeddings()`; if no embedding provider is configured it prints an error telling the operator to set `OPENAI_API_KEY` or `VOYAGE_API_KEY` and exits with code 1. Otherwise it logs the embedding model, times a call to `backfillAll()`, prints how many chunks were embedded versus skipped (unchanged), and exits 0. It effectively reads `OPENAI_API_KEY/VOYAGE_API_KEY` indirectly through the embeddings module.

src/ws/chatHub.js

This module is the real-time chat hub: a WebSocket server that pushes new direct/group messages, presence changes, and typing indicators to connected clients instantly, so the Chats page doesn't wait for the next poll. Messages are still sent over the REST API for reliable persistence; this socket is push-only for delivery. It maintains a module-level `clients` map from user id to a `Set` of that user's live sockets (supporting multiple tabs/devices). It reads no environment variables and authenticates via a token query parameter.

- **attachChatHub(server)** — parameter: the shared HTTP server; returns the `WebSocketServer`. It creates a `noServer` WS server and registers an upgrade handler that only claims the `/api/chat/ws` path (so it can coexist with the transcription socket). On each connection it verifies the token, closing with code 4401 if unauthorized; otherwise it registers the socket under the user, sends a ready message and a `presence-list` of currently-online users, and — if this was the user's first socket — broadcasts a `presence online` event to everyone. It wires an inbound message handler that relays ephemeral typing signals only to the other participants of a conversation the sender actually belongs to (verified against `chat_participants`),

sets up heartbeat bookkeeping (`isAlive/pong`), and a detach cleanup on `close/error` that removes the socket, and when the user's last socket goes away stamps `users.last_seen` and broadcasts an offline presence event. It also starts a 30-second heartbeat interval that terminates any socket that failed to pong and pings the rest, clearing the interval when the server closes.

- **`pushToUser(userId, payload)`** — parameters: a user id and a JSON-serializable payload; returns nothing. It looks up the user's socket set and sends the serialized payload to every open socket, no-op if the user is offline (they'll see the message on next load/poll). This is the exported entry point other modules (e.g. the chat route) use to deliver new messages.
- **`broadcastAll(payload)`** — parameter: a payload; returns nothing. It serializes once and sends to every open socket of every connected user, used for presence changes.
- **`getOnlineUsers()`** — no parameters; returns an array of user ids that currently have at least one live socket, used to build the initial presence list.

src/ws/liveTranscribe.js

This module is a live-transcription WebSocket proxy that sits between the browser and Sarvam's speech-to-text service: the browser streams PCM16 audio to this server, which relays it to Sarvam and relays transcripts back. The proxy exists because the browser can't attach Sarvam's `Api-Subscription-Key` header to a WebSocket handshake and the key must never reach the client. It reads `SARVAM_LANGUAGE` (default `en-IN`), `TRANSCRIPTION_PROVIDER`, `SARVAM_API_KEY`, and `SARVAM_MODEL` (default `saarika:v2.5`), and authenticates via a token query parameter.

- **`attachLiveTranscribe(server)`** — parameter: the shared HTTP server; returns the `WebSocketServer`. It creates a `noServer` WS server and an upgrade handler that only claims the `/api/meetings/live` path so it coexists with the chat hub. On connection it determines the locked language (from the requested `en/hi/te` or the `SARVAM_LANGUAGE` default, so audio is never mis-transcribed across Indian languages), then enforces two gates: it verifies the token and requires a manager/admin role (closing 4401 otherwise), and it requires `TRANSCRIPTION_PROVIDER=sarvam` with a `SARVAM_API_KEY` present (closing 4400 otherwise). It then opens an upstream Sarvam socket with the language/model/sample-rate query parameters and the subscription-key header. Inbound client audio frames (base64 PCM) are wrapped into Sarvam's `{ audio: { ... } }` JSON and forwarded, buffering any frames that arrive before the upstream socket opens and flushing them on open. Sarvam's data messages are relayed to the client as `{ transcript }` and its error messages as `{ error }`; upstream errors/closes close the client socket, and client `close/error` closes the upstream socket. A small `sendToSarvam` helper manages the pending-frame buffer and a `toClient` helper guards against sending on a non-open socket.

8. API Routes (server/src/routes/)

routes/assistant.js

This router powers the AI assistant and hands-free voice-control features. It is mounted behind `authRequired` (every route requires a valid bearer token), and it stores per-user chat conversation threads server-side so they sync across devices. It bridges the HTTP layer to the AI modules (`answerQuery`, `chatAnswer`, `interpretCommand`, `resolveUser`,

parseDueDate) and records AI usage against the caller's org. When a real LLM is configured (hasLLM()), it prefers the LLM path and gracefully falls back to an offline rule-based engine when no key is set or the provider call throws.

Helper functions:

- rowToConvo(row) — maps a conversations DB row into { id, title, msgs (parsed JSON), updated (epoch ms) }.
- hydrate(tasks) — projects an array of task objects down to a compact shape (id, title, status, priority, due_date, assignee_name, project_name) for assistant responses.
- commandScopedTasks(user) — queries all top-level tasks in the user's org (joined to the assignee's name); for employee role it filters to only tasks assigned to that user.
- navUrl(nav, user) — translates a voice "navigate" intent target into a client URL, or returns { deny } / null. Handles targets overdue, completed, active, all/my_tasks, dashboard, status, priority, and person (person navigation is denied for employees and resolves the named user via resolveUser).
- clarify(say) — returns a { mode: 'clarify', say } object with a default fallback prompt.

Routes:

- GET /conversations — Auth: any logged-in user. No params. Returns all of the caller's conversation threads (WHERE user_id), newest first by updated_at. Responds 200 with { conversations: [rowToConvo...] }.
- POST /conversations — Auth: any user. Body: { title?, msgs? }. Creates a new conversation row with a generated conv id, org and user stamped, title truncated to 80 chars (defaults to "New chat"), and messages JSON-encoded (defaults to []). Returns 200 with the created conversation in rowToConvo shape.
- PUT /conversations/:id — Auth: any user; ownership enforced by user_id. Body: { title?, msgs? }. Upserts: if no conversation with that id owned by the caller exists, it inserts one with the given id; otherwise it updates the title (falling back to the existing title) and messages. Title truncated to 80. Returns 200 with the resulting conversation.
- DELETE /conversations/:id — Auth: any user; scoped to id AND user_id. Deletes the conversation. Returns 200 { ok: true }.
- POST /query — Auth: any user. Body: { query, history? }. Returns 400 { error: 'query required' } if query is missing. If an LLM is configured, calls chatAnswer(query, user, history, onUsage) (recording usage under feature assistant) and returns its result with tasks hydrated; on any LLM error it logs a warning and falls through. The fallback path calls the rule-based answerQuery(query, user) and returns its result with hydrated tasks plus engine: 'rule-based'. Response shape is whatever the engine returns spread with a tasks array; status 200.
- POST /command — Auth: any user. Body: { transcript, history? }. This is the voice-command brain. Returns 400 if transcript is empty. If no LLM is configured, returns 200 { mode: 'answer', say: 'Voice control needs an AI engine...' }. Otherwise it gathers org-scoped tasks and non-admin users (with aliases) and calls interpretCommand, recording usage under feature voice_command; interpret failures return a clarify message. It then dispatches on intent.intent:

- `create_task` — requires a title (else clarifies); resolves the named assignee via `resolveUser`, defaults priority to Medium, parses `due_date_raw` to an absolute date, and returns mode: `'confirm'` with a spoken confirmation and an action of kind `create_task` carrying the task body.
 - `update_status` — finds the task, clarifies if missing or if no status; returns mode: `'confirm'` with an `update_status` action.
 - `assign_task` — finds the task and resolves the assignee; clarifies if either is unresolved; returns a confirm with an `assign_task` action.
 - `set_priority` — finds the task, clarifies if no priority; returns a confirm with a `set_priority` action.
 - `set_due_date` — finds the task, parses the spoken date; clarifies if unresolved; returns a confirm with a `set_due_date` action.
 - `navigate` — builds a URL via `navUrl`; returns mode: `'answer'` with a deny message, a clarify, or mode: `'navigate'` with the target URL.
 - `answer` — returns mode: `'answer'` with the spoken answer.
 - `default` — returns a clarify. All mutations come back as mode: `'confirm'` so the client can ask a yes/no before committing. Status 200 throughout (except the 400 validation).
- `GET /suggestions` — Auth: any user. No params. Returns a base list of suggested prompts; for non-employee roles it appends team/workload-oriented prompts. Responds 200 { suggestions: [...] }.
-

routes/auth.js

Handles authentication and account lifecycle: self-serve signup (which provisions a whole organization), password login, Google Sign-In (login only), the current-user endpoint, email verification, and password reset. It is not globally guarded — most routes are public — with `authRequired` applied per-route where needed (`/me`, `/resend-verification`). It exports the router plus the `publicUser` helper for reuse by other routers.

Helper functions:

- `userWithWorkspace(user)` — wraps `publicUser` and adds `workspace_personal` (1 if the org is personal/solo) and `platform_admin` flags so the client can tailor UI.
- `sendVerificationEmail(user)` — generates a token, inserts an `email_verifications` row (7-day expiry), and emails a verification link via `sendMail`.
- `publicUser(u)` (exported) — the safe user projection: `id`, `name`, `email`, `role`, `org_id`, `phone`, `department_id`, `preferred_language`, `avatar_color`, `avatar_file`, `email_verified`.

Module constants: `GOOGLE_CLIENT_ID`, `GOOGLE_AUDIENCES` (accepted aud values incl. mobile client ids), `googleClient`, `DEFAULT_DEPARTMENTS` (IT, Marketing, Sales, Management), `EMAIL_RE`.

Routes:

- `POST /signup` — Auth: none. Body: { `personal?`, `name`, `email`, `password`, `company?` }. For personal (solo) workspaces the company name is auto-derived from the first name; company signups require company. Validates required fields, email format, password length ≥ 8 , and rejects common passwords. Rejects duplicate

emails with 409. In one atomic transaction it creates the organization (seeding allowed_domains from the founder's email domain for company mode), the four default departments, and the first user as role manager (linked to the Management department). Writes an audit event and fires a verification email (non-blocking). Returns 201 { token, user: userWithWorkspace }. Errors: 400 (validation), 409 (email taken / unique race), 500 (creation failure).

- POST /login — Auth: none. Body: { email, password }. Returns 400 if either missing, 401 for invalid credentials. On success it syncs the user's platform_admin flag from the env allowlist (platformAdminEmails()), writes an auth.login audit event, and returns 200 { token, user: userWithWorkspace }.
 - POST /google — Auth: none. Body: { credential } (a Google ID token). Returns 501 if Google Sign-In isn't configured, 400 if no credential. Verifies the token against GOOGLE_AUDIENCES; verification failure → 401. Requires a Google-verified email (401 otherwise). Login-only: an unknown email returns 404 (no auto-provisioning). On first login it stamps google_id and marks the email verified; it also syncs platform_admin. Writes auth.login_google audit. Returns 200 { token, user: userWithWorkspace }.
 - GET /me — Auth: authRequired. Returns 200 { user: userWithWorkspace(req.user) }.
 - POST /verify-email — Auth: none. Body: { token }. Looks up the verification row; if missing, used, or expired returns 400. Otherwise, in a transaction, marks the user email_verified = 1 and the token used. Returns 200 { ok: true }.
 - POST /resend-verification — Auth: authRequired. No body. If already verified returns { ok: true, already: true }. Otherwise re-sends the verification email (errors swallowed) and returns { ok: true }.
 - POST /forgot-password — Auth: none. Body: { email }. Always returns 200 { ok: true } regardless of whether the email exists (prevents account enumeration). If the email matches a user, it creates a password_resets token (24-hour expiry) and emails a reset link.
 - POST /reset-password — Auth: none. Body: { token, password }. Validates the new password length ≥ 8 and not common (400 otherwise). Validates the token (missing/used/expired → 400; unknown user → 400). In a transaction it sets the new password hash, marks the token used, and invalidates all other outstanding reset tokens for the user. Writes an auth.password_reset audit. Returns 200 { ok: true }.
-

routes/chat.js

Implements an internal WhatsApp-style team chat: 1:1 and group conversations, file attachments (via multer, 15 MB cap, stored on disk in data/chat_uploads), real-time delivery over WebSockets (pushToUser/pushToConversation), replies, reactions, stars, edits, per-message deletes, read receipts, mute/pin, and RAG indexing of messages. The two file-serving routes are declared **before** r.use(authRequired) because they must accept a token via query string (so /<a> can load them); all other routes require auth.

Helper functions:

- `member(convId, userId)` — returns the participant row or undefined (membership check).
- `participantsOf(convId)` — array of participant user ids.
- `pushToConversation(convId, payload, exceptUserId?)` — WS-push to every participant, optionally excluding one.
- `touchConvo(convId)` — bumps `updated_at` so a conversation floats to the top.
- `reactionsByMessage(ids)` — maps message ids to arrays of { emoji, user_id }.
- `snippet(row)` — one-line preview (handles deleted tombstones and attachments with a 🗑️ label).
- `shapeMessage(row, viewerId, ctx)` — the client-facing message shape, including reply preview, file metadata, reactions, starred, and a per-viewer seen flag (true when everyone else's `last_read_at` is at/after the message).
- `findOrCreateDirect(orgId, a, b)` — returns the existing 2-person direct conversation or creates one.
- `summarizeConvo(conv, viewerId)` — the list-view summary (name/avatar resolved for direct vs group, members, unread count, last message, mute/pin, my role).
- `deliver(conv, msgRow, sender)` — touches the conversation, notifies non-muted other participants, pushes the shaped message to all participants over WS, and RAG-indexes the message.

Routes:

- `GET /file/:messageId` — Auth: token via Authorization header or `?token=`. Verifies the token; 401 if invalid. Loads the message; 404 if missing/not-a-file/deleted; 403 if the caller isn't a member of the conversation; 404 if the file is missing on disk. Streams the file with the stored content type and a Content-Disposition of attachment when `?download=1` else inline.
- `GET /conversations/:id/avatar` — Auth: token via header or `?token=`. Serves a group's photo. 401 if unverified, 404 if no avatar, 403 if not a member, 404 if the file is missing. Streams with a 5-minute private cache.

(from here authRequired applies)

- `GET /users` — Lists org users except the caller (id, name, email, role, avatar_color, avatar_file) for starting chats/adding to groups. 200 { users }.
- `GET /presence` — Returns { online: [...] }, the user ids with a live WebSocket connection.
- `GET /conversations` — Lists the caller's conversations (summarized), sorted pinned-first then by last-activity. 200 { conversations }.
- `GET /unread` — Sums unread messages across all the caller's conversations (excluding own and hidden messages). 200 { unread }.
- `POST /conversations` — Body: { type: 'direct'|'group', ... }. For direct requires `userId` (must be a different user in the same org, else 400) and finds/creates the direct conversation. For group requires a non-empty name (400) and a `memberIds` array with at least one valid org member (400); creates the group, makes the caller admin, adds members, and pushes a created event. Returns 201 with the summarized conversation. 400 if type is neither.

- GET /conversations/:id — 404 if the caller isn't a member. Loads the conversation, up to 800 non-hidden messages ascending, reactions, the caller's stars, and computes others' minimum last_read_at for seen-ticks. Marks the conversation read for the caller and pushes a read event to others. Returns 200 { conversation, messages, last_read_at (previous) }.
- POST /conversations/:id/prefs — Body: { muted?, pinned? }. 404 if not a member. Updates the per-user mute/pin flags. 200 { ok: true }.
- POST /conversations/:id/clear — 404 if not a member. Hides every current message in the conversation for the caller only (inserts into chat_message_hidden), marks read, and pushes a cleared event to the caller's other tabs. 200 { ok: true, cleared: n }.
- POST /conversations/:id/avatar — multipart file. 404 if not a member, 403 if the caller isn't the group admin, 400 if the file isn't an image or the conversation isn't a group. Replaces the stored avatar (deleting the old file) and pushes an updated event. 200 { ok: true }. On any early rejection the uploaded temp file is cleaned up.
- POST /conversations/:id/read — 404 if not a member. Lightweight mark-as-read used on live inbound; updates last_read_at and pushes a read event. 200 { ok: true }.
- PATCH /conversations/:id — Body: { name }. 404 if not a member, 403 if not the group admin, 400 if name empty. Renames the group and pushes updated. 200 { ok: true }.
- POST /conversations/:id/members — Body: { userIds: [...] }. 404/403 (admin only). Adds each valid org user as a member and pushes updated. 200 { ok: true }.
- DELETE /conversations/:id — 404 if not a member, 400 if not a group, 403 if not the group admin. Deletes all group files from disk, then in a transaction wipes reactions, stars, hidden rows, RAG embeddings, messages, participants, and the conversation. Pushes a removed event to each former member. 200 { ok: true }.
- DELETE /conversations/:id/members/:userId — 404 if the caller isn't a member. Leaving (self) is allowed; removing another member requires admin (403 otherwise). Removes the participant, pushes updated to the conversation and removed to the target. 200 { ok: true }.
- POST /conversations/:id/messages — Body: { body, replyTo? }. 404 if not a member, 400 if body empty or > 4000 chars. replyTo is validated against the same conversation. Inserts the message, calls deliver, and returns 201 with the shaped message.
- POST /conversations/:id/upload — multipart file plus optional body (caption ≤ 4000) and replyTo. 404 if not a member, 400 if no file. Inserts a message with file metadata, delivers it, returns 201 with the shaped message. Cleans up the temp file if the membership check fails.
- PATCH /message/:id — Body: { body }. Sender-only: 404 if the message doesn't exist or isn't the caller's, 400 if it was deleted, 400 if body empty. Updates the body with an edited_at, pushes an edit event, re-indexes for RAG. 200 { ok, body, edited_at }.

- POST /message/:id/forward — Body: { conversationIds: [...] }. 404 if the source is missing/deleted or the caller isn't a member. For each target conversation the caller belongs to, it copies any attached file (so deletes don't cascade), inserts a forwarded message, and delivers it. 200 { ok: true, forwarded_to: [...] }.
 - DELETE /message/:id — 404 if the message is missing or the caller isn't a member. If the caller is the sender it tombstones the message for everyone (clears body/file, deletes reactions, removes the RAG embedding, unlinks the file, pushes a delete scope all); otherwise it hides the message for the caller only and pushes a delete scope me. 200 { ok: true }.
 - POST /message/:id/reactions — Body: { emoji } (≤ 8 chars). 404 if the message is missing, the caller isn't a member, or emoji empty. One reaction per user per message; posting the same emoji removes it (toggle). Pushes a reaction event with the full reaction list. 200 { reactions }.
 - POST /message/:id/star — 404 if missing or not a member. Stars the message for the caller. 200 { ok: true, starred: true }.
 - DELETE /message/:id/star — Unstars the message for the caller. 200 { ok: true, starred: false }.
 - GET /starred — Returns up to 100 of the caller's starred, non-deleted messages, most recent first, each shaped with starred: true and a starred_at timestamp. 200 { items }.
-

routes/dashboards.js

Read-only dashboard aggregations. Mounted behind authRequired. Provides an employee's personal view, a manager team view (optionally date-scoped), a date-ranged manager report, and an org-wide admin view. Uses requireRole('manager', 'admin') on the manager/report/admin routes. Local helpers: today() (YYYY-MM-DD) and the OPEN SQL fragment listing open statuses.

Routes:

- GET /employee — Auth: any user. No params. Aggregates the caller's own top-level tasks (joined to meeting title). Returns 200 with counts (assigned, pending, completed, overdue, blocked), upcoming (up to 6 open dated tasks sorted by due date), by_status counts across the five statuses, and needs_confirmation count.
- GET /manager — Auth: manager/admin. Query: optional from, to (YYYY-MM-DD) that scope every figure to tasks **created** in that inclusive window (swapped if reversed; ignored unless both are valid dates). Considers only top-level, manager-visible tasks. Returns 200 with counts (total, open, completed, overdue, blocked, needs_confirmation, meetings), by_priority (open counts per Critical/High/Medium/Low), by_status counts, workload per employee (open/done/overdue counts, sorted by open desc), projects with a computed progress percentage, and up to 8 overdue tasks.
- GET /report — Auth: manager/admin. Query: **required** from, to (YYYY-MM-DD); 400 if either is invalid; reversed ranges are swapped. Computes created (in range), completed (Done and completed in range), dueInRange, and overdue sets over top-

level manager-visible tasks. Returns 200 with range, counts (created, completed, due, open, overdue), by_priority and by_status over the created set, workload per contributing employee (created/completed/overdue counts, filtered to non-empty, sorted by completed desc), and up to 60 each of completed_tasks and overdue_tasks.

- GET /admin — Auth: manager/admin (the manager is the org admin). No params. Returns 200 with totals (users, tasks, meetings, projects), users_by_role counts, tasks_by_status counts, and the 25 most recent audit-log entries with actor names.
-

routes/digest.js

A small admin-facing router for the daily email/Zoho Cliq digest. Mounted behind authRequired; both routes further require manager/admin.

Routes:

- GET /status — Auth: manager/admin. Returns 200 with the current delivery mode ('Cliq (live)' when Cliq is enabled, else 'preview/log'), the cliq boolean, the email mailer mode, and the configured digest hour (from DIGEST_HOUR, default 8).
 - POST /send-now — Auth: manager/admin. Triggers sendDailyDigests() immediately. Returns 200 with the summary object it produces, or 500 { error } on failure.
-

routes/invites.js

Manages organization invitations. Managers/admins create, list, and revoke invites (authenticated, org-scoped); invitees look up and accept them through public, unauthenticated routes. Uses per-org allowed-domain enforcement and 7-day token expiry.

Helper functions:

- validInvite(token) — returns the invite row only if it exists, is pending, and hasn't expired; else null.
- inviteEmailHtml(inviterName, orgName, role, link) — builds the invitation email body.

Constants: EMAIL_RE, INVITE_TTL_DAYS = 7.

Routes:

- POST / — Auth: authRequired + manager/admin. Body: { email, role?, department_id? }. Validates email format (400), enforces the org's allowed domains (400 with the permitted list), normalizes role to one of manager/employee/admin (defaulting to employee), blocks managers from inviting admins (403), rejects emails that already have an account (409), and validates the department belongs to the org (400). Re-inviting the same email revokes any earlier pending invite. Creates a pending invite (7-day expiry), writes an invite.create audit, and emails the accept link. Always returns the link (so it can be shared manually in preview mode). Returns 201 { id, email, role, department_id, status: 'pending', link, emailed }.

- GET / — Auth: authRequired + manager/admin. Lists the org's pending invites (id, email, role, department_id, status, created_at, expires_at), newest first. 200 array.
 - DELETE /:id — Auth: authRequired + manager/admin. 404 if the invite isn't in the caller's org. Marks it revoked, writes an invite.revoke audit. 200 { ok: true }.
 - GET /lookup — Auth: none. Query: token. Returns 404 if the invite is invalid/expired; otherwise 200 { email, role, org_name } so the accept page can render org context.
 - POST /accept — Auth: none. Body: { token, name, password }. 404 if the invite is invalid/expired. Validates a non-empty name (400), password length ≥ 8 (400), and not-common (400). If the email already has an account it marks the invite accepted and returns 409. Otherwise, in a transaction, it creates the user with the invite's org/department/role, email_verified = 1 (the link proved the address), and marks the invite accepted. Writes an invite.accept audit and logs the user in. Returns 201 { token, user: publicUser }.
-

routes/meetings.js

The meeting-intelligence router: managers/admins upload meeting transcripts or audio, which are transcribed and analyzed by AI into a summary plus a queue of suggested tasks that managers review, edit, merge, reject, restore, and finally assign into real tasks. Mounted behind authRequired; mutating routes add manager/admin. Uses multer in-memory storage (25 MB cap) for audio, records AI usage, and RAG-indexes meetings and created tasks. Exports the router plus the persistMeeting helper (used by the seed).

Helper functions:

- recordAnalysisUsage(req, transcript, analysis) — records an estimated usage event for a meeting LLM analysis, mapping the engine to a provider and approximating tokens from text; skips rule-based/offline analyses.
- filterLanguages(langs) — keeps only en/hi/te, deduped, defaulting to ['en'].
- deriveSegments(transcript) — builds display segments from raw text when the AI engine emits none (splits on newlines, else groups ~3 sentences), capped at 2000.
- attendeesFor(orgId, participantIds) — the users the AI may assign work to (selected participants, or the whole org if none), with role and department.
- createTaskFromSuggestion(s, { orgId, actorId, notifyAssignee }) — turns an approved suggestion into a real task (auto-filling the due date from priority when absent), marks the suggestion approved and links the task, optionally notifies the assignee, and RAG-indexes the task.
- persistMeeting({...}, analysis, opts) — inserts the meeting, participants, and transcript segments, then queues de-duped AI tasks as **pending** suggestions (not yet assigned); opts.autoApprove immediately assigns those with a resolved owner (used by the seed). Writes a meeting.process audit and indexes the meeting. Returns { mid, suggestionCount, assignedCount }.

Constants: ENGINE_PROVIDER, STT_PROVIDER, ALLOWED_LANGUAGES.

Routes:

- GET / — Auth: any user. Lists the org's meetings, each with a task_count and a pending-suggestion pending_count, ordered by meeting date then created date descending; detected_languages and summary are JSON-parsed. 200 array.
- GET /:id — Auth: any user. 404 if not found in the org. Returns the meeting plus parsed languages/summary, its segments, tasks (with assignee names), participants, and suggestions (with suggested-assignee name/color). 200.
- PATCH /:id — Auth: manager/admin. Body: { title?, meeting_date? }. 404 if not found. Updates title (defaulting to "Untitled Meeting") and/or date (sliced to 10 chars). Writes a meeting.update audit and re-indexes. 200 { ok: true }.
- DELETE /:id — Auth: manager/admin. 404 if not found. Removes RAG embeddings, then in a transaction deletes the meeting's tasks (cascading subtasks/comments/deps), transcript segments, and the meeting. Writes a meeting.delete audit. 200 { ok: true }.
- POST / — Auth: manager/admin. Body: { title?, description?, meeting_date?, transcript, summary_language?, participant_ids? }. 400 if the transcript is empty. Runs analyzeMeetingTranscript over the attendees, records usage, and persists the meeting with pending suggestions. Returns 201 { id, suggestion_count, engine, fallback_reason }, or 500 on processing failure.
- POST /transcribe — Auth: manager/admin. multipart audio. 400 if no file. Transcribes one short audio chunk (auto language detection) via transcribeAudio, records a transcription usage event, and returns 200 { text, language }. Errors: 400 when no STT provider is configured (NO_PROVIDER), else 502.
- POST /audio — Auth: manager/admin. multipart audio plus body { title?, description?, meeting_date?, participant_ids? (JSON string), summary_language? }. 400 if no file. Transcribes the whole file (routing long uploads to OpenAI Whisper when an OpenAI key is present, since Sarvam's instant endpoint caps at ~30s), records usage, 422 if the transcription is empty, then analyzes and persists as above. Returns 201 { id, suggestion_count, engine }. Errors: 400/502 on transcription failure.
- PATCH /suggestions/:sid — Auth: manager/admin. 404 if the suggestion isn't in the caller's org. Body may include title, description, priority, due_date, due_date_raw, assignee_reasoning, confidence (clamped 0-100), suggested_assignee_id (validated against the org, 400 if invalid). Updates the given fields and updated_at; returns the refreshed suggestion (or the unchanged one if nothing was provided). 200.
- POST /suggestions/:sid/reject — Auth: manager/admin. 404 if not found. Marks the suggestion rejected. 200 { ok: true }.
- POST /suggestions/:sid/restore — Auth: manager/admin. 404 if not found. Resets a rejected/merged suggestion back to pending (clearing merged_into) and returns the refreshed row. 200.
- POST /suggestions/:sid/merge — Auth: manager/admin. Body: { into }. 404 if either suggestion isn't found; 400 if merging into itself. Marks the source merged with merged_into set to the target. 200 { ok: true }.

- `POST /:id/assign` — Auth: manager/admin. Body: { `ids?` }. `404` if the meeting isn't found. Takes the meeting's pending suggestions (optionally filtered to `ids`), and in a transaction creates a real task from each that has a resolved assignee (notifying them), skipping those without one. Writes a `meeting.assign` audit. Returns `200` { `assigned`, `skipped` }.
-

routes/notifications.js

Manages in-app notifications and native push device-token registration. Mounted entirely behind `authRequired`.

Routes:

- `POST /register-device` — Body: { `token`, `platform?` }. `400` if no token. Saves the FCM device token for the user (`platform` defaults to `android`) so they get native push. `200` { `ok`: `true` }.
 - `POST /unregister-device` — Body: { `token` }. Removes the device token (called on logout so a shared device stops receiving the user's pushes). `200` { `ok`: `true` }.
 - `GET /` — Returns the caller's 50 newest notifications plus an unread count. `200` { `items`, `unread` }.
 - `POST /read-all` — Marks all of the caller's unread notifications read. `200` { `ok`: `true` }.
 - `POST /:id/read` — Marks a single notification (scoped to the caller) read. `200` { `ok`: `true` }.
-

routes/platform.js

Cross-organization super-admin routes — the only place in the app that reads and writes across org boundaries. Mounted behind `authRequired` **and** `requirePlatformAdmin`, so only a flagged platform admin can reach any of these. Normal tenant isolation is preserved elsewhere.

Constant: `ORG_TABLES` — the org-scoped tables listed in dependency order for a full org wipe.

Routes:

- `GET /stats` — Platform-wide totals: { `orgs`, `users`, `tasks`, `ai_cost` (sum of usage cost), `ai_calls` (count of usage events) }. `200`.
- `GET /usage` — Per-organization AI/API usage (`calls`, `tokens`, `cost`, and the `usage_access` flag), highest cost first, plus a total grand total across all usage events. `200` { `orgs`, `total` }.
- `PATCH /orgs/:id/usage-access` — Body: { `enabled` }. `404` if the org doesn't exist. Grants or revokes that org admin's ability to view their own usage. Writes a `platform.usage_access` audit. `200` { `ok`: `true`, `usage_access` }.

- GET /orgs — Every organization with per-org counts: user_count, task_count, last_activity (latest audit timestamp), owner_email (earliest-created user), usage_calls, usage_cost, plus parsed allowed_domains and a boolean usage_access. Ordered by creation date descending. 200 array.
 - GET /orgs/:id — 404 if the org doesn't exist. Returns the org (with parsed domains and boolean usage_access), its members (id, name, email, role, platform_admin, created_at), a task breakdown by_status, a derived task_count, and per-org usage (via usageForOrg). Only counts cross the boundary — no task content leaks. 200.
 - DELETE /orgs/:id — 404 if the org doesn't exist; 400 if it's the caller's own org (you cannot delete your own org from here). Disables FK enforcement (outside any transaction), then in a transaction deletes each user's password_resets and email_verifications, deletes rows from every ORG_TABLES table for that org (errors on tables without an org_id are caught and skipped), and finally deletes the organization; FK enforcement is re-enabled in a finally. Writes a platform.org_delete audit. 200 { ok: true }.
-

routes/tasks.js

The core task-management router: listing/filtering, CRUD, voice dictation/parsing/search, status-workflow transitions with approval semantics, task splitting into subtasks with parent roll-up, comments, dependencies, and RAG indexing. Mounted behind authRequired. Audio dictation uses multer in-memory storage with a 10 MB cap. Employees see only their own tasks (including private drafts); managers/admins see manager-visible tasks plus their own drafts.

Helper functions:

- hydrate(t) — expands a task with its assignee, assignedBy, project, subtasks, comments (with user name/color), dependencies, and attachments.
- syncParentStatus(parentId) — rolls a split parent's status up from its children: auto-completes the parent when all parts are Done; reopens it (progress capped at 80) if a part is later un-done. Re-indexes and notifies on completion.

Constants: STT_PROVIDER, audioUpload, VALID_STATUS (To Do, In Progress, Blocked, In Review, Done, Reopened).

Routes:

- GET / — Query filters: status, priority, assignee (or 'unassigned'), project, meeting, mine, q, confidence. Employees are constrained to their own tasks; managers/admins see manager-visible tasks plus their own. q matches title, description, or the assignee's name. Results are top-level tasks only, ordered by priority then due date, each hydrated. 200 array.
- GET /:id — 404 if not in the org. Returns the hydrated task. 200.
- POST /transcribe — multipart audio. Available to everyone (unlike the meetings transcriber). 400 if no file. Transcribes one dictated clip, records a transcription usage event, returns 200 { text, language }. Errors 400 (NO_PROVIDER)/502.

- POST /voice-search — multipart audio. 400 if no file. Transcribes a spoken query, then (if an LLM is configured) interprets it into structured filters via interpretVoiceSearch, recording transcription and voice_search usage; falls back to the raw transcript as the query. Resolves any named person to an assignee id/name. Returns 200 { transcript, query, assignee_id, assignee_name, status, priority } (query left empty when a person matched but no topic words, so the assignee filter returns all their tasks). Errors 400/502.
- POST /parse-voice — Body: { transcript }. 400 if empty. If no LLM, returns a fallback object using the transcript as the title. Otherwise parses the sentence into draft fields via parseSpokenTask (recording voice_task usage), resolves the spoken assignee and deadline (trying the AI phrase then scanning the whole transcript). Returns 200 { title, description, assignee_id, assignee_name, priority, due_date, due_date_raw, engine }; on failure it returns the fallback.
- POST / — Body: task fields including title (required, 400 otherwise), personal?, assignee_id?, priority?, due_date?, and others. A personal task is owned by the creator, hidden from managers, and high-confidence; otherwise anyone may assign to anyone in the org or leave it unassigned. The due date auto-fills from priority when omitted. Writes a task.create audit, notifies the assignee (unless it's the creator), and RAG-indexes. Returns 201 with the hydrated task.
- PATCH /:id — 404 if not in the org. Body may include title, description, priority, due_date, due_date_raw, project_id, department_id, progress, ownership_confidence, assignee_id, status. Reassignment updates confidence and assigned_at and flags a new assignment; a status must be valid (400 otherwise) and Done/In Review stamp completed_at/submitted_at. When (re)assigning or (re)prioritizing a task that has no due date, one is auto-filled from priority unless explicitly set. Writes a task.update audit, notifies a newly-assigned user, rolls up the parent on status changes, and re-indexes. Returns 200 with the hydrated task.
- POST /:id/status — Body: { status }. 400 if invalid, 404 if not found. Applies workflow semantics: In Review sets approval pending, stamps submitted_at, and surfaces private drafts to managers; Done sets progress 100 and completed_at; Reopened clears approval, caps progress at 80, and clears completion. Writes a task.status audit, pings managers on submission, rolls up the parent, and re-indexes. Returns 200 hydrated task.
- POST /:id/split — Body: { parts: [{ title, assignee_id }] }. 404 if not found; 403 unless the caller is a manager/admin or the task's owner; 400 if the task is itself a shared part or if no valid parts are given. Creates a child task per valid part (inheriting due/priority/project/department/meeting), audits each task.split, indexes it, and notifies each assignee. Splitting forces the parent visible to managers. Returns 201 with the hydrated parent.
- POST /:id/approve — Auth: manager/admin. Body: { decision: 'approved' | 'rejected' }. 404 if not found. Sets the task to Done/approved (stamping completion) or Reopened/rejected. Writes a task.approval audit and notifies the assignee of the verdict. Returns 200 hydrated task.
- POST /:id/comments — Body: { body }. 400 if empty, 404 if not found. Inserts a comment, audits task.comment, and notifies the assignee and the assigner (except the commenter) with a snippet. Returns 201 hydrated task.

- POST `/:id/dependencies` — Body: `{ depends_on }`. 404 if either task isn't in the org; 400 if a task depends on itself. Inserts the dependency (ignoring duplicates). Returns 200 hydrated task.
 - DELETE `/:id` — 404 if not found. Managers/admins may delete any task; an employee may delete only their own unsubmitted private draft (403 otherwise). Deletes the task, audits `task.delete`, and removes its embedding. 200 `{ ok: true }`.
-

routes/usage.js

A single-purpose router giving an org admin a read-only view of their **own** organization's AI/API usage. Mounted behind `authRequired` and `requireRole('admin')`, and additionally gated by the `usage_access` flag (which only a platform super-admin can grant); platform admins always have access. Never crosses org boundaries.

Routes:

- GET `/` — Loads the caller's org; access is allowed if the caller is a platform admin or the org's `usage_access` flag is set. If not allowed, returns 403 `{ error, code: 'USAGE_DISABLED' }`. Otherwise returns 200 `{ org_name, ...usageForOrg(org) }`.
-

routes/users.js

Manages user accounts, self-service profile updates, avatars, admin/manager user CRUD, bulk Excel/CSV import, and org-metadata endpoints (departments, projects, allowed email domains). The avatar-serving route is declared **before** `authRequired` so `<img>` tags can pass a token via query string; everything else requires auth. Avatars are stored on disk in `data/avatars`.

Helper functions:

- `domainError(orgId, email)` — returns an error string if the email's domain isn't allowed for the org, else null.
- `onlyDigits(s)` — strips non-digits (phone normalization).
- Local `pick(row, ...keys)` inside import — case-insensitive column getter.

Constants: `AVATAR_DIR`, `upload` (memory, 5 MB, for import files), `avatarUpload` (disk, 5 MB, images-only filter), `VALID_EMAIL`.

Routes:

- GET `/:id/avatar` — Auth: token via header or `?token=`. 401 if the token is invalid. Enforces tenant isolation by only serving avatars of users in the requester's own org. 404 if no avatar or the file is missing. Streams the image with a 5-minute private cache.

(from here `authRequired` applies)

- POST `/me/avatar` — multipart file (images only). 400 if no file. Replaces the caller's avatar, deleting the old file. 200 `{ ok: true, avatar_file }`.

- PATCH /me — Self-service profile update. Body may include name (non-empty, 400 otherwise), preferred_language, and a password change via new_password (≥ 8 , 400 otherwise) which requires a correct current_password (400 if wrong). Applies the changes, writes a user.self_update audit listing changed fields, and returns 200 with the publicUser shape. Declared before PATCH /:id so "me" isn't captured as an id.
- GET / — Any user. Lists org users with minimal fields (id, name, email, role, phone, department_id, avatar_color, avatar_file, aliases, preferred_language), ordered by name, for assignment dropdowns. 200 array.
- POST / — Auth: manager/admin. Body: { name, email, password, role, department_id?, aliases?, preferred_language?, phone }. Requires name/email/password/role (400), a valid role (400), blocks managers from creating admins (403), validates email format (400) and org domain (400), requires exactly 10 phone digits (400), and rejects duplicate emails (409). Creates the user with a random avatar color, audits user.create. Returns 201 with publicUser.
- PATCH /:id — Auth: manager/admin. 404 if not in the org. Blocks managers from modifying admin accounts or granting admin (403). Validates a changed email's format and domain (400) and uniqueness (409), and a changed phone as 10 digits (400). Updates name, role, department_id, aliases, preferred_language, phone, email, password as provided, audits user.update, and returns 200 with publicUser.
- DELETE /:id — Auth: manager/admin. 404 if not found; 400 if it's the caller's own account; 403 if a manager targets an admin. In a transaction it unassigns the user's tasks, detaches them from the AI review queue, deletes their comments, and deletes the user (notifications cascade). Audits user.delete. 200 { ok: true }.
- POST /import — Auth: manager/admin. multipart file (Excel/CSV). 400 if no file or the file can't be parsed. Reads the first sheet; expected case-insensitive columns include name, email, role, department, aliases, language, phone, password. Per row it validates name/email presence, email format, and org domain (collecting per-row errors), normalizes role (managers can't grant admin), maps the department by name, and **upserts by email** — skipping admin accounts when the caller is a manager. New users without a password get a default (password123). Wrapped in a transaction (500 on failure). Audits user.import. Returns 200 { created, updated, errors, rows }.
- GET /meta/departments — Any user. Lists the org's departments ordered by name. 200 array.
- GET /meta/projects — Any user. Lists the org's projects ordered by name. 200 array.
- GET /meta/org — Any user. Returns the org's id, name, is_personal plus its allowed_domains list. 200.
- PATCH /meta/org — Auth: manager/admin. Body: { allowed_domains: [...] }. Cleans the list to valid, unique, lowercase domains (stripping a leading @); an empty list means no restriction. Persists them (comma-joined or null), audits org.domains_update, and returns 200 { allowed_domains }.

9. AI Modules (server/src/ai/)

ai/assistant.js

This module implements a fully offline, rule/intent-based natural-language assistant and search over the organization's tasks. It uses **no external AI provider or model** — instead it pattern-matches the user's query against a set of regular expressions and keyword checks, then queries the SQLite database directly. It exists so the chat feature always works even when no LLM key is configured (it is the fallback for `assistantChat.js`). All results are RBAC-scoped: employees only ever see their own tasks; managers and admins see the whole org. It reads no environment variables.

- **taskRows(orgId)** — Parameters: `orgId`. Returns: an array of top-level task rows (parent tasks only) for the org, each joined with assignee name, assigned-by name, and project name. It runs a single SQL SELECT with LEFT JOINs against `users` (twice) and `projects`, filtered to the org and to tasks with no parent. It is the raw data source every query path filters over.
- **today()** — Parameters: none. Returns: today's date as a YYYY-MM-DD string. It is an arrow constant that slices the current ISO timestamp. Used throughout for overdue and due-today comparisons.
- **isOverdue(t)** — Parameters: a task `t`. Returns: a boolean. It reports true when the task has a due date earlier than today and its status is not Done. Used to build overdue lists and reports.
- **scope(tasks, user)** — Parameters: the full task array and the requesting user. Returns: the subset the user may see. Employees get only tasks whose `assignee_id` equals their own id; managers and admins get the whole array unfiltered. This enforces per-role visibility before any query logic runs.
- **answerQuery(rawQuery, user)** (exported) — Parameters: the raw query string and the user. Returns: an object { `answer`, `tasks` } (and sometimes `data`). It lowercases/trims the query, scopes the tasks, then tries a cascade of intent matchers in order: "who owns/is responsible for X" (keyword ownership lookup); "tasks assigned to/for " (resolves the name via `resolveUser` and filters by assignee); overdue; priority (critical/high/medium/low); pending/open; meeting-derived tasks (optionally "yesterday's" meeting); daily status report; weekly progress report; workload/imbalance; and completed. If nothing matches it strips filler words and does a keyword substring search across titles/descriptions, and finally returns a help message listing example queries. Each branch returns a human-readable answer plus the matching task list.
- **dailyReport(tasks)** — Parameters: the scoped task array. Returns: a multi-line status string. It computes counts of due-today, overdue, in-progress, and blocked tasks, formats them with emoji bullet points, and appends a warning line listing blocked task titles if any exist. Empty lines are filtered out before joining.
- **weeklyReport(tasks)** — Parameters: the scoped task array. Returns: a multi-line progress string. It counts done, still-open, and overdue tasks, computes a completion percentage ($\text{done} \div \text{total}$), and formats them as a bulleted report.

- **workloadAnswer(orgId)** (exported) — Parameters: orgId. Returns: { answer, tasks, data }. It runs a SQL query counting each non-admin user's open tasks, computes the average open count per person, flags anyone with more than 1.5× the average (and at least 3 tasks) as "overloaded," and builds a per-person line list. If overloaded people exist it appends a suggestion recommending reassignment to the lightest-loaded (below-average) users. The raw per-user rows are returned in data.

ai/assistantChat.js

This module is the conversational LLM-backed assistant ("TaskBot") strictly scoped to the Task Manager's data. Its provider chain is **Claude first (via ANTHROPIC_API_KEY, default model claude-opus-4-8), then OpenAI (via OPENAI_API_KEY, default model gpt-4o-mini)**; if neither key is set or the call/parse fails, the calling route falls back to the rule-based answerQuery. It layers RAG on top: when an embedding index exists it retrieves the most question-relevant chunks across meetings, transcripts, chat, and tasks (RBAC-filtered) and appends them as a "RELEVANT CONTEXT" block, letting the bot answer from meeting/chat content beyond the raw snapshot cap. Environment variables read: ANTHROPIC_API_KEY, ANTHROPIC_MODEL, OPENAI_API_KEY, OPENAI_MODEL.

- **today()** / **isOverdue(t)** — Same helper pattern as in assistant.js: today() returns the YYYY-MM-DD string; isOverdue(t) is true when a task is past due and not Done.
- **hasLLM()** (exported) — Parameters: none. Returns: boolean. It reports true if either ANTHROPIC_API_KEY or OPENAI_API_KEY is set, letting the route decide whether to attempt the LLM path at all.
- **scopedTasks(user)** — Parameters: user. Returns: an array of the user's visible tasks. It runs a detailed SQL SELECT joining users (assignee and assigned-by), projects, departments, and meetings, ordered by most-recently-updated, filtered to top-level tasks in the org. Employees get only their own tasks (post-filtered in JS); managers/admins get all.
- **workloadRows(orgId)** — Parameters: orgId. Returns: per-person open-task counts for non-admins, with role and department. It uses a correlated subquery counting each user's open tasks. This is only surfaced to managers/admins in the context.
- **buildContext(user)** — Parameters: user. Returns: { context, tasks }. It gathers scoped tasks, computes summary stats (total, open, done, overdue, due-today, blocked), then renders up to MAX_TASKS_IN_CONTEXT (200) tasks as compact pipe-delimited lines (id, title, owner, status, priority, and optional due/project/dept/progress/meeting fields). If more tasks exist than are shown it appends a truncation note. For non-employees it appends a "TEAM WORKLOAD" section. The assembled text becomes the DATA SNAPSHOT.
- **ragBlock(hits)** — Parameters: an array of RAG hits. Returns: a formatted "RELEVANT CONTEXT" string (or empty string when no hits). Each hit is rendered as a labeled bullet; task hits keep task id=<id> so the model can cite them for clickable cards, while meetings/transcripts/chat get friendlier labels from the RAG_LABELS map. The block instructs the model that these are semantic-search excerpts to answer from.
- **systemPrompt(user)** — Parameters: user. Returns: the system-prompt string. **Summary of the prompt:** it casts the model as "TaskBot" inside "SmartTask," strictly limited to discussing this task manager's data (tasks, workload,

meetings/transcripts, internal chat, and derived reports). It orders the model to refuse anything out of scope in one polite sentence, to answer ONLY from the DATA SNAPSHOT and RELEVANT CONTEXT (never inventing data), to prefer the RELEVANT CONTEXT for "what was said/decided" questions, and to respect a role-specific scope note (employees only see their own tasks). It injects today's date and the user's name, demands concise chat-bubble style with no markdown headers/tables, and requires the reply to be a JSON object `{"answer": ..., "task_ids": [...]}` with up to 12 relevant task ids for clickable cards.

- **normalizeHistory(history)** — Parameters: client-supplied history. Returns: cleaned alternating turns. It filters to well-formed user/ai messages, keeps the last 8, and maps them to `{ role: 'user' | 'assistant', content }` for the API.
- **parseModelJson(text)** — Parameters: raw model text. Returns: `{ answer, task_ids }` or null. It slices from the first `{` to the last `}`, JSON-parses it, and coerces answer to a string and task_ids to an array of strings; on parse failure it returns null.
- **callClaude(system, messages, onUsage)** — Parameters: the system prompt, the message array, and an optional usage callback. Returns: the concatenated text of Claude's response. It reads `ANTHROPIC_MODEL` (default `claude-opus-4-8`), POSTs to the Anthropic Messages API with `max_tokens: 1024`, throws on a non-OK response, and — if `onUsage` is given — reports `{ provider: 'anthropic', model, inputTokens, outputTokens }` from `data.usage`.
- **callOpenAI(system, messages, onUsage)** — Parameters: same shape. Returns: the assistant message content. It reads `OPENAI_MODEL` (default `gpt-4o-mini`), POSTs to the OpenAI chat completions API with `response_format: json_object`, `temperature: 0.2`, `max_tokens: 1024`, throws on error, and reports usage as `{ provider: 'openai', model, inputTokens: prompt_tokens, outputTokens: completion_tokens }`.
- **toCard(t)** — Parameters: a task. Returns: a trimmed display object (id, title, status, priority, due_date, assignee_name, project_name) matching the rule-based path's card shape.
- **chatAnswer(query, user, history, onUsage)** (exported, async) — Parameters: the query, user, optional history, optional usage callback. Returns: `{ answer, tasks, engine }`. It throws if no LLM is configured. It builds the context snapshot, attempts RAG retrieval (swallowing retrieval errors and continuing without it), assembles the system prompt and messages (history plus a user turn containing the snapshot, RAG text, and question), then runs the provider chain: try Claude first and fall through to OpenAI if Claude fails and OpenAI is available. It parses the JSON reply (throwing on empty), and hydrates only the task ids the user is actually allowed to see (via a lookup map, never trusting the model to leak out-of-scope tasks), returning them as cards along with the engine name.

ai/claude.js

This module performs Claude-powered multilingual meeting analysis — reading a transcript and extracting a structured summary plus assignable tasks. Provider/model: **Anthropic Claude via `ANTHROPIC_API_KEY`, model from `ANTHROPIC_MODEL` (default `claude-opus-4-8`)**, calling the Messages API directly. It is used by the extractor

orchestrator when the Anthropic key is present; otherwise the orchestrator falls back to OpenAI or the rule engine. It supports English/Hindi/Telugu and code-mixed speech and only ever reports those three language codes.

- **keepAllowedLangs(val)** — Parameters: a value that is either an array or a +-joined string of language codes. Returns: an array filtered to only en, hi, te. It normalizes (trim/lowercase) then drops any other language the model emitted, so unwanted languages never leak through.
- **clampScore(v, fallback)** — Parameters: a raw confidence v and a fallback. Returns: a 0–100 integer. It rounds the number and clamps to the range; if the value is not finite it returns the fallback. Used to sanitize the model's per-task confidence.
- **SYSTEM_PROMPT** (constant) — **Summary:** it casts the model as an expert multilingual meeting analyst that must understand English/Hindi/Telugu and any code-mixed blend including accents and mixed scripts. Its core rules: always write summary fields and every task title/description in English (translating as needed), keeping only source_quote in the original language; a task exists whenever someone is asked to do or commits to something; each task has exactly one assignee and a single sentence assigning different work to different people must become separate tasks; assignees must be chosen only from the provided attendee list; infer priority from urgency cues; capture the raw natural-language due phrase and also resolve an absolute date; provide reasoning and a 0–100 confidence per assignee; ignore greetings/jokes and merge duplicate items. It ends by specifying the exact JSON output shape (detected_languages, participants, segments, summary block, and tasks array).
- **analyzeWithClaude(transcript, opts)** (exported, async) — Parameters: the transcript string and opts (which may include attendees, knownNames, meetingDate, summaryLanguage). Returns: the parsed analysis object with normalized tasks, engine: 'claude', and cleaned detected_languages. It throws if ANTHROPIC_API_KEY is missing, builds an attendee list block (with roles/departments, or known names, or "none provided"), constructs a user message carrying meeting date, attendees, requested summary language, and the transcript, then POSTs to the Anthropic API with max_tokens: 4096. It slices the JSON out of the response, parses it, and maps each task into the internal shape (assignee_name_raw, assigned_by_name_raw, clamped confidence with a fallback of 80/30 depending on whether an assignee was named, validated priority defaulting to Medium, ownership confidence, source quote, and a whitelisted language falling back to script-detection). It also whitelists the detected languages and falls back to detectLanguages(transcript) if none survive.

ai/dates.js

This module is a pure, dependency-light natural-language deadline parser across English, Hindi, and Telugu (both romanized and native scripts). It uses **no AI provider** and reads **no environment variables** — it is deterministic string matching against weekday and relative-date cues. It returns { date, raw } where date is a resolved YYYY-MM-DD (or null) and raw is the matched phrase; a reference date (the meeting date) anchors all relative phrases.

- **toISO(d)** — Parameters: a Date. Returns: a YYYY-MM-DD string built from the local year/month/day parts. It deliberately avoids toISOString() (which is UTC) so a midnight-local date does not shift back a day on servers east of UTC (e.g. IST), which would otherwise make "tomorrow" resolve to today.

- **addDays(refDate, n)** — Parameters: a base date and an integer n. Returns: a new Date offset by n days. It clones the reference date and mutates the copy's day-of-month.
- **nextWeekday(refDate, target, forceNextWeek)** — Parameters: a base date, a target weekday index (0–6), and an optional forceNextWeek flag. Returns: the Date of the next occurrence of that weekday. It computes the forward difference; if the target equals the current day it jumps a full week ("Friday" said on Friday means next Friday); when forceNextWeek is set and the difference is under a week it adds seven more days.
- **parseDueDate(text, refISO)** (exported) — Parameters: the phrase text and the reference ISO date refISO. Returns: { date, raw }. It builds the reference Date (defaulting to today), normalizes the text (lowercase, punctuation to spaces, whitespace collapsed, space-padded for whole-word matching), then checks cue groups in priority order: day-after-tomorrow (before "tomorrow"), tomorrow, today/EOD/tonight, "next ", end-of-week (resolves to Friday), next week, a bare weekday name, event-based anchors like "before deployment" (kept as raw phrase with null date), and finally an explicit ISO/YYYY-MM-DD date in the text. Each cue set includes English, romanized Hindi/Telugu, and native-script variants. If nothing matches it returns { date: null, raw: null }.

ai/embeddings.js

This module is the provider-agnostic embedding layer for RAG — turning text into vectors for semantic similarity. Providers: **Voyage (Anthropic's recommended embeddings partner) via VOYAGE_API_KEY, model from VOYAGE_EMBED_MODEL (default voyage-3)** wins when its key is present; otherwise **OpenAI via OPENAI_API_KEY, model from OPENAI_EMBED_MODEL (default text-embedding-3-small, 1536 dims)**. When neither key is set, hasEmbeddings() is false and the assistant simply skips RAG. It also provides vector serialization and cosine-similarity helpers.

- **provider()** — Parameters: none. Returns: 'voyage', 'openai', or null. It prefers Voyage when its key exists, else OpenAI when that key exists, else null. This single function decides which backend and model the rest of the module uses.
- **hasEmbeddings()** (exported) — Parameters: none. Returns: boolean (true when a provider is configured). Callers use it to gate all RAG work.
- **embedModel()** (exported) — Parameters: none. Returns: the active model name (Voyage's or OpenAI's) so the indexer can store which model produced each vector.
- **hashText(text)** (exported) — Parameters: a string. Returns: its SHA-1 hex digest. Used to detect unchanged chunks and skip re-embedding.
- **toBlob(vec)** (exported) — Parameters: a numeric vector. Returns: a Node Buffer holding the Float32 bytes, for storing the vector as a SQLite BLOB.
- **fromBlob(buf)** (exported) — Parameters: a stored buffer. Returns: a Float32Array view over its bytes, reconstructing a vector from the BLOB.
- **cosineSim(a, b)** (exported) — Parameters: two equal-length vectors. Returns: their cosine similarity in $[-1, 1]$ (0 if either has zero magnitude). It iterates over the shorter length accumulating the dot product and each magnitude, then divides the dot by the

product of the norms.

- **embedTexts(texts)** (exported, async) — Parameters: an array of strings. Returns: an array of Float32Array vectors in the same order. It short-circuits to [] for empty input, throws if no provider is configured, and replaces empty strings with a space (providers reject blanks). For Voyage it POSTs all inputs in one batched call and maps the returned embeddings; for OpenAI it POSTs one batched call, then sorts the results by their .index (to be safe) before mapping. It throws on any non-OK HTTP response.
- **embedQuery(text)** (exported, async) — Parameters: a single string. Returns: one Float32Array. It delegates to embedTexts([text]) and returns the first vector — a convenience for embedding a search query.

ai/extractor.js

This module is the orchestrator for meeting-transcript analysis and the resolver that maps spoken names to real user records. It selects an AI engine by the **priority chain** **OpenRouter** → **Claude** → **OpenAI** → **offline rules**, each tier attempted only if its key is set and falling through on error. Environment variables checked: OPENROUTER_API_KEY, ANTHROPIC_API_KEY, OPENAI_API_KEY (the actual model names live in the delegated modules). It imports the three provider analyzers plus the rule engine.

- **resolveUser(orgId, rawName)** (exported) — Parameters: an org id and a raw spoken/typed name. Returns: the matching user row or null. It loads all org users and tries five increasingly loose but still whole-word strategies in order: exact full name, an exact match against a comma-separated alias list, first-name-equals-spoken, any name token (≥ 2 chars) equal to a spoken token, and finally a clear prefix match on the first name for fragments ≥ 4 chars. Matching is deliberately whole-word (never loose substring) so "an" never matches "Ananya."
- **resolveUserAmong(orgId, rawName, allowedIds)** (exported) — Parameters: org id, raw name, and a list of allowed user ids (the meeting attendees). Returns: the resolved user or null. It calls resolveUser and then rejects the result if it is not in the allowed set, ensuring the AI can never assign work to a non-attendee.
- **analyzeMeetingTranscript(transcript, opts)** (exported, async) — Parameters: the transcript and opts. Returns: the unified analysis result. It checks which keys are present and tries OpenRouter first (analyzeWithOpenRouter), then Claude (analyzeWithClaude), then OpenAI (analyzeWithOpenAI), logging a warning and falling through on each failure. If OpenAI itself fails it runs the offline analyzeTranscript and attaches a fallback_reason. If no keys at all are configured it goes straight to the rule engine.

ai/openai.js

This module performs OpenAI (GPT) multilingual meeting analysis, used when OPENAI_API_KEY is set (and typically no Anthropic key). Provider/model: **OpenAI chat completions via OPENAI_API_KEY, model from OPENAI_MODEL (default gpt-4o-mini)**, using JSON-object response format. It is the middle-to-last AI tier before the rule engine, extracting the same structured summary and task shape as claude.js.

- **clampScore(v, fallback)** — Same as in `claude.js`: rounds and clamps a confidence into 0–100, returning the fallback when the value is not finite.
- **SYSTEM_PROMPT** (constant) — **Summary:** nearly identical in intent to `claude.js`'s prompt. It casts the model as an expert multilingual meeting analyst for English/Hindi/Telugu and code-mixed speech, requires all summary/task text in English while keeping `source_quote` verbatim, defines task-extraction rules (task = asked-to-do or committed-to; assignee must come from the attendee list; assignee reasoning; 0–100 confidence tiers; priority from urgency; raw and resolved due dates), tells it to ignore casual chatter and merge duplicates, and specifies the exact JSON shape (`detected_languages`, `participants`, `segments`, `summary block`, `tasks array`).
- **analyzeWithOpenAI(transcript, opts)** (exported, async) — Parameters: the transcript and `opts` (`attendees`, `knownNames`, `meetingDate`). Returns: the parsed analysis with normalized tasks, `engine: 'openai'`, and `detected_languages`. It throws if `OPENAI_API_KEY` is missing, builds the attendee block, composes the user message (meeting date, attendees, an English-output reminder, and the transcript), and POSTs to the chat completions API with `response_format: json_object` and `temperature: 0.2`. It JSON-parses the message content and maps each task into the internal shape (raw name fields, clamped confidence with 80/30 fallback, validated priority defaulting to Medium, ownership confidence, source quote, and a language falling back to `detectLanguages`). Unlike `claude.js` it does not whitelist languages to en/hi/te here; it only defaults `detected_languages` via script detection when the model omits them.

ai/openrouter.js

This module is the **primary** meeting analyzer and the most sophisticated of the three AI engines, used when `OPENROUTER_API_KEY` is set. Provider/model: **OpenRouter's OpenAI-compatible chat completions API via `OPENROUTER_API_KEY`, model from `OPENROUTER_MODEL` (default `google/gemini-2.5-pro`)**, chosen for its 1M-token context and strong Telugu/Hindi/English understanding. Its distinctive feature is a long-meeting strategy: short transcripts go in one shot, while long ones use map-reduce with a rolling memory so names/decisions/ownership stay consistent across a 2h+ meeting. Additional env vars: `OPENROUTER_SINGLESOT_CHARS` (default 280,000), `OPENROUTER_CHUNK_CHARS` (default 60,000), `OPENROUTER_OVERLAP_CHARS` (default 2,000), `OPENROUTER_SITE_URL`, and `OPENROUTER_APP_NAME` (attribution headers, defaulting to `http://localhost` and SmartTask AI).

- **keepAllowedLangs(val) / clampScore(v, fallback)** — Same helpers as `claude.js`: whitelist language codes to en/hi/te, and round-and-clamp a confidence into 0–100.
- **Prompt constants** — `LANGUAGE_RULE` (a shared block requiring understanding of En/Hi/Te code-mixed speech, English output except `source_quote`, and `detected_languages` restricted strictly to en/hi/te with related Indian languages treated as Hindi), `TASK_RULES` (the shared task-extraction rules: one assignee per task, split multi-person sentences, attendee-only assignees, `needs_confirmation` when unclear, reasoning, confidence tiers, priority, raw and resolved due dates, source quote, ignore chatter), and `TASK_SHAPE` (the JSON schema fragment for tasks). These are composed into three system prompts: **SINGLE_SYSTEM** (one-shot full analysis), **MAP_SYSTEM** (analyze ONE chunk using a supplied running memory, extracting tasks only from

the current chunk and emitting an updated ≤ 200 -word running memory), and **REDUCE_SYSTEM** (final synthesis: merge all chunk memories and the combined raw task list into one cohesive English summary and a de-duplicated task set).

- **callOpenRouter(systemPrompt, userMsg)** — Parameters: a system prompt and a user message. Returns: the parsed JSON object of the model reply. It throws if the key is missing, POSTs to the OpenRouter endpoint with the model, both messages, `response_format: json_object`, `temperature: 0.2`, and optional attribution headers (HTTP-Referer, X-Title), throws on a non-OK response, and passes the content through `parseJson`.
- **parseJson(text)** — Parameters: raw model text. Returns: the parsed object. It trims, strips markdown code fences if present, slices from the first { to the last }, and JSON-parses it, throwing if no JSON object is found.
- **normalizeTask(t)** — Parameters: a model task. Returns: the internal task shape (raw name fields, clamped confidence with 80/30 fallback, null-safe due dates, validated priority defaulting to Medium, ownership confidence, source quote, and a whitelisted-or-script-detected language). Mirrors the normalization in `claude.js/openai.js`.
- **chunkTranscript(text)** — Parameters: the full transcript. Returns: an array of overlapping chunks. It walks the text in `~CHUNK_CHARS` steps, preferring to break on a newline or sentence boundary within the last 25% of each chunk so it never cuts mid-sentence, and starts each next chunk `OVERLAP_CHARS` before the previous end so context bleeds across boundaries.
- **attendeeBlock(opts)** — Parameters: `opts`. Returns: a formatted attendee list string (names with roles/departments, or known names, or "none provided"). Reused across all prompt paths.
- **analyzeWithOpenRouter(transcript, opts)** (exported, async) — Parameters: the transcript and `opts`. Returns: the parsed analysis with normalized tasks, `engine: 'openrouter'`, and whitelisted detected languages. It throws without the key. If the transcript is at or below `SINGLE_SHOT_CHARS` it takes the one-shot path (one `SINGLE_SYSTEM` call with meeting date, attendees, and full transcript). Otherwise it takes the map-reduce path: it chunks the transcript, then iterates chunks calling `MAP_SYSTEM` with the running memory and the current chunk (accumulating tasks and updating the rolling memory, skipping a chunk on error), and finally calls `REDUCE_SYSTEM` with all per-chunk memories and the combined raw task list to synthesize one summary and de-duplicate tasks. As a safety net, if reduce returns no tasks it keeps the raw union. It then normalizes all tasks and whitelists/falls-back the detected languages.

ai/ragIndex.js

This module is the RAG indexer: it turns org data (tasks, meetings, transcript segments, chat messages) into embedding rows stored in SQLite for semantic retrieval. It reads no API keys directly but delegates all embedding work to `embeddings.js` (so the active provider/model is whatever that module selects). Every incremental hook is best-effort — it swallows errors and no-ops when no embedding provider is configured — so it can be sprinkled into routes safely. `BATCH` is 96 inputs per embedding API call.

- **taskChunk(t)** — Parameters: a task row. Returns: the text string embedded for that task. It joins the title, the description (only if different from the title), and a parenthetical metadata clause (owner, status, priority, due, project) into one sentence-like blob.
- **meetingChunk(m)** — Parameters: a meeting row. Returns: the text to embed. It parses `summary_json`, preferring the executive summary, else joining action items, and prefixes the meeting title and date. Bad JSON is silently ignored.
- **indexItems(items)** (exported, async) — Parameters: an array of item descriptors (`source_type`, `source_id`, `org_id`, optional `ref_user_id/ref_convo_id`, and `text`). Returns: { `embedded`, `skipped` }. It no-ops when embeddings are off or the list is empty. It loads existing rows' content hashes, computes each item's hash, and keeps only items whose text changed (stale). It then embeds the stale texts in batches of BATCH, and upserts each row (`id`, `org`, `source`, `refs`, `chunk text`, `hash`, `dim`, `vector BLOB`, `model`, `timestamp`) inside a DB transaction, using an `ON CONFLICT(source_type, source_id) update clause`.
- **taskItems(where, params) / meetingItems(...) / segmentItems(...) / chatItems(...)** — Parameters: an optional extra SQL where fragment and its params. Each returns an array of item descriptors for its source type. `taskItems` selects top-level tasks (joined to assignee and project) and builds a `taskChunk`, carrying `ref_user_id`. `meetingItems` selects meetings and builds a `meetingChunk`. `segmentItems` joins transcript segments to meetings and builds a "title: speaker: text" chunk. `chatItems` selects non-deleted, non-empty chat messages and uses their body as text, carrying `ref_convo_id`. These collectors are shared by both the full backfill and the incremental hooks (which pass an `id` filter).
- **backfillAll()** (exported, async) — Parameters: none. Returns: { `embedded`, `skipped` }. It gathers all task, meeting, segment, and chat items and passes them to `indexItems` — a one-time or batch reindex (also runnable via `npm run rag:index`).
- **pruneOrphans()** (exported) — Parameters: none. Returns: the number of rows removed. It deletes embeddings whose source row no longer qualifies for indexing (deleted rows, unsent/soft-deleted chat, emptied segments); the `NOT IN` predicates mirror each collector's `WHERE` so no live vector is pruned. Errors are caught and logged, returning 0.
- **syncAll()** (exported, async) — Parameters: none. Returns: { `embedded`, `skipped`, `pruned` }. It runs `backfillAll` then `pruneOrphans`, making the index fully current; this is what the server runs on boot and on a timer. It is cheap on repeat (near-zero API calls when nothing changed).
- **safeIndex(items)** — Parameters: an item array. Returns: nothing meaningful. It calls `indexItems` only when embeddings are configured, wrapped in `try/catch` so a failure just warns. It backs the incremental hooks.
- **indexTask(taskId) / indexMeeting(meetingId) / indexChatMessage(msgId)** (exported) — Parameters: the changed row's id. Each returns the `safeIndex` promise. They call the matching collector(s) with an `id` filter to (re)embed a single changed row; `indexMeeting` reindexes both the meeting and all its segments.
- **removeEmbedding(sourceType, sourceId)** (exported) — Parameters: a source type and id. Returns: nothing. It deletes the single embedding row, best-effort (errors swallowed).

- **removeMeetingEmbeddings(meetingId)** (exported) — Parameters: a meeting id. Returns: nothing. It deletes all embeddings derived from a meeting — the meeting itself, its transcript segments, and the tasks it produced — and is meant to be called before those rows are wiped. Best-effort.

ai/ragRetrieve.js

This module is the RAG retrieval side: given a question and the requesting user, it embeds the question, restricts the candidate embedding set to only what that user may see, ranks by cosine similarity, and returns the top chunks. It reads no API keys directly (embedding is delegated to `embeddings.js`). Its crucial security property is that RBAC is applied to the candidate set **before** ranking, so a retrieved chunk can never be one the user couldn't otherwise access. Constants: `DEFAULT_TOP_K = 12`, `MIN_SCORE = 0.2` (weak matches dropped).

- **visibility(user)** — Parameters: user. Returns: a visibility descriptor. For managers/admins it returns `{ all: true }`. For employees it builds Sets of the conversation ids, meeting ids, and transcript-segment ids the user participates in (via three DB queries), returning `{ all: false, convos, meetings, segments }`.
- **canSee(row, user, vis)** — Parameters: an embedding row, the user, and the visibility descriptor. Returns: boolean. It returns true immediately when `vis.all` is set; otherwise it switches on the row's source type — a task is visible only if its `ref_user_id` is the user, chat only if the convo is in the user's set, meetings and segments only if their id is in the respective set, and anything else is hidden.
- **retrieve(query, user, options)** (exported, async) — Parameters: the query string, the user, and an options object (topK defaulting to 12, and an optional types allowlist). Returns: `{ hits, used }` where each hit is `{ source_type, source_id, text, score }` and used indicates RAG actually ran. It returns an empty, unused result when embeddings are off or nothing is indexed for the org. It embeds the query (returning empty on embed failure), computes the user's visibility, loads all org embedding rows, and for each row skips those failing the optional type filter or the `canSee` check, computes cosine similarity, drops scores below `MIN_SCORE`, sorts descending, and returns the top-K hits.

ai/rules.js

This module is the offline, zero-dependency, rule-based multilingual meeting analyzer — the fallback when no AI key is configured. It uses **no external provider or model and reads no environment variables**; it handles English plus romanized/script Hindi and Telugu and code-mixed lines using hand-built lexicons and heuristics. It parses speaker turns, splits sentences, classifies action items vs. status updates, resolves ownership, and builds a summary. It also exports `detectLanguages`, reused by all the AI provider modules.

- **Lexicon constants** — `ACTION_CUES` (action/assignment verbs in En/Hi/Te, romanized and script), `COMMITMENT_CUES` (first-person "I will do it" markers implying the speaker is the assignee), `PRIORITY_RULES` (cue lists mapping to Critical/High/Low), `DECISION_CUES`, `BLOCKER_CUES`, `RISK_CUES`, `FOLLOWUP_CUES` (signal lexicons), `ROLE_WORDS` (job titles that may appear in speaker labels), `DONE_VERBS` (past-tense

completion verbs marking status updates, not tasks), FUTURE_CUES (forward-looking modals that keep a sentence a real action item), and CASUAL_RE (a regex of greetings/pleasantries to ignore).

- **ownershipScore(kind)** — Parameters: a qualitative ownership kind (vocative, self, matched, low, or default). Returns: { confidence, reasoning } mapping the kind to a numeric confidence (90/85/80/40/25) and an explanatory sentence.
- **detectLanguages(text)** (exported) — Parameters: text. Returns: an array of detected language codes. It adds en for Latin letters, te for Telugu script, hi for Devanagari, and additionally uses romanized-word heuristics to add hi or te.
- **containsAny(haystack, cues)** — Parameters: a string and a cue list. Returns: the first cue found (case-insensitive substring) or undefined. A generic cue-matcher used across the signal detectors.
- **detectPriority(text)** — Parameters: text. Returns: a priority string. It checks the PRIORITY_RULES in order and returns the first matching priority, defaulting to Medium.
- **isRoleWord(s)** — Parameters: a token. Returns: boolean — true if it equals or contains a known role word. Helps distinguish a person's name from a job title in a speaker label.
- **personFromLabel(label)** — Parameters: a speaker label. Returns: the human name. It splits the part inside parentheses from the part outside and, using isRoleWord, picks whichever is the actual name (handling both "Manager (Rahul)" and "Priya (Business Analyst)" forms).
- **parseSegments(transcript)** (exported) — Parameters: the transcript. Returns: an ordered array of { seq, speaker, text, language } segments. It splits into non-empty lines, matches a "Speaker: text" pattern (allowing parenthesized role labels), attaches continuation lines to the previous speaker, and detects each segment's language.
- **splitSentences(text)** — Parameters: text. Returns: an array of trimmed sentences, split on sentence-ending punctuation (including the Devanagari danda ।) or newlines.
- **detectVocative(sentence, knownNames)** — Parameters: a sentence and the known-name list. Returns: the addressed person's name or null. It first looks for a name-like token before a leading comma/colon ("Munidhar, ..."), accepting a known participant or a plausible candidate; otherwise it searches for any known name appearing anywhere in the sentence.
- **cleanTitle(sentence, vocative)** — Parameters: a sentence and an optional vocative name. Returns: a cleaned imperative title. It strips the leading vocative, removes up to three layers of conversational filler prefixes ("So I will..."), collapses whitespace, drops trailing punctuation, capitalizes, and truncates over-long titles to ~117 chars with an ellipsis.
- **escapeRe(s)** — Parameters: a string. Returns: the string with regex metacharacters escaped, for safe dynamic regex building.
- **findAssigneeIn(text, knownNames)** — Parameters: text and known names. Returns: the earliest-occurring known participant (matched on first name) or null. It scans each known name's first name and keeps the one appearing earliest in the text.

- **clauseToTitle(text, assignee)** — Parameters: an assignment clause and the assignee. Returns: a clean title. It strips the "assign [the] [task] to " directive and the assignee's name, drops trailing connectors ("and"/"aur"/"mariyu"/"మరియు"), and runs cleanTitle.
- **splitMultiAssignment(sentence, knownNames)** — Parameters: a sentence and known names. Returns: an array of { assignee, title, quote } when the sentence contains 2+ distinct "assign ... to " directives to different people, else null. It finds each "assign to" anchor, slices out each clause, resolves its assignee, and only returns a split when at least two distinct assignees result.
- **analyzeTranscript(transcript, opts)** (exported) — Parameters: the transcript and opts (meetingDate, knownNames). Returns: { engine: 'rule-based', segments, tasks, detected_languages, participants, summary }. It parses segments, builds the participant set (known names plus speakers), then for each sentence collects decision/blocker/risk/follow-up signals and evaluates action/commitment/future cues and any vocative. It filters out questions, casual greetings (unless someone is addressed), first-person or passive completion reports and meta-goals (unless a future modal makes them real). Sentences that qualify as tasks are emitted: multi-assignment sentences yield one task per person; otherwise it resolves ownership (vocative → self-commitment → unresolved/low), parses the due date, detects priority (sentence first, then the whole turn), and pushes a fully-populated task. Finally it computes detected languages and builds the summary.
- **buildSummary({...})** — Parameters: an object with segments, tasks, decisions, blockers, risks, followups, and participants. Returns: the summary object. It splits tasks into assigned vs. needs-confirmation, composes an executive-summary sentence with counts, and maps each signal list into its summary field (key_decisions, action_items, risks, blockers, follow_ups, assigned_tasks with "title → assignee", and unassigned_tasks).

ai/transcribe.js

This module is server-side speech-to-text with automatic language detection, tuned for Telugu/Hindi/English and code-mixing. Providers: **Sarvam AI (saarika) is primary**, with **OpenAI and Groq Whisper** as alternatives, selected by TRANSCRIPTION_PROVIDER (default none). It relies on Node 18+ global fetch/FormData/Blob. Environment variables: TRANSCRIPTION_PROVIDER, SARVAM_API_KEY, SARVAM_MODEL (default saarika:v2.5), SARVAM_LANGUAGE (default en-IN), OPENAI_API_KEY, OPENAI_TRANSCRIBE_MODEL (default gpt-4o-transcribe), GROQ_API_KEY, GROQ_TRANSCRIBE_MODEL (default whisper-large-v3).

- **transcribeAudio(buffer, filename, mimetype, opts)** (exported, async) — Parameters: the audio buffer, a filename, a mimetype, and opts (which may include a provider override and a prompt). Returns: { text, language }. It resolves the provider from opts.provider or TRANSCRIPTION_PROVIDER; if none it throws a NO_PROVIDER-coded error with setup guidance. It dispatches to sarvam for Sarvam, or whisper for openai/groq, and throws on an unknown provider name. The opts.provider override lets long file uploads use OpenAI Whisper even when the live recorder is set to Sarvam (whose instant endpoint caps audio at 30s).
- **sarvam(buffer, filename, mimetype)** — Parameters: the audio and its metadata. Returns: { text, language }. It requires SARVAM_API_KEY, builds a multipart form with the file, the model, and a language_code (default en-IN; can be set to unknown to

re-enable full auto-detect), POSTs to the Sarvam speech-to-text endpoint with the subscription-key header, throws on error, and returns the transcript and detected language code. Locking the language prevents mis-transcription into other Indian languages.

- **whisper(buffer, filename, mimetype, provider, opts)** — Parameters: the audio, its metadata, the provider (openai or groq), and opts. Returns: { text, language }. It picks the key, base URL, and model per provider, builds a multipart form (file, model, response_format: json, and — if opts.prompt is given — the last ~450 chars of prior context to keep names/spelling consistent across live segments), POSTs to the audio-transcriptions endpoint with a Bearer token, throws on error, and returns the text and language.

ai/usage.js

This module handles per-organization usage and cost tracking for all external AI/transcription API calls. It uses **no AI provider itself**; it logs every provider call into the `usage_events` table with token counts and an estimated USD cost derived from hard-coded public list prices, so the super-admin can see per-org consumption. It reads no environment variables — pricing lives in in-file tables. Costs are explicitly best-effort estimates, never an invoice.

- **Pricing tables** — `TOKEN_PRICES` maps each provider (anthropic, openai, openrouter, groq) to per-model input/output prices per 1M tokens, with a '*' default per provider. `PER_CALL_PRICES` gives flat per-call estimates for audio-billed providers (sarvam ~0.003, groq 0.001).
- **estimateTokens(text)** (exported) — Parameters: text. Returns: a rough token count (`ceil(length / 4)`), used when a provider does not return exact usage.
- **priceFor(provider, model)** — Parameters: a provider and model name. Returns: the { in, out } price object or null. It looks up the provider's table, finds the first keyed substring the model name contains, and otherwise returns the provider's '*' default.
- **estimateCost({ provider, model, inputTokens, outputTokens })** (exported) — Parameters: a usage descriptor. Returns: an estimated USD cost. It starts from any flat per-call price, then adds the token-based cost (input and output tokens times their per-million rates) when a price row is found.
- **usageForOrg(orgId, { since })** (exported) — Parameters: an org id and an optional since ISO lower-bound. Returns: { total, by_provider, by_feature }. It runs three aggregate SQL queries — a grand total (call count, token sums, cost sum), a breakdown grouped by provider, and one grouped by feature — all optionally windowed by `created_at >= since`.
- **recordUsage({ orgId, userId, provider, feature, model, inputTokens, outputTokens })** (exported) — Parameters: a usage event. Returns: nothing. It never throws (usage tracking must not break a feature). It ignores calls missing org/provider/feature, rounds and floors token counts to non-negative integers, computes the estimated cost, and inserts a row into `usage_events` (id, org, user, provider, feature, model, token counts, total, cost, timestamp). Any DB error is caught and logged.

ai/voiceCommand.js

This module is the voice-COMMAND interpreter — the brain behind hands-free control ("hey BTM"). It converts one spoken utterance plus the running conversation into a single structured intent (create task, change status, reassign, set priority/due date, navigate/filter, answer, or clarify). It **reuses the exact provider chain and helpers from voiceTask.js: OpenRouter → Claude → OpenAI**, so the model names/keys are those defined there (OPENROUTER_MODEL default google/gemini-2.5-flash, ANTHROPIC_MODEL default claude-opus-4-8, OPENAI_MODEL default gpt-4o-mini). It gives the model a compact, RBAC-scoped snapshot so it can resolve references like "the logo task" to a real id, and it can only act within what the caller may already see. Env vars checked here: OPENROUTER_API_KEY, ANTHROPIC_API_KEY, OPENAI_API_KEY.

- **Constants** — INTENTS (the eight allowed intents), STATUSES, PRIORITIES, and NAV_TARGETS (allowed navigation targets), all used to validate the model's output.
- **systemPrompt(user)** — Parameters: user. Returns: the system prompt. **Summary:** it casts the model as "BTM," the hands-free voice controller inside SmartTask, that must convert the latest utterance (using conversation context) into exactly one intent, always writing task text and the spoken "say" reply in English regardless of the spoken language. It injects a role-specific scope note (employees can only view/act on their own work), today's date, and the user's name, then describes each of the eight intents and its required fields (create_task, update_status, assign_task, set_priority, set_due_date, navigate with its target/person/status/priority/query sub-object, answer, and clarify). It explains task resolution (pick a task_id from the snapshot by matching the spoken description, and use clarify when ambiguous or not found), requires "say" to be a short natural confirmation, and mandates a specific flat JSON output object.
- **snapshot(tasks, users, user)** — Parameters: the task list, the user roster, and the requesting user. Returns: the RBAC-scoped snapshot string. It renders up to 120 tasks as compact id/title/owner/status/priority (plus overdue-flagged due date) lines, and appends an assignable-people roster only for non-employees (employees don't get the roster because they can't assign to others here).
- **historyBlock(history)** — Parameters: the conversation history. Returns: a "CONVERSATION SO FAR" block or empty string. It keeps the last 6 well-formed user/ai turns and labels them "User:"/"BTM:".
- **interpretCommand(transcript, { user, tasks, users, history, onUsage })** (exported, async) — Parameters: the transcript and a context object. Returns: a fully validated, normalized intent object (intent, say, task_id, title, description, assignee_name, priority, status, due_date_raw, and a navigate sub-object). It throws if no provider is configured, builds the system prompt and the user message (snapshot + history + "USER JUST SAID"), then tries each configured provider in order (OpenRouter → Claude → OpenAI) via the imported callOpenRouter/callClaude/callOpenAI, falling through on error. It parses the JSON, validates the intent against the allowed set (defaulting to "clarify"), and coerces/whitelists every field (priority/status against their allowed lists, navigate.target against NAV_TARGETS, etc.). It re-exports STATUSES and PRIORITIES.

ai/voiceSearch.js

This module turns a spoken task-SEARCH request into structured search filters. The speaker may use any language and names may arrive in non-Latin script, so it translates/transliterates everything to English/Latin to match the English task and employee data, splitting the request into free-text keywords, an optional named person, and an optional status/priority. It **reuses the same provider chain and JSON parsing as voiceTask.js (OpenRouter → Claude → OpenAI)** and re-exports its hasLLM. Env vars checked: OPENROUTER_API_KEY, ANTHROPIC_API_KEY, OPENAI_API_KEY (models default as in voiceTask.js).

- **SYSTEM_PROMPT** (constant) — **Summary:** it tells the model to convert a spoken search request (in any of English/Hindi/Kannada/Telugu/Tamil or a code-mixed blend, with names possibly in non-Latin script) into structured filters, always outputting English/Latin. It defines four extraction fields — query (English topic keywords, empty when only a person or status/priority was named), person (the employee, transliterated and matched to the provided team list when one clearly fits, else null), status, and priority — and rules forbidding invented values and instructing that a name-only request leaves query empty so all of that person's tasks are returned. It requires a specific JSON object with no markdown.
- **buildUserMsg(transcript, users)** — Parameters: the transcript and the searchable team members. Returns: the user message string. It lists team names (with roles) so the model can match/transliterate the spoken person, then appends the spoken search request.
- **interpretVoiceSearch(transcript, { users, onUsage })** (exported, async) — Parameters: the transcript and a context object. Returns: { query, person, status, priority }. It throws if no provider is configured, builds the user message, and tries OpenRouter → Claude → OpenAI in order (falling through on error). It parses the JSON and coerces the fields: query to a trimmed string, person to a trimmed string or null, and status/priority validated against the allowed lists (else null).

ai/voiceTask.js

This module turns a single spoken sentence into structured task fields (title, description, assignee_name, priority, due_date_raw) and is the shared home of the voice provider chain and HTTP callers reused by voiceCommand.js and voiceSearch.js. Its provider chain mirrors the meeting extractor: **OpenRouter first (OPENROUTER_API_KEY, OPENROUTER_MODEL default google/gemini-2.5-flash), then Claude (ANTHROPIC_API_KEY, ANTHROPIC_MODEL default claude-opus-4-8), then OpenAI (OPENAI_API_KEY, OPENAI_MODEL default gpt-4o-mini)**. When no key is set the caller falls back to using the raw transcript as the title. Additional env vars used in the OpenRouter caller: OPENROUTER_SITE_URL and OPENROUTER_APP_NAME (attribution headers, defaulting to http://localhost / SmartTask AI).

- **hasLLM()** (exported) — Parameters: none. Returns: boolean — true if any of the three provider keys is set.
- **SYSTEM_PROMPT** (constant) — **Summary:** it casts the model as a task-intake assistant hearing one dictated task in English/Hindi/Telugu or a code-mixed blend, and defines five fields to extract: title (short imperative English, ≤8 words, core action only), description (always a 1–2 sentence English explanation, never empty and never a verbatim repeat of the title), assignee_name (the person as spoken, matched to the

team list when clear, else null), priority (Critical/High/Medium/Low inferred from urgency, default Medium), and due_date_raw (the natural-language deadline exactly as spoken, or null). It stresses precision (invent nothing), always translating title/description to English, and ignoring filler/self-corrections to capture the final intent, ending with the required JSON shape.

- **buildUserMsg(transcript, users)** — Parameters: the transcript and the assignable team. Returns: the user message string listing team names (with roles) followed by the spoken task.
- **parseJson(text)** (exported) — Parameters: raw model text. Returns: the parsed object or null. It slices from the first { to the last } and JSON-parses, returning null on any failure. This is the shared parser used by all three voice modules.
- **callOpenRouter(system, userMsg, onUsage)** (exported, async) — Parameters: the system prompt, user message, and optional usage callback. Returns: the model's reply text. It reads OPENROUTER_MODEL (default google/gemini-2.5-flash), POSTs to OpenRouter's chat completions with response_format: json_object, temperature: 0.2, and attribution headers, throws on error, reports usage as { provider: 'openrouter', model, inputTokens: prompt_tokens, outputTokens: completion_tokens }, and returns the message content.
- **callClaude(system, userMsg, onUsage)** (exported, async) — Parameters: same shape. Returns: the concatenated reply text. It reads ANTHROPIC_MODEL (default claude-opus-4-8), POSTs to the Anthropic Messages API with max_tokens: 512, throws on error, and reports usage as { provider: 'anthropic', model, inputTokens: input_tokens, outputTokens: output_tokens }.
- **callOpenAI(system, userMsg, onUsage)** (exported, async) — Parameters: same shape. Returns: the reply content. It reads OPENAI_MODEL (default gpt-4o-mini), POSTs to OpenAI chat completions with response_format: json_object, temperature: 0.2, max_tokens: 512, throws on error, and reports usage as { provider: 'openai', model, inputTokens: prompt_tokens, outputTokens: completion_tokens }. These three callers are the shared HTTP layer for all voice features.
- **parseSpokenTask(transcript, { users, onUsage })** (exported, async) — Parameters: the transcript and a context object. Returns: { title, description, assignee_name, priority, due_date_raw }. It throws if no provider is configured, builds the user message, and tries OpenRouter → Claude → OpenAI in turn (falling through on error so one out-of-credits provider doesn't break voice). It parses the JSON and throws on an empty result or missing title, then returns the coerced fields — title and description trimmed, assignee_name trimmed or null, priority validated against the allowed list (default Medium), and due_date_raw trimmed or null.

Cross-cutting notes. Two distinct provider chains run through these files: the **meeting-extraction and voice features prioritize OpenRouter → Claude → OpenAI → (rules)**, while the **conversational assistant (assistantChat.js) uses only Claude → OpenAI**, both ultimately backstopped by the offline rule engines (assistant.js, rules.js) so every feature degrades gracefully when keys are missing or a provider errors. **Usage/cost tracking** is uniform: the HTTP caller functions accept an onUsage callback and report { provider, model, inputTokens, outputTokens } pulled from each provider's usage payload; usage.js turns those into usage_events rows with an estimated USD cost from its in-file price tables (per-token for LLMs, flat per-call for audio providers), aggregated per org by usageForOrg.

Part III — Frontend: App Shell & Pages (client/src/)

The client is a Vite + React 18 (TypeScript) single-page application wrapped by Capacitor for an Android build. It talks to an Express backend over a small `fetch` wrapper, uses React Router for navigation, and layers voice/AI features (task voice entry, live meeting transcription, a hands-free assistant) on top of a role-aware task/meeting/chat workspace.

10. Build & Shell Configuration

client/package.json

Defines the `smarttask-client` package (private, ESM). Scripts: `dev` (runs vite), `build` (`tsc -b && vite build`), `preview` (`vite preview`). Runtime dependencies include React 18.3, `react-dom`, `react-router-dom` 6.26, `onnxruntime-web` (on-device ML, e.g. wake-word/voice models), and a set of Capacitor 8 packages: `@capacitor/core`, `@capacitor/android`, `@capacitor/app`, `@capacitor/push-notifications`, `@capacitor-community/keep-awake`, and `@capacitor-community/text-to-speech`. Dev dependencies cover the Capacitor CLI/assets, the Vite React plugin, TypeScript 5.5, and Vite 8.

client/vite.config.ts

Vite configuration. Registers the React plugin and configures the dev server to run on port 5173 with a proxy: any request to `/api` is forwarded to the Express backend at `http://localhost:4000`, with `changeOrigin: true` and `ws: true` so the WebSocket upgrade for live meeting transcription (`/api/meetings/live`) and chat (`/api/chat/ws`) is proxied too.

client/capacitor.config.ts

Capacitor native shell configuration. `appId` is `io.smarttask.app`, `appName` is "Befach Task Manager", and `webDir` is `dist` (Vite's build output that Capacitor copies into the native app). The `server` block sets `androidScheme: 'http'` and `cleartext: true` so the `WebView` serves over plain http and can call an http backend without "mixed content" blocking; `android.allowMixedContent: true` reinforces this. Commented-out sections document how to enable native TTS and native Google Sign-In (GoogleAuth plugin with `serverClientId`).

client/index.html

The SPA host document. Sets the page title, the favicon (`/logo.png`), a mobile viewport meta, an empty `<div id="root">`, and loads `/src/main.tsx` as a module.

11. App Shell & Core Modules

main.tsx

Application entry point. Before first paint it restores the user's chosen wallpaper by calling `applyWallpaper(getWallpaperId())` (from `./lib/wallpaper`). It then mounts the app into `#root` with `ReactDOM.createRoot(...).render(...)`, wrapping `<App />` in three providers in order: `React.StrictMode`, `BrowserRouter` (router context), and `AuthProvider` (auth context). It imports the global `./styles.css`. There are no components or functions declared here beyond this bootstrap.

App.tsx

Defines the entire routing table, the authenticated app shell (sidebar + topbar layout), route protection, the Android hardware-back integration, and the email-verification banner. It imports every page and several shared components (`NotificationBell`, `ProfileModal`, `VoiceAssistant`, `ToastHost`, `ConfirmHost`).

Module-level constants:

- **Icon** — a small functional component rendering an 18×18 SVG with `stroke="currentColor"`, used as a wrapper so nav icons inherit color (turning white when active). Props: `{ children }`.
- **ICONS** — a frozen map of pre-built SVG icons: `dashboard`, `tasks`, `mytasks`, `chats`, `meetings`, `assistant`, `admin`, `platform`.
- **NAV** — the sidebar navigation model. Each entry has `to`, `label`, `icon`, `roles` (which roles may see it), and optional `teamOnly` (hidden in a personal/solo workspace) or `platformOnly` (only for platform admins). Entries: `My Tasks (/my-tasks, manager)`, `Dashboard (/)`, `Tasks (/tasks)`, `Chats (/chats, teamOnly)`, `Meetings (/meetings, manager)`, `AI Assistant (/assistant, manager)`, `Administration (/admin, manager, teamOnly)`, `Platform (/platform, platformOnly)`.
- **TITLES** — maps a base path to a `{ t (title), s (subtitle) }` pair shown in the topbar header.

Component `Layout({ children })` — the authenticated shell. Uses `useAuth()` for user and logout. State: `showProfile` (profile modal visibility), `open` (mobile sidebar open state), `chatUnread` (unread chat badge count). An effect polls `GET /chat/unread` immediately and every 10 seconds, updating `chatUnread`; it also listens for a `window chat-unread-changed` event to refresh on demand, cleaning up the interval and listener on unmount. It computes base (first path segment) and picks meta from `TITLES`. If there is no user it renders `null`. Otherwise it renders: an `<aside>` sidebar with brand logo, a filtered `<nav>` (filters by `platformOnly/roles/personal-workspace` rules, renders `NavLinks`, and shows a nav-badge on Chats when `chatUnread > 0`), a sidebar-user block (avatar button opening the profile modal, name/role, and a logout button), and a Privacy Policy link. It also renders a mobile backdrop, the main region with a topbar (menu toggle, page title/subtitle, and `NotificationBell` keyed by `user.id`), the page children wrapped by a `VerifyEmailBanner`, the `ProfileModal` (when `showProfile`), and the always-mounted `VoiceAssistant`.

Component `VerifyEmailBanner()` — a dismissible, non-blocking banner for unverified users. Uses `useAuth()` for user. State: `dismissed`, `sent`. Returns `null` if there is no user, the email is verified, or it was dismissed. `resend` calls `POST /auth/resend-verification`

and sets sent. The UI shows the user's email, a "Resend link" button (or a "sent" confirmation), and a dismiss button.

Component Protected({ children, roles, platform }) — route guard. Uses `useAuth()` for user and loading. While loading it shows a centered spinner. If there is no user it redirects to `/login`. If `platform` is required but `user.platform_admin` is falsy, it redirects to `/`. If `roles` is given and the user's role is not included, it redirects to `/`. Otherwise it renders `<Layout>{children}</Layout>`.

Component Home() — renders `<Dashboard />` (which branches internally by role).

Hook useAndroidBackButton() — wires the Android hardware back button. Uses `useNavigate()` and `useLocation()`, keeping the current path in `pathRef`. An effect (only on native platforms) dynamically imports `@capacitor/app` and adds a `backButton` listener: it first calls `runBackHandlers()` (page-registered handlers) and returns if one consumed the press; else if not on `/` it calls `navigate(-1)`; else it calls `CapApp.exitApp()`. Cleans up the listener on unmount.

Component App() (default export) — top-level. Reads user from `useAuth()`, invokes `useAndroidBackButton()`, and renders `<ToastHost />`, `<ConfirmHost />`, and the `<Routes>` table.

Routing table:

Path	Element	Guard
<code>/login</code>	Login	Redirects to <code>/</code> if already logged in
<code>/signup</code>	Signup	Redirects to <code>/</code> if logged in
<code>/accept-invite</code>	AcceptInvite	Redirects to <code>/</code> if logged in
<code>/forgot-password</code>	ForgotPassword	Redirects to <code>/</code> if logged in
<code>/reset-password</code>	ResetPassword	Redirects to <code>/</code> if logged in
<code>/verify-email</code>	VerifyEmail	Public (no guard)
<code>/privacy</code>	PrivacyPolicy	Public
<code>/</code>	Home (Dashboard)	Protected (any authenticated)
<code>/my-tasks</code>	Tasks personal	Protected roles= <code>{['manager']}</code>
<code>/tasks</code>	Tasks	Protected
<code>/chats</code>	Chats	Protected
<code>/meetings</code>	Meetings	Protected roles= <code>{['manager']}</code>
<code>/meetings/:id</code>	MeetingDetail	Protected roles= <code>{['manager']}</code>
<code>/assistant</code>	Assistant	Protected roles= <code>{['manager']}</code>
<code>/admin</code>	Admin	Protected roles= <code>{['manager']}</code>
<code>/platform</code>	Platform	Protected platform
<code>*</code>	Redirect to <code>/</code>	—

api.ts

The central HTTP client: a thin `fetch` wrapper that injects the JWT, unwraps JSON, and throws on error. Also defines shared TypeScript types and avatar URL helpers.

Constants / config:

- `TOKEN_KEY = 'smarttask_token'` (localStorage key).
- `API_BASE` — from `import.meta.env.VITE_API_BASE` (trailing slash stripped); empty in dev (Vite proxies `/api`), set to the hosted backend for the Android build.

Exported helpers/functions:

- `wsUrl(path)` — builds a WebSocket URL against the backend. Uses `API_BASE` (or the current page origin when empty), replaces the leading `http` with `ws` (so `http`→`ws`, `https`→`wss`), and appends path.
- `getToken()` — returns the JWT from localStorage.
- `setToken(t)` — stores the token, or removes it when `t` is null.
- `request(path, opts)` (internal) — sets `content-type: application/json` plus any override headers, adds `Authorization: Bearer <token>` when present, fetches `${API_BASE}/api${path}` with `cache: 'no-store'`, parses the response text as JSON (or null), and throws `Error(data.error || 'Request failed (status)')` on a non-OK response; otherwise returns the parsed data.
- `uploadFile(path, file, field='file')` (internal) — builds a `FormData`, appends the file under `field`, POSTs it with the auth header (no `content-type` so the browser sets the multipart boundary), and unwraps/throws like `request`.
- `api` — the exported object with methods: `get(p)`, `post(p, body?)`, `put(p, body?)`, `patch(p, body?)`, `del(p)` (DELETE), and `upload(path, file, field)`. Bodies are JSON-stringified when provided.
- `userAvatarUrl(userId, ver?)` — builds `${API_BASE}/api/users/${userId}/avatar?token=...&v=...`, embedding the token in the query so plain `<img>` tags can load an authenticated avatar; `ver` (stored filename) busts the browser cache.
- `groupAvatarUrl(convId, ver?)` — same pattern for `${API_BASE}/api/chat/conversations/${convId}/avatar`.

Exported types:

- `Role = 'admin' | 'manager' | 'employee'`.
- `User` — `id`, `name`, `email`, `role`, `org_id`, plus optional `phone`, `department_id`, `avatar_color`, `avatar_file`, `aliases`, `preferred_language`, `email_verified`, `workspace_personal`, `platform_admin`.
- `Suggestion` — an AI-suggested task awaiting review: `id`, `meeting_id`, `title`, `description`, suggested-assignee fields (`suggested_assignee_id/name/color/raw`, `assignee_reasoning`), `confidence`, `priority`, `due_date`, `due_date_raw`, `source_quote`, `status` (`pending|approved|rejected|merged`), `created_task_id`.
- `Task` — `id`, `title`, `description`, `priority` (`Critical|High|Medium|Low`), `status`, `due_date`, `due_date_raw`, `progress`, `ownership_confidence`, `approval_status`, `source_quote`, related users (`assignee`, `assignedBy`, raw name fields), `project`, `meeting_id`, timestamps (`assigned_at`, `submitted_at`, `completed_at`, `created_at`, `updated_at`), `visible_to_manager`, `parent_task_id`, and nested subtasks, comments, dependencies, attachments.

auth.tsx

React context provider for authentication. Exposes the current user and the login/signup/logout flows, and drives push registration.

Types/interfaces: SignupInput (company, name, email, password, personal?), AcceptInviteInput (token, name, password), and AuthCtx (the context shape: user, loading, login, loginWithGoogle, signup, acceptInvite, logout, refresh).

useAuth() — exported hook returning the AuthCtx via useContext.

Component AuthProvider({ children }) — holds user and loading state. On mount, an effect: if there's no stored token it sets loading=false; otherwise it calls GET /auth/me, and on success sets the user and calls registerPush(), on failure clears the token, and finally sets loading=false.

Functions provided through context:

- login(email, password) — POST /auth/login, stores the returned token, sets the user, calls registerPush().
- loginWithGoogle(credential) — POST /auth/google with a Google ID token, stores token, sets user, registers push. (Backend is login-only, so unknown emails fail.)
- signup(input) — POST /auth/signup, stores token, sets user, registers push (creates the org + first account).
- acceptInvite(input) — POST /invites/accept, stores token, sets user, registers push.
- logout() — calls unregisterPush(), clears the token, clears the user.
- refresh() — GET /auth/me and updates the user (swallows errors).

googleAuth.ts

Google Sign-In helper providing one API over two implementations: web (Google Identity Services) and native (a Capacitor plugin). Both yield a Google ID token posted to POST /api/auth/google.

Config: GOOGLE_CLIENT_ID (from VITE_GOOGLE_CLIENT_ID, empty when unconfigured), googleEnabled() (true when the client id is set), isNativePlatform() (delegates to Capacitor.isNativePlatform()).

Web (GIS):

- loadGis() (internal) — loads the https://accounts.google.com/gsi/client script exactly once (memoized in gisPromise), resolving with window.google; rejects with a friendly error on load failure.
- renderGoogleButton(container, onCredential, onError?) — exported async function. If no client id, calls onError and returns a no-op cleanup. Otherwise loads GIS, calls google.accounts.id.initialize with the client id and a callback that fires onCredential(resp.credential) (or onError if cancelled), then renders Google's official button (outline, large, pill, "continue_with") into container. Returns a cleanup function that cancels the GIS prompt.

Native (Capacitor plugin):

- GoogleAuthPlugin interface (initialize, signIn) and a GoogleAuth proxy registered via registerPlugin('GoogleAuth') so the web bundle never imports the npm package.
- nativeGoogleSignIn() — exported async function. On first call it calls GoogleAuth.initialize() (guarded by nativeInit). Then calls

`GoogleAuth.signIn()`, throwing a friendly error if unavailable, and returns `result.authentication.idToken` (throwing if none is returned).

push.ts

Native push registration via FCM. No-ops on the website (only runs inside Capacitor). Module state: `started` (guard against double init) and `lastToken` (the current device token).

registerPush() — exported async. Returns early if already started or not native. Sets `started`. Dynamically imports `@capacitor/push-notifications`. It deletes the legacy `smarttask` notification channel and creates a fresh `smarttask_v2` channel (importance 5 → heads-up + default sound, visibility 1) — the id was bumped because Android freezes a channel's sound at creation time. It checks permissions and requests them if prompt-state; if not granted, it resets `started` and returns. It calls `PushNotifications.register()` and adds listeners: `registration` (saves the token to `lastToken` and posts it to `POST /notifications/register-device` with platform: `'android'`), `registrationError` (logs), and `pushNotificationActionPerformed` (on tap, navigates to `/chats` for `chat_message` data type, else `/tasks`). Any thrown error resets `started`.

unregisterPush() — exported async. No-ops off native. If a `lastToken` exists it posts to `POST /notifications/unregister-device`, then removes all push listeners, clears `lastToken`, and resets `started`.

back.ts

Android hardware "back" coordination — a small handler stack so pages can intercept the back button (e.g. to close a modal) before the app navigates or exits.

- `BackHandler` type — `() => boolean` (returns true if it consumed the press).
- `stack` — module-level array of handlers.
- `pushBackHandler(handler)` — pushes a handler and returns an `unregister` function that splices it out (designed for use as a `useEffect` cleanup).
- `runBackHandlers()` — iterates the stack most-recent-first, returning true at the first handler that consumes the press (faulty handlers that throw are ignored); returns false if none consume it.

report.ts

Builds a printable manager report and provides date-range presets. Dependency-free (the report is self-contained HTML opened in a new window).

- `ReportRange` type — `{ from, to, label }`.
- `downloadManagerReport(range, orgName='Befach Task Manager')` — exported async. Calls `GET /dashboards/report?from=...&to=...`. It formats the generation timestamp and the period, defines HTML-escaping (`esc`) and small builders (`kpi` for a stat tile, `rows` for table rows), then assembles a full styled HTML document: a header with org name + label, a KPI strip (created, completed, due, open, overdue), an "Activity by team member" table (from `d.workload`), "New tasks by priority" (`d.by_priority`), "New tasks by status" (`d.by_status`), "Completed in period" (`d.completed_tasks`), and "Overdue tasks — as of now" (`d.overdue_tasks`).

It opens a new window (alerting if pop-ups are blocked), writes the HTML, and calls `window.print()` after a short delay.

- `pad, ymd (internal)` — zero-pad and format a `Date` as `YYYY-MM-DD` in local time.
- `presetRange(kind)` — returns a `ReportRange` for `'daily'` (today→today), `'weekly'` (last 7 days), or `'monthly'` (from the 1st of the month → today).
- `todayYmd()` — today's local date as `YYYY-MM-DD`.

ui.tsx

Shared UI primitives, color maps, and formatting helpers used across pages.

Color/label maps: `PRIORITY_COLORS` (Critical/High/Medium/Low), `STATUS_COLORS` (To Do/In Progress/Blocked/In Review/Done/Reopened), `LANG_LABEL` (en/hi/te → localized names).

Components:

- `Badge({ children, color, soft })` — a pill badge; `soft` renders a translucent tint, otherwise a solid fill.
- `PriorityBadge({ p })` — a soft `Badge` colored by priority.
- `StatusBadge({ s })` — a soft `Badge` colored by status.
- `Avatar({ name, color, size=28, src })` — renders a circular photo when `src` loads, else initials (first letters of up to two name words) on a colored circle. State broken tracks image load failure; an effect resets broken whenever `src` changes so a new URL retries.
- `ConfidenceTag({ c })` — a warning badge for `'needs_confirmation'` or `'low'`, else null.
- `ConfidenceScore({ score })` — a 0–100 numeric AI-confidence meter (bar + percentage), colored green ≥ 70 , amber 40–69, red < 40 (value clamped/rounded).
- `EmptyState({ icon, title, hint, action })` — a friendly full-area placeholder with an optional icon, title, hint line, and call-to-action.
- `Stat({ label, value, accent, hint })` — a stat card with a colored value, label, and optional hint.
- `Bar({ value, max, color })` — a horizontal progress bar (value/max as a percentage).
- `Donut({ data, size=150, thickness=24, onSegmentClick })` — an SVG ring split into colored segments with the total drawn in the center over an "OPEN" caption. Zero-value slices are skipped; when `onSegmentClick` is provided, non-zero segments (and their tooltips) are clickable to drill in.

Helpers:

- `fmtDateTime(iso)` — formats an ISO string as e.g. "4 Jun 2026, 3:42 PM"; returns `'–'` for missing/invalid dates.
- `DUE_DAYS_BY_PRIORITY (internal)` — lead-time-in-days per priority (Critical 0, High 1, Medium 3, Low 5).
- `defaultDueDate(priority)` — returns the local `YYYY-MM-DD` that many days ahead, mirroring the server default (falls back to 3 days).
- `dueLabel(t)` — renders a due-date span: red "(overdue)" when the date is past, an italic quoted raw phrase when only `due_date_raw` exists, else a muted dash.

12. Pages (client/src/pages/)

pages/AcceptInvite.tsx

Public page reached from an emailed invite link; confirms the invite, then lets the invitee set a name + password to join the org. Local EyeIcon/EyeOffIcon SVGs power the password reveal toggle.

Component AcceptInvite() (default export) — uses `acceptInvite` from `useAuth()`, `useNavigate`, and reads `token` from the query string. State: `loading`, `info` (`{ email, role, org_name }`), `name`, `password`, `showPw`, `err`, `busy`. On mount, an effect (keyed by `token`): if there is no token it sets an error; otherwise it calls `GET /invites/lookup?token=...`, storing the `org/role/email` info or an "invalid or expired" error. `submit(e)` validates the password (≥ 8 chars, not a common password via `isCommonPassword`, and `passwordStrength(...).ok`), then calls `acceptInvite({ token, name, password })` and navigates to `/` on success. The UI: a branded card that shows a spinner while loading, an error + "Go to login" link when the invite is invalid, or the name + password fields (with a `PasswordStrength` meter) and a "JOIN TEAM" button when valid.

pages/Admin.tsx

The manager's Administration hub — a tabbed console for org overview, user management, audit log, and (conditionally) AI usage.

Component AllowedDomains() — manages which email domains may join the org. State: `domains` (string list), `input`, `loaded`, `saving`. On mount, `GET /users/meta/org` loads `allowed_domains`. `add()` normalizes/validates the typed domain (regex, strips a leading `@`) and appends it (toast on invalid). `save()` PATCHes `/users/meta/org` with the domains and toasts success/failure. UI: a card explaining the restriction, chips for each domain (with remove buttons), an input + "Add domain" + "Save".

Component Admin() (default export) — holds tab (`overview|users|audit|usage`), usage data, and `usageAllowed`. On mount it calls `GET /usage`; success reveals the AI Usage tab and stores the data, failure hides the tab. Renders a toolbar of tab buttons (Usage only when allowed) and the active panel: `Overview`, `AllowedDomains` + `UserManagement`, `Audit`, or `UsagePanel`.

Helpers `fmtUsd`, `fmtNum` — compact currency and number (k/M) formatting.

Types `UsageRow`, `UsageData` describe the usage payload.

Component UsagePanel({ data }) — this org's own AI usage. Shows a spinner if no data; otherwise KPI stats (AI spend, API calls, tokens) and two cards breaking usage down "By provider" and "By feature", plus a disclaimer that costs are estimates.

Component Overview() — calls `GET /dashboards/admin` on mount. Shows a spinner until loaded, then KPI stats (users, tasks, meetings, projects) and two cards: "Users by role" and "Tasks by status".

Component Audit() — calls `GET /dashboards/admin` on mount and takes `recent_audit`. Renders a table (When / Actor / Action / Entity / Detail) with the action shown as a badge; shows "No activity yet." when empty.

pages/Assistant.tsx

The AI Assistant chat page (manager-only): a conversation sidebar plus a chat pane where the user asks natural-language questions about tasks, with server-synced conversations and voice input.

Types `Msg` (`role: 'user' | 'ai', text, tasks?`) and `Convo` (`id, title, msgs, updated`). Constants: `MicIcon`, `GREETING` (the initial AI message), and `relTime(ts)` (relative "Xm/h/d ago" formatter).

Component `Assistant()` (**default export**) — the whole page. State includes `convo`s, `activeId`, `loading`, `input`, `busy`, `suggestions` (suggested prompts), `openId` (task drawer), `navOpen` (mobile history drawer), `collapsed` (sidebar collapse, persisted in `localStorage`), plus voice state `listening`, `transcribing` and the `MediaRecorder/stream/chunk` refs. `canRecord` checks device recording support.

Functions:

- `releaseMic()` — stops the recorder/stream/tracks; an effect calls it on unmount.
- `transcribe(blob)` — POSTs the recorded audio to `POST /api/tasks/transcribe`, appends the returned text into the input box (so the user can review/edit), toasting on empty/failed transcription.
- `toggleMic()` — starts/stops recording via `getUserMedia + MediaRecorder`; on stop it builds a blob and calls `transcribe`.
- `setSidebar(c)` — sets collapse state and persists it.
- `active/msgs` — memoized current conversation and its messages.
- `migrateLocal(serverConvo`s) — one-time migration of any chats saved in the old `localStorage` store into the server (`POST /assistant/conversations`), then clears the local key.
- Load effect — on `mount/user.id` change: `GET /assistant/conversations`, runs `migrateLocal`, and if none exist creates a "New chat" (`POST /assistant/conversations`); on failure falls back to a purely-local conversation.
- Suggestions effect — `GET /assistant/suggestions`.
- Auto-scroll effect — scrolls the log to the bottom on new messages / `busy` changes.
- `patchActive(fn)` — updates the active conversation in state.
- `startNew()` — creates a new server conversation and selects it.
- `deleteConvo(cid, e)` — `DELETE /assistant/conversations/:id`, updates local state and selection (creating a new chat if the last one was deleted).
- `send(q)` — the core query flow: appends the user message optimistically, derives a title from the first user message, sends `POST /assistant/query` with `{ query, history }` (last 8 messages), appends the AI answer (and any returned tasks), and persists the conversation via `PUT /assistant/conversations/:id`.

UI: a chat-history sidebar (title, "New chat", conversation list with relative time and delete buttons), and a chat-pane with a mobile bar, the message log (user/AI bubbles; AI messages may render an embedded clickable task table opening `TaskDrawer`), a suggestions row, and the input row with a mic button and Send. Renders `TaskDrawer` when `openId` is set.

pages/Chats.tsx

A full WhatsApp-style team messaging page: conversation list, real-time messaging over `WebSocket`, replies, reactions, edits, deletes, stars, forwarding, file/image attachments, typing indicators, presence, read receipts, and group management.

Interfaces Member, Conversation (direct/group with last-message/unread/mute/pin metadata), Reaction, ReplyPreview, ChatFile, Msg (full message shape including reactions, starred, seen, deleted, uploading), and OrgUser. Constants: EMOJIS, MAX_FILE (15 MB), `fileUrl(m, download)` (builds `/api/chat/file/:id?token=...`). Helpers: `relTime`, `fmtSize`, `fmtTime`, `dayLabel`, `lastSeenLabel`.

Component PresenceAvatar — an Avatar with an optional green "online" dot.

Component GroupAvatar — a group photo (with broken-image fallback to a # tile).

Component Chats() (default export) — the page. Extensive state: `convo`s, `activeId`, `messages`, `composer input`, `sidebar search`, `in-conversation inSearch/inSearchOpen`, `replyTo`, `editing`, `loading`, `busy`, `menuId` (open message menu), `reactFor` (open reaction picker), `typingName`, `modal toggles` (`showNew`, `showInfo`, `showStarred`, `forwardMsg`), `online` (Set of user ids), `lastSeen map`, `threadLastRead`, and `convoMenu`. Refs: `logRef`, `fileRef`, `wsRef` (WebSocket), `activeIdRef`, `typing timers`.

Key functions:

- `pingNav()` — dispatches `chat-unread-changed` so the sidebar badge refreshes.
- `loadConvos()` — GET `/chat/conversations`.
- `mergeIncoming(m)` — merges a message in, reconciling an optimistic `tmp_` placeholder with the saved server message.
- `loadThread(cid)` — GET `/chat/conversations/:id`, loads messages, last-read time, updates `members/role/mute/pin`, and zeroes unread.
- Mount effect — loads conversations and GET `/chat/presence`.
- `activeId` effect — loads the thread and resets `reply/edit/search` state when a conversation is opened.
- Outside-click effect — closes open menus/pickers.
- Android back handler (`pushBackHandler`) — closes the top-most open layer (menu → forward modal → starred → info → new → in-search → open conversation) before leaving.
- WebSocket effect — connects to `wsUrl('/api/chat/ws?token=...')` with auto-reconnect; handles message types: `message` (merge + auto-mark-read + refresh list), `edit`, `reaction`, `delete` (for-all vs for-me), `read` (mark own messages seen), `typing`, `conversation` (membership changes / removed), `cleared`, `presence`, and `presence-list`.
- Polling effect — every 25 s refreshes conversations and the active thread.
- `sendTyping(isTyping)` — throttled typing signal over the WS.
- `send()` — optimistic send of a text message via POST `/chat/conversations/:id/messages` (or delegates to `saveEdit` when editing); rolls back on failure.
- `onPickFile(e)` — validates size, optimistically shows an "uploading" bubble, and POSTs multipart to `/api/chat/conversations/:id/upload` (with optional caption + reply).
- `saveEdit()` — PATCH `/chat/message/:id`.
- Message actions: `react` (POST `/chat/message/:id/reactions`), `toggleStar` (POST/DELETE `.../star`), `del` (DELETE `/chat/message/:id`, confirm dialog, optimistic), `copy` (clipboard), `share` (Web Share API / clipboard), `download` (anchor click), `startEdit`, `startReply`.
- Conversation actions: `setPref` (POST `/chat/conversations/:id/prefs` for `mute/pin`), `clearChat` (POST `.../clear`).
- Helpers: `senderName/senderColor` (resolve from members), `filteredConvos`, `shownMessages` (in-search filter), and `logItems` (interleaves day separators and an "Unread messages" divider).

UI: a sidebar (search, "New"/starred buttons, conversation list with avatars, previews, unread badges, per-conversation ... menu for pin/mute/clear) and a conversation pane (peer header with presence/typing/last-seen, in-chat search, the message log with bubbles/replies/reactions/files/images/tick receipts/message tools, a typing bubble, and the composer with reply/edit banners, attach button, and Send).

Sub-components:

- `ForwardModal({ message, convos, onClose, onDone })` — pick target conversations and `POST /chat/message/:id/forward`.
- `StarredModal({ onClose, onOpen })` — lists starred messages from `GET /chat/starred`, opening a message's conversation on click.
- `NewChatModal({ user, convos, onClose, onOpen })` — Direct/Group tabs; loads org users via `GET /chat/users`; startDirect posts `POST /chat/conversations {type:'direct'}`, createGroup posts `{type:'group', name, memberIds}`.
- `GroupInfo({ conv, user, onClose, onChangeed, onLeft })` — group admin: change photo (`POST .../avatar`), rename (`PATCH /chat/conversations/:id`), add members (`POST .../members`), remove member / leave / delete group (`DELETE` variants), with confirm dialogs.

pages/Dashboard.tsx

The role-aware home. `Dashboard` (default export) reads user and renders the employee dashboard (with a greeting) for employees, else the manager dashboard (`admin prop true` when the role isn't manager).

Component `Greeting({ name, style })` — a time-of-day greeting ("Good morning/afternoon/evening, ").

Hook `useDrawer()` — manages an opened task drawer: returns `openId`, `setOpenId`, a tick counter (bumped on change to force list refreshes), and a node (the `TaskDrawer` element or null).

Component `EmployeeDash()` — state data, error, plus `useDrawer` and `useNavigate`. `load()` calls `GET /dashboards/employee`; an effect reloads whenever the drawer tick changes. Renders an error `EmptyState` with `Retry`, skeletons while loading, else: a KPI strip (Assigned, Pending, Completed, Overdue [blinking when >0], Blocked) whose cards deep-link into `/tasks` with the appropriate `view/status` query; an "Upcoming deadlines" card (each row opens the drawer); and a "My work by status" card with `Bars` (plus a note when tasks need ownership confirmation).

Constants `KpiSvg`, `KPI_ICONS` — line-style KPI icons. **Component `Kpi(...)`** — a KPI card supporting an accent color, an optional blink, and click/keyboard activation.

Constants `RANGE_TABS` / `EPOCH_YMD` — the date-range tabs (Today/Week/Month/All) and the epoch used for "All tasks".

Component `ManagerDash({ admin, name })` — state data, error, active (range key, default monthly), to ("till" date for All), `datePopOpen`, downloading, plus `useDrawer/useNavigate` and a `datePopRef`. An effect closes the "till date" popup on outside click. It resolves the selected tab into a concrete range (using `presetRange` or the epoch/to window). `loadDash()` calls `GET /dashboards/manager?from=...&to=...`; an effect reloads on range/tick changes. `download()` calls `downloadManagerReport(range)`. UI: a toolbar (greeting, range segmented control with an "All tasks" date popup, and a

"Download report" button); a KPI strip (Total, Completed, Overdue [blink], Blocked) deep-linking into /tasks; and a body of cards — "Team workload" (horizontal bars per person, overload flagged, click routes to /tasks?assignee=), "Open by priority" (Donut + legend, segments route to /tasks?priority=...&view=active), "Tasks by status" (Bars, click routes to /tasks?status=), and "Overdue tasks" (table, rows open the drawer). Error and loading states render EmptyState/skeletons.

pages/ForgotPassword.tsx

Public page to request a password-reset link. State: email, sent, busy, err. submit(e) calls POST /auth/forgot-password with the email and sets sent (the server always responds OK to avoid revealing whether an email is registered). UI: a branded card showing either a success confirmation (with a "Back to login" link, noting a 24-hour expiry) or the email field + "SEND RESET LINK" button + a "Log in" link.

pages/Login.tsx

The login page, featuring an animated panda "mascot" that reacts to typing. Local SVG components: EyeIcon, EyeOffIcon, GoogleG (Google's four-color "G", used only on the native button).

Component Login() (default export) — uses login and loginWithGoogle from useAuth(). State: email, password, err, busy, focus flags emailFocused/pwFocused, showPw, shake (error animation), and a googleBtnRef. submit(e) calls login(email, password), shaking on error. finishGoogle(credential) calls loginWithGoogle (shared by both platforms). onNativeGoogle() calls nativeGoogleSignIn() then finishGoogle. An effect (web + configured) renders Google's official button into the ref via renderGoogleButton, cleaning up on unmount. It derives mascot props (covering, peeking, lookX, lookY) from focus/reveal state. UI: the mascot, a branded form with email + password (reveal toggle) fields, a "LOGIN" button, an "or" divider with either a native "Continue with Google" button or the GIS button slot (only when googleEnabled()), and footer links (Forgot password / Create your company / Privacy Policy), plus decorative paws.

Component Paw() — a decorative SVG paw. **Component Mascot({ covering, peeking, lookX, lookY, happy })** — the SVG panda whose eyes track the active field, paws cover the eyes while typing a hidden password (peeking when revealed), and mouth smiles while happy (busy).

pages/ResetPassword.tsx

Public page reached from the reset-password email link. Local EyeIcon/EyeOffIcon. Reads token from the query. State: password, showPw, done, busy, err. submit(e) validates the token presence and password strength (≥ 8 , not common, passwordStrength(...).ok), then calls POST /auth/reset-password with { token, password } and sets done. UI: either a success message with a "GO TO LOGIN" link, or the new-password field (with PasswordStrength meter and reveal toggle) + "UPDATE PASSWORD" button + a back-to-login link.

pages/Signup.tsx

Self-serve company (or personal) onboarding that creates the org + first account and logs straight in. Local CompanyIcon, PersonIcon, EyeIcon, EyeOffIcon.

Component Signup() (default export) — uses `signup` from `useAuth()` and `useNavigate`. State: `mode ('company' | 'personal')`, `company`, `name`, `email`, `password`, `showPw`, `err`, `busy`, `shake`; derived `personal`. `submit(e)` validates (company name required unless personal; password ≥ 8 , not common, strong), then calls `signup({ company, name, email, password, personal })` and navigates to `/`, shaking on error. UI: a branded form whose copy switches by mode, a two-button mode toggle (My company / Just me), a conditional Company-name field, Name, Work email, and Create-a-password fields (with reveal toggle and `PasswordStrength`), a submit button ("CREATE COMPANY"/"CREATE ACCOUNT"), and footer links (log in, Privacy Policy).

pages/VerifyEmail.tsx

Public page reached from the verify-email link; confirms the token on load. Reads token from the query, and uses `user/refresh` from `useAuth()`. State: `state ('working' | 'ok' | 'fail')`, `msg`, plus a `ran` ref to ensure the verification runs once. On mount, if there's no token it fails; otherwise it calls `POST /auth/verify-email` with `{ token }`, setting `ok` (and calling `refresh()` if logged in) or `fail` with the error message. UI: a branded card showing a spinner while working, a success box, or an error, with a "Continue to app" link.

pages/Meetings.tsx

The Meetings list plus the upload / live-recording / edit modals (manager-only actions). Handles transcript paste, full audio-file upload, and live recording via three transcription strategies.

Constants: `MEETING_TITLES` (preset titles) and `SAMPLE` (a mixed-language sample transcript).

Component MeetingTitleSelect({ value, onChange }) — a preset dropdown plus an "Other" option revealing a free-text input; an existing custom title opens as "Other".

Component Meetings() (default export) — uses `user` and `useNavigate`. State: `meetings`, `showUpload`, `showLive`, `editing`, `loaded`, `error`. `load()` calls `GET /meetings`. `del(m)` confirms then `DELETE /meetings/:id` and reloads. UI: a toolbar (meeting count; for managers, "Start meeting" and "Upload meeting"); a responsive grid of meeting cards (title, engine badge, date, summary snippet, detected-language tags, task/pending-review count, and Edit/Delete for managers) with skeleton, error, and empty states; and the three modals wired to reload/navigate.

Component EditMeetingModal({ meeting, onClose, onSave }) — edits title/date and `PATCH /meetings/:id`.

Component UploadModal({ onClose, onDone }) — state for `mode ('text' | 'audio')`, `title`, `description`, `date`, `participants`, `transcript`, `audioFile`, `provider`, `busy`, `err`. On mount it checks `GET /api/health` to learn the server transcription provider (drives whether audio is available). `uploadAudio()` POSTs multipart to `/api/meetings/audio`

(audio + title/description/date/participant_ids) and returns the new meeting id. process() either uploads the audio or posts a transcript via POST /meetings (with participant_ids), then calls onDone(id). UI: a source toggle, title/date, description, a ParticipantPicker, and either the audio file drop (with an inactive-STT warning) or the transcript textarea (with "Insert sample"), plus the "Analyze & extract tasks" action.

Constants/helpers: REC_LANGS (en/hi/te India locales), recordSegment(stream, ms) — records one self-contained audio Blob of a given length.

Component LiveMeetingModal({ defaultSpeaker, onClose, onDone }) — the live recorder. State: title, description, participants, computed date, speaker, lang, provider, mode ('auto' | 'browser'), recording, paused, transcribing, transcript, interim, seconds, busy, err, plus refs for the recognizer, WebSocket, PCM stream, media stream, and recording/paused/speaker/transcript mirrors. It detects the server provider from /api/health (preferring server STT when present) and manages a seconds timer, cleanup on unmount, native screen-keep-awake, and app-backgrounded → pause handling. Core functions:

- appendLine(text) / commitBrowserFinal(text) — append recognized speech as "Speaker: text" lines; the browser path de-duplicates growing-prefix finals on Android.
- uploadChunk(blob, prompt) — POST /api/meetings/transcribe (with prior text as a prompt for name/spelling consistency).
- startAuto() — AUTO segmented mode: records 12 s segments in a loop and transcribes each via uploadChunk; treats mic mute/end as an interruption.
- startSarvamStream() — SARVAM streaming mode: opens wsUrl('/api/meetings/live?token=...&language=...'), streams PCM16 via startPcmStream, and appends transcripts pushed back over the socket.
- startBrowser() — BROWSER mode: uses the Web Speech API (SpeechRecognition) with continuous/interim results, auto-restarting on end.
- beginCapture / teardownEngines / start / pauseRecording / resumeRecording / stop — lifecycle helpers spinning up or tearing down the correct engine, and pausing/resuming after interruptions.
- process() — stops, then POST /meetings with the transcript + participants and calls onDone(id). UI: title/speaker, description, participants, a recognition-mode toggle (with provider notes and a setup hint when no STT is configured), a language selector, live status (REC/PAUSED/timer), an interruption banner with Resume, Start/Stop/Resume controls, and an editable live-transcript textarea with an interim caption line, ending with the "Analyze & extract tasks" action.

pages/MeetingDetail.tsx

A single meeting's detail: executive summary, AI review queue, rejected suggestions, assigned tasks, structured summary sections, and a transcript tab — plus the manager Review & Assign modal.

Component AutoTextarea(props) — a textarea that auto-grows to fit its content (used for editing suggestion titles). **Constant SUMMARY_SECTIONS** — the ordered summary categories (Key Decisions, Action Items, Risks, Blockers, Follow-ups) with icons.

Component MeetingDetail() (default export) — reads id from the route. State: m (meeting), openId (task drawer), tab ('summary' | 'transcript'), review (modal), restoringId, restoreErr. load() calls GET /meetings/:id. restore(sid) calls POST /meetings/suggestions/:id/restore then reloads and opens the review screen. It

derives pending and rejected suggestion lists. UI: a back link, header (title, date, engine, language tags, tab buttons), optional description and participant chips; on the Summary tab a two-column layout — left: Executive Summary card, the pending "AI Suggested Tasks" review queue (each card shows title, priority, suggested assignee or a warning, confidence, due, reasoning, source quote, and a "Review & Assign Tasks" button), a "Rejected Suggestions" card with per-item "Restore & edit", and an "Assigned Tasks" table (rows open the drawer); right: the non-empty summary sections as lists. The Transcript tab lists speaker segments. Renders TaskDrawer and the ReviewAssignModal.

Constant PRIORITIES. Component ReviewAssignModal({ meeting, pending, onClose, onChange }) — the manager review screen. It seeds rows from the pending suggestions (pre-filling each due date from priority via defaultDueDate), each augmented with `_status` (pending|busy|assigned|rejected|merged), `_error`, `_showMerge`, `_mergeInto`. State also: `bulkBusy`, `bulkErr`; participants from the meeting. `set(i, patch)` updates a row. Functions:

- `approveAssign(i)` — requires an assignee, then PATCH `/meetings/suggestions/:id` (title/assignee/priority/due) and POST `/meetings/:mid/assign {ids:[id]}`, marking the row assigned and calling `onChange`.
- `reject(i)` — POST `/meetings/suggestions/:id/reject`.
- `restoreRow(i)` — POST `.../restore` to make a rejected/merged row editable again.
- `doMerge(i)` — POST `.../merge {into}` to fold one suggestion into another.
- `assignAllRemaining()` — batch: PATCHes every still-pending owned row, then a single POST `/meetings/:mid/assign` with all ids. It computes counts (done, assignedCount, remaining, pendingWithOwner, noOwnerCount). UI: acted-on rows collapse into a confirmation strip (with Restore for rejected/merged); active rows show the auto-growing title, a ConfidenceScore, assignee/priority/due controls, reasoning/quote, and Approve/Reject/Merge actions; a footer shows progress and an "Assign all remaining" button.

pages/Platform.tsx

The platform-admin (super-admin) console overseeing every organization on the platform.

Interfaces: Org, Member, UsageRow, Usage, OrgDetail. Exported helpers `fmtUsd`, `fmtNum`.

Component Platform() (default export) — state: `orgs`, `stats` (platform totals), `error`, `q` (search), and `detail` (org drill-down: null / loading / loaded). `load()` runs GET `/platform/stats` and GET `/platform/orgs` in parallel. `openOrg(id)` sets a loading detail then fetches GET `/platform/orgs/:id`. `removeOrg(o)` confirms a destructive delete then DELETE `/platform/orgs/:id` and reloads. UI: a KPI strip (Organizations, Total users, Total tasks, AI spend + calls), a search toolbar, and a table of orgs (name button opening detail, type badge, owner, users, tasks, AI usage, created/last-activity, and a Delete button), with error/loading/empty states, plus the OrgDetailModal.

Component OrgDetailModal({ state, onClose, onRetry, onChange }) — the drill-down. State: `access` (whether the org's admins may view their own usage) and `accessBusy`; an effect syncs `access` from the loaded data. `toggleAccess()` calls PATCH `/platform/orgs/:id/usage-access {enabled}`, updates local state, toasts, and calls `onChange`. UI: a modal showing "Tasks by status" (Bars), "AI / API usage" (totals + by-provider / by-feature breakdowns and the access checkbox), and a "Members" table (name with a "Super admin" badge, email, role, joined date), with error/loading fallbacks.

pages/PrivacyPolicy.tsx

A public, standalone privacy policy at /privacy (required for the Play listing, linked from login and the sidebar). Constants `EFFECTIVE_DATE` and `CONTACT_EMAIL`. The default-export component renders a static, self-contained document: a header with logo and effective date, then numbered sections covering information collected, how it's used, AI/third-party processors (naming Sarvam AI, OpenAI, Anthropic, OpenRouter, Groq), data sharing, retention, security, user rights, children's privacy, policy changes, and a mailto contact link, ending with a "Back to sign in" link. It makes no API calls.

pages/Tasks.tsx

The Tasks workspace, reused for both the shared /tasks view and the manager's private /my-tasks (via the `personal` prop). Provides list and board views, quick views, filtering, sorting, inline manager actions, a voice-capable search dialog, and a voice-capable new-task modal.

Module helpers/constants:

- `givenOf(t) / givenLabel(t)` — the date a task was given to its owner (`assigned_at` || `created_at`) and a label (" Assigned" / " Created"); used for grouping/ordering.
- `DUE_DAYS_BY_PRIORITY / dueDateForPriority(priority)` — mirror the server's `priority→due-date` default so the New Task form previews the real date.
- `AutoTextarea(props)` — an auto-growing textarea (title field).
- `SearchIcon, MicIcon` — line-art SVGs.
- Sorting: `SortKey` type; `PRIORITY_RANK, STATUS_RANK`; `cmpAsc(a, b, key)` — ascending comparator per column (unassigned/no-due sort last). `SORT_OPTIONS, SortGlyph`.

Component `SortMenu({ sortKey, onPick })` — a themed sort dropdown (replacing the unstyleable native select) that shows the active option, opens a menu with a check mark, and closes on outside-click/selection.

Component `SearchDialog({ initial, onApply, onClose })` — a search modal supporting typed or spoken queries. Type `SearchApply = { q?, assignee?, status?, priority? }`. State: `q`, `listening`, `parsing`, `voiceFilters` (assignee/status/priority resolved from voice), plus `recorder/stream/chunk` refs; uses `useEscape(onClose)`. `transcribe(blob)` POSTs audio to `POST /api/tasks/voice-search`, which returns a translated query plus resolved `assignee_id/status/priority` (e.g. "Shabbir's blocked tasks"); it fills `q` and `voiceFilters`. `toggleMic()` records/stops. `submit()` applies `{ q, ...voiceFilters }` and closes. UI: a search field, a "Speak to search" mic button, resolved voice-filter chips, and Cancel/Search.

Component `Tasks({ personal=false })` (default export) — the page. Reads user, and `searchParams/setSearchParams` (dashboard KPIs deep-link with `view/status/priority/assignee/task`). It seeds `quickView` from `?view` (mapping a `status=Done` deep-link to the Completed view). State: `tasks`, `users`, `openId` (task drawer, seeded from `?task`), `showNew`, `view ('list' | 'board')`, `searchOpen`, `quickView` (`active|overdue|today|completed`), `filters` (`q, priority, status, assignee`), `sort ({key, dir})`, `loadError`, and a `reqIdRef` to ignore out-of-order responses.

Functions/effects:

- `load()` — builds a query from filters (adds `mine=1` in personal mode) and calls `GET /tasks?...` , applying results only for the latest request.
- Effects: reload on filters/personal change; `GET /users` once; listen for a window `tasks-changed` event (fired by the voice assistant) to reload; open `?task=` in the drawer.
- `closeDrawer()` — closes the drawer and strips the `?task` param.
- Android back handler — closes the new-task modal / drawer first.
- Derived: `isManager` (non-employee and not personal), `narrowed` (any filter/non-default quick view active), `clearFilters()`, `todayStr`, `matchesQuick(t, key)` (active/overdue/today/completed predicate), `visibleTasks`, `QUICK_CHIPS`, `rowClass(t)` (urgency accent), `groupedByDay` (group by given-day, sort within a day by the active column, days newest-first), `sortedByPriority` (priority sort flattens across days), `dayHeading(day)` (Today/Yesterday/full date).
- Inline manager actions: `assign(taskId, userId)` (`PATCH /tasks/:id {assignee_id}`), `changePriority` (`PATCH /tasks/:id {priority}`), `markDone` (`POST /tasks/:id/approve {decision:'approved'}`), `moveStatus` (optimistic board move + `POST /tasks/:id/status {status}`).
- Renderers: `sortTh(label, key)` (sortable header), `renderRow(t)` (list row with title/confidence tag, priority [editable select for managers], status [with a Done-approve button for In Review or a tick to complete], assignee [avatar or inline assign select], due, and the given-time column), and `renderCompletedRow(t)` (the completed archive row with assigned + completed dates).

UI: a toolbar (search chip/icon opening `SearchDialog`, the mobile `SortMenu`, "New task"; desktop filter dropdowns for priority/status/assignee), a view toggle (List/Board) and quick-view chips with live counts, then either the board (`TaskBoard`), the completed table, or the sortable day-grouped/priority-flat list, with tailored empty/error states. Renders `TaskDrawer`, `NewTaskModal`, and `SearchDialog`.

Component `NewTaskModal({ users, personal, onClose, onCreate })` — create-task modal with AI voice entry. Uses `user`, `useEscape(onClose)`. `asPersonal` marks the task private; `isEmployee` defaults the owner to self (managers default to `Unassigned`). State: form (title, description, priority, `assignee_id`, `due_date`), `dueManual` (stop auto-syncing the date once set by hand), `busy`, plus voice state (listening, parsing, heard) and recorder refs; `setPriority` keeps the due date in step with priority unless manual. `save()` `POST /tasks` with the form (+ personal). Voice: `applyVoice(transcript)` `POSTs` to `POST /tasks/parse-voice` and merges the AI-extracted title/description/priority/due/assignee into the form; `transcribeAndApply(blob)` `POSTs` audio to `POST /api/tasks/transcribe`, stores what was heard, and calls `applyVoice`; `toggleMic()` records/stops. UI: an optional privacy note, a title area with a "Speak your task" mic button (showing recording/thinking states and the heard text) and the auto-growing title textarea, a description field, and a priority/assignee/due-date row (assignee hidden in personal mode; due date labeled "auto from priority" until edited), ending with `Cancel/Create`.

Part IV — Frontend: Components, Voice Pipeline, Libraries & Styles

This part covers the shared UI components (`client/src/components`), the voice-assistant pipeline (`client/src/voice`), the standalone utility libraries (`client/src/lib`), and a summary of the global stylesheet (`client/src/styles.css`). All files are TypeScript/React (`.tsx/.ts`).

13. Shared Components (`client/src/components/`)

ConfirmHost.tsx

Renders the application's single active confirmation dialog. It is the visual half of the promise-based confirm bus in `lib/confirm.ts`; it is meant to be mounted once near the app root and it subscribes to the confirm store, showing at most one dialog at a time. It replaces the browser's blocking `window.confirm()` with a themed, keyboard-navigable modal.

Component: ConfirmHost (default export, no props)

- **State:** `c` — the currently pending confirm request (`PendingConfirm | null`).
- **Effects:**
 1. On mount, calls `subscribeConfirm(setC)`; the returned unsubscribe function is the effect cleanup. This immediately pushes the current value and re-pushes on every change.
 2. Keyed on `c`: while a dialog is showing, installs a document keydown listener. Escape calls `resolveConfirm(false)` (cancel); Enter calls `resolveConfirm(true)` (confirm). Both call `preventDefault()`. The listener is removed on cleanup or when `c` changes.
- **UI behavior:** Returns `null` when no dialog is pending. Otherwise renders a modal-center overlay; clicking the overlay backdrop resolves `false`. The inner `.modal.confirm-modal` (`role="alertdialog"`, `aria-modal`) stops click propagation. It shows either a danger triangle icon (when `c.danger`) or an info circle icon (inline SVGs), an optional title, and the message. Two buttons: a Cancel button (`c.cancelText` or "Cancel") resolving `false`, and a confirm button (`c.confirmText` or "Confirm") resolving `true`, styled `btn-danger-solid` when `danger` else `btn-primary`, and `autoFocused`.

NotificationBell.tsx

A topbar bell button with an unread badge and a dropdown notification panel. It polls the backend for notifications, animates/plays a chime on new arrivals, lets the user mute the sound, and deep-links each notification to its related task drawer or the Chats page.

Local icon components:

- **BellIcon**(`{ size = 33 }`) — a 3D glossy round brand-orange bell button drawn as an SVG with a `linearGradient` (`#e8853c` → `#c5560f` → `#a3450b`), a dark rim circle, gradient body, a semi-transparent white top-gloss ellipse, and a white outline bell. `aria-hidden`.
- **BellLineIcon**(`{ size = 17 }`) — a line-style bell (`stroke` `currentColor`) for the panel's mute toggle.
- **BellOffIcon**(`{ size = 17 }`) — a line-style crossed-out bell for the muted state.

Constant ICON — a `Record<string, string>` mapping notification type values (`task_submitted`, `task_approved`, `task_reopened`, `task_assigned`, `task_comment`, `chat_message`) to emoji glyphs.

Interface Notif — `{ id, type, message, task_id?, read (number), created_at }`.

Component: NotificationBell (default export, no props)

- **Hooks/State:**

- `navigate` from `useNavigate` (`react-router`).
- `items: Notif[]` — the notification list.
- `unread: number` — unread count.
- `open: boolean` — panel open state.
- `vibrating: boolean` — drives the bell shake animation.
- `soundOn: boolean` — sound preference, initialized lazily from `localStorage.getItem('notifSound') !== 'off'` (on by default).
- **Refs:** `ref` (panel wrapper, for outside-click), `prevUnread` (previous unread count, null until first load so it doesn't shake on mount), `audioCtx` (a lazily created shared `AudioContext`), `soundOnRef` (mirrors `soundOn` so the once-created polling interval reads the current value, not a stale closure).

- **Function `playChime()`** — synthesizes a two-note "ding-dong" chime using the Web Audio API (no audio file). Resolves the `AudioContext` constructor (`AudioContext` or `webkitAudioContext`), lazily creates/reuses `audioCtx.current`, resumes it if suspended, then for two notes (880 Hz at 0s, 1174.7 Hz at 0.13s) creates a `sine OscillatorNode + GainNode`, ramps the gain up to 0.2 then exponentially down to near-zero over ~0.4s, and starts/stops each oscillator. Wrapped in `try/catch` to fail silently when `autoplay` is blocked or unsupported.
- **Effect (mirror):** keeps `soundOnRef.current` in sync with `soundOn`.
- **Function `load()`** — `GET /notifications`; sets `items` and `unread`. If `prevUnread.current` is non-null and `unread` increased, sets `vibrating` true for 800ms and plays the chime if `soundOnRef.current`. Updates `prevUnread`. Errors are swallowed.
- **Effects:**
 1. On mount: calls `load()` then `setInterval(load, 15000)` (15-second polling); clears interval on cleanup.
 2. On mount: installs a `document mousedown` listener to close the panel when clicking outside ref.
- **Function `toggle()`** — flips `open`. If opening and `unread > 0`, calls `POST /notifications/read-all`, sets `unread` to 0, and reloads.
- **Function `openNotif(n)`** — closes the panel; for a `chat_message` navigates to `/chats`, otherwise if `n.task_id` navigates to `/tasks?task=<id>`.
- **UI behavior:** The bell button shows `BellIcon` wrapped in a `bell-vibrate` span when vibrating and a red unread badge (capped at "9+"). When open, renders a floating `.card` panel with a header ("Notifications") and a mute toggle button (`aria-pressed={soundOn}`) that flips `soundOn`, persists it to `localStorage`, and previews the chime when turning it on. The list shows an empty state ("You're all caught up 🎉") or notification rows; actionable rows (chat messages or those with a `task_id`) get `role="button"`, tab focus, Enter/Space handling, and an `onClick` to `openNotif`. Each row shows the type emoji, message, and localized timestamp; unread rows get a tinted background.

ParticipantPicker.tsx

A meeting-attendee picker. Selected people appear as removable chips; the full employee list is hidden behind an " + Add members" toggle that reveals a filterable, scrollable dropdown. Supports pre-selecting everyone for a new meeting.

Component: `ParticipantPicker({ value, onChange, autoSelectAll? })`

- **Props:** `value: string[]` (selected user IDs), `onChange: (ids: string[]) => void`, `autoSelectAll?: boolean` (pre-add everyone once when the list loads).
- **State:** `users: User[]`, `q: string` (filter text), `open: boolean` (dropdown visibility). **Ref** seeded — ensures auto-select-all runs only once so the user can still remove people.
- **Effect (mount):** `GET /users`; sets `users`. If `autoSelectAll` and not yet seeded and `value` is empty, calls `onChange(list.map(u => u.id))`; marks `seeded.current = true`. Errors swallowed.
- **Derived:** `selected` = users whose IDs are in `value`. `available` (memoized on `users`, `q`, `value`) = users not selected, filtered by case-insensitive name/email match against the trimmed query.
- **Functions:** `add(id)` appends an ID to `value`; `remove(id)` removes it; `addAll()` selects all users and closes the dropdown.
- **UI behavior:** Renders selected attendees as chips (avatar + name + × remove button), or "No attendees yet." A row of controls: " + Add members" toggle (highlighted when open) and, when not everyone is selected, an "Add everyone" button. When open, a `.card` dropdown with a filter input (`autoFocus`), a scrollable list of available users (avatar, name, capitalized role, " + Add" button), an empty message ("Everyone has been added." / "No matching employees."), and a "Done" button.

PasswordStrength.tsx

A small visual password-strength meter shown live under a password field.

Constant COLORS — 5 hex colors indexed by score (red, red, amber, blue, green).

Component: `PasswordStrength({ password })`

- **Props:** `password: string`.
- **Behavior:** Returns null for an empty password. Otherwise calls `passwordStrength(password)` (from `lib/passwordStrength`) to get `{ score, label }`, picks `COLORS[score]`, and renders a `.pw-strength` container (`aria-live="polite"`) with a 4-segment bar (segments below score filled with the score color, others with `--border`) and the strength label colored to match. No state or effects.

ProfileModal.tsx

A self-service profile hub modal: change profile photo, edit name & preferred language, change password, pick an app wallpaper (live preview), and log out. Opens from the sidebar avatar.

Component: `ProfileModal({ onClose })`

- **Props:** onClose: () => void.
- **Hooks:** useAuth() → { user, refresh, logout }; useEscape(onClose) (Escape closes). Returns null if no user.
- **State/Refs:** fileInput ref (hidden file input); name, lang (default user.preferred_language || 'en'), curPw/newPw/confirmPw, wallpaper (default getWallpaperId()), busy: string | null (which section is saving), and msg — a tagged message { kind: 'ok'|'err', text, section: 'profile'|'password' } so an error appears next to the relevant action.
- **Function flash(kind, text, section='profile')** — sets msg and auto-clears after 3500ms.
- **Function onPhoto(e)** — handles file selection. Reads files[0], resets the input value, validates it is an image and ≤ 5 MB (else flashes an error). Sets busy='photo', builds a FormData with the file, adds a Bearer auth header from getToken(), and fetches POST \${API_BASE}/api/users/me/avatar. On failure throws the server error; on success calls refresh() and flashes success. Always clears busy.
- **Function saveProfile()** — validates non-empty name; sets busy='profile'; PATCH /users/me with { name, preferred_language }; refresh(); flashes result.
- **Function changePassword()** — validates new password ≥ 8 chars and matches confirmation (errors tagged 'password'); PATCH /users/me with { current_password, new_password }; clears the password fields; flashes result in the password section.
- **Function pickWallpaper(id)** — sets local wallpaper state and calls applyWallpaper(id) for instant live preview + persistence.
- **UI behavior:** A modal-center overlay (backdrop click closes) containing a .profile-modal. Header with title and × close. Body sections: **Identity** (an avatar-edit button opening the file picker, showing a "..." spinner while busy or "📷"; hidden file input accept="image/*"; name, email, role badge); a profile-scoped message; **Account details** (name input, preferred-language <select> populated from LANG_LABEL, "Save changes" button with spinner); **App wallpaper** (swatch grid from WALLPAPERS, active swatch marked with ✓); **Change password** (password-scoped message; current/new/confirm password inputs with proper autoComplete; "Update password" button disabled until current+new are filled); and a **Log out** button that closes then calls logout().

TaskBoard.tsx

A drag-and-drop Kanban board grouping tasks into one column per status. Dropping a card on another column moves the task via a callback (which calls the status API).

Constant COLUMNS — ['To Do', 'In Progress', 'Blocked', 'In Review', 'Done', 'Reopened'], mirroring the task lifecycle order.

Component: TaskBoard({ tasks, onOpen, onMove })

- **Props:** tasks: Task[]; onOpen: (id) => void (open a task); onMove: (id, status) => void (move a task to a new status).
- **State:** dragId: string | null (card being dragged), overCol: string | null (column hovered during drag).
- **Function drop(status)** — captures dragId, clears drag/over state, and if the dragged task exists and its status differs, calls onMove(id, status).
- **UI behavior:** Renders a .board. For each column: filters tasks by status; wires HTML5 drag events — onDragOver (prevents default, sets overCol), onDragLeave (clears overCol when leaving to a non-child), onDrop (calls drop). The column gets

.drop-over while hovered. A header shows a status-colored dot (STATUS_COLORS), the title, and a count. Each card is draggable, sets/clears dragId on drag start/end (with a .dragging class), and opens the task on click. Cards display the title, a ConfidenceTag (ownership confidence), a footer with PriorityBadge and due label (dueLabel(t)), and — when assigned — an avatar + assignee name. Empty columns show "Drop tasks here".

TaskDrawer.tsx

A large right-side drawer showing a single task's full detail and all its actions: inline editing, assignee/priority/progress/status changes, approval workflow, splitting into shared subtasks, dependencies, and comments. Behavior varies by the current user's role (manager vs. employee).

Constant STATUSES — the six lifecycle statuses.

Component: TaskDrawer({ taskId, onClose, onChange? })

- **Props:** taskId: string, onClose: () => void, onChange?: () => void (invoked after any mutation so parent lists refresh).
- **Hooks:** useAuth() → { user }; useEscape(onClose).
- **State:** task: Task | null, users: User[], comment (draft text), busy: boolean, pendingAssignee (member picker selection), pendingStatus (a status picked but not yet confirmed), editing: boolean, editForm ({ title, description, due_date }), showSplit: boolean, and parts ({ title, assignee_id }[] for the split modal).
- **Function updatePart(i, patch)** — immutably patches the i-th split part.
- **Function load()** — GET /tasks/\${taskId} → sets task.
- **Effects:**
 1. On taskId change: resets editing, loads the task, and GET /users.
 2. Syncs pendingAssignee to the task's current assignee whenever it changes.
 3. Syncs pendingStatus to the task's saved status whenever it changes (incl. after Accept).
- **Function mutate(fn)** — generic mutation wrapper: sets busy, runs fn(); if it returns an object with an id, sets it as the task, else reloads; calls onChange?.(); always clears busy.
- **Mutation helpers (each via mutate):**
 - setStatus(status) — POST /tasks/\${taskId}/status { status }.
 - setAssignee(assignee_id) — PATCH /tasks/\${taskId} { assignee_id: id || null }.
 - setPriority(priority) — PATCH /tasks/\${taskId} { priority }.
 - setProgress(progress) — PATCH /tasks/\${taskId} { progress }.
 - approve(decision) — POST /tasks/\${taskId}/approve { decision }.
 - addComment() — if the trimmed comment is non-empty, POST /tasks/\${taskId}/comments { body } then clears the draft.
- **Function startEdit()** — seeds editForm from the current task and opens the inline editor.
- **Function saveEdit()** — validates a non-empty title, then PATCH /tasks/\${taskId} with trimmed title/description, due_date (or null), and due_date_raw: null (so the picked date becomes authoritative); closes the editor.
- **Function openSplit()** — seeds parts with one row (title = task title, no assignee) and opens the split modal.
- **Function doSplit()** — filters parts to those with a title and assignee, and if any exist POST /tasks/\${taskId}/split { parts }; closes the modal.

- **Function del()** — awaits `confirmDialog` (danger); if confirmed, sets busy, `DELETE /tasks/${taskId}`, calls `onChange?.()`, closes.
- **Derived flags:** `isManager` (`user.role !== 'employee'`); `canDelete` (managers, or an employee owning their own private draft — `visible_to_manager === 0 && assignee.id === user.id`); `canSplit` (manager or owner, and not itself a subtask); `subs = subtasks`, `subDone/subPct` progress; `lookupUser(uid)`.
- **UI behavior:** Shows a loading spinner drawer until the task loads. Otherwise renders an overlay + drawer (backdrop click closes). Header: priority & status badges, a "🔒 Private" badge when applicable, and (for managers) an Edit button, a Delete button (when `canDelete`), and × close. Body toggles between an inline edit form (title, description, save/cancel) and read-only display (title, confidence tag, description, and a "FROM MEETING (original language)" source-quote block when present). A 2-column grid shows **Assignee** (manager: a member <select> excluding admins + Assign/Unassign button; employee: read-only), a "Heard as" raw-name hint, **Priority** (manager: <select>; else badge), **Assigned by**, and **Due date** (date input while editing, else `dueLabel`). A **Timeline** lists created/assigned/submitted/completed timestamps (`fmtDateTime`). A **Progress** range slider (0–100, step 10) calls `setProgress`. The **Status** section differs by role: managers get all statuses as toggle buttons with an "Accept change → X" confirm/cancel; employees see a "Completed" card when Done, a "Submitted for review" card with a Withdraw button when approval is pending, or To Do/In Progress/Blocked toggles plus a "Submit as complete" button (sets status to "In Review"). A **Shared parts** section (for top-level tasks with subtasks or split capability) shows a progress bar, a per-part list (status glyph, title, assignee avatar/name, status badge), or an explainer, plus a "⌘ Split & share" button. Managers see an **Approval requested** card (Approve & close / Reopen). A **Depends on** list appears when dependencies exist. A **Comments** section lists comments (avatar, name, timestamp, body) and an input (Enter posts) with a Post button. Finally, a **Split & share** modal (when `showSplit`): an explainer, editable part rows (title input + assignee <select> excluding admins, with "(me)" suffix, removable when >1), an "Add another part" button, and Cancel / "✉ Share parts" actions (disabled until at least one valid part).

ToastHost.tsx

Renders the live, non-blocking toast stack, mounted once near the app root. It is the visual half of the toast bus in `lib/toast.ts`.

Constant ICON — a `Record<string, ReactNode>` mapping toast kinds to inline SVG icons: success (checkmark), error (circle with exclamation), info (circle with "i").

Component: ToastHost (default export, no props)

- **State:** `items: ToastItem[]`.
- **Effect (mount):** `subscribeToasts(setItems)`; the returned `unsubscribe` is the cleanup.
- **UI behavior:** Returns `null` when empty. Otherwise a `.toast-host` region (`aria-live="polite"`). Each toast is a button (`toast toast-<kind>`) that dismisses itself on click via `dismissToast(t.id)`, showing the kind icon and text.

UserManagement.tsx

The Administration page's user-management UI. Lets a manager/admin list, add, edit, and remove users; import users from Excel/CSV; download a CSV template; invite teammates by email; view/revoke pending invitations; and trigger the daily task digest. Contains two nested form components.

Component: UserManagement (default export, no props)

- **Hooks:** `useAuth()` → { `user` }; `isAdmin` = `user.role === 'admin'`.
- **State:** `users`, `depts`, `invites`, `editing` ('new' | `user` | null), `inviting`: boolean, `importMsg`: string (status line), `digest`: { `mode`, `hour` } | null. **Ref** `fileRef` (hidden import file input).
- **Loaders:** `load()` = GET /users; `loadInvites()` = GET /invites (errors swallowed).
- **Effect (mount):** `load()`, `loadInvites()`, GET /users/meta/departments → `depts`, GET /digest/status → `digest`.
- **Helper** `deptName(id)` — maps a department id to its name (or "").
- **Function** `revokeInvite(inv)` — confirms (danger), then DELETE /invites/<id> and reloads invites; errors set `importMsg`.
- **Function** `onImport(e)` — reads the selected file, sets "Importing...", calls `api.upload('/users/import', file)`, reports created/updated/skipped counts, reloads; errors set `importMsg`; always resets the file input.
- **Function** `downloadTemplate()` — builds a CSV template string, creates a Blob URL, and triggers a client-side download of `users-template.csv` via a temporary <a> (then revokes the URL).
- **Function** `sendDigestNow()` — confirms, sets "Sending digest...", POST /digest/send-now, reports the Cliq/email result (noting "preview" mode); errors set `importMsg`.
- **Helper** `canEdit(u)` — admins can edit anyone; managers can edit anyone except admins.
- **Function** `remove(u)` — blocks removing your own account; otherwise confirms (danger) and DELETE /users/<id>, reports and reloads; errors set `importMsg`.
- **UI behavior:** A toolbar shows the user count and digest schedule, plus buttons: Template, Import Excel/CSV (triggers the hidden .xlsx/.xls/.csv input), Send digest now, Invite teammate, and "+ Add user". Below: the status message; a "Pending invitations" card (email, role, Revoke) when any exist; and a responsive .table-cards table of users (avatar+name, email, phone, role badge, department, and — when `canEdit` — Edit/Remove actions, hiding Remove for the current user). Conditionally renders <UserForm> (add/edit) and <InviteForm>.

Component: InviteForm({ depts, isAdmin, onClose, onDone })

- **Purpose:** Invite a teammate by email and surface the resulting accept-link for manual sharing (needed when SMTP isn't configured).
- **State:** f({ `email`, `role`, `department_id` }), `busy`, `err`, `result` ({ `link`, `emailed` } | null), `copied`. `roleOptions` = ['employee', 'manager', 'admin'] for admins else without admin.
- **Function** `send()` — normalizes/validates the email with a regex; POST /invites { `email`, `role`, `department_id` || null }; on success stores `result` and calls `onDone()`; errors set `err`.
- **Function** `copyLink()` — copies the link via `navigator.clipboard.writeText` and shows "Copied!" for 1500ms.
- **UI behavior:** A modal. Before sending: email input, Role and Department selects, error text, Cancel / "Send invite". After sending: a success (emailed) or warning

(email not configured) note, a read-only link field (selects on focus) with a Copy button, and Done.

Component: `UserForm({ user, depts, isAdmin, onClose, onDone })`

- **Purpose:** Add or edit a user. `isEdit = !!user`.
- **State:** `f` seeded from the user (or defaults; new users default password 'password123', language 'en'), `busy`, `err`. `roleOptions` as above.
- **Function** `save()` — normalizes/validates the email to `@gmail.com` or `@befach.com` only; strips the phone to digits and requires exactly 10; then for edit `PATCH /users/<id>` (deleting a blank password so it isn't overwritten), or for create `POST /users`; calls `onDone()`; errors set `err`.
- **UI behavior:** A modal with a 3-column grid (Name, Email, Phone — `maxLength 10`, numeric), a 2-column grid (Role, Department selects), a Password field (with "leave blank to keep current" hint when editing), error text, and Cancel / Create-or-Save actions (disabled until name+email present, spinner while busy).

VoiceAssistant.tsx

The floating voice-assistant widget: a mic FAB plus an expandable conversation panel. It is a thin presentational shell over the `useVoiceAssistant` hook and the `useWakeWord` hook; all conversation logic lives in those.

Local icon components: `MicIcon({ size = 24 })`, `SpeakerIcon({ size = 20 })`, `SpeakerOffIcon({ size = 20 })` — inline stroked SVGs, all `aria-hidden`.

Constant `STATUS_LABEL` — maps each `VoiceState` (`idle`, `listening`, `processing`, `speaking`, `confirming`, `error`) to a human status line (e.g. `confirming` → 'Say "yes" to confirm, or "no"').

Component: `VoiceAssistant` (default export, no props)

- **Hooks:** `v = useVoiceAssistant()` (the engine); `logRef` (conversation-log element for auto-scroll).
- **Wake word:** `useWakeWord({ enabled: !v.open, onWake: v.start })` — the wake word is active only while no session is open (so it doesn't retrigger mid-conversation) and starts a session when heard.
- **Effect:** on any change to `v.messages/v.state`, scrolls the log to the bottom.
- **Derived:** `ringScale = 1 + Math.min(0.6, v.level * 0.9)` — the FAB's listening-glow scale driven by mic level.
- **UI behavior:** A `.va-root` fixed container. When `v.open`, a `.va-panel` (`role="dialog"`) with: a header (a state-colored dot, "BTM Voice" title, a TTS mute/unmute toggle calling `v.setTtsOn(!v.ttsOn)`, and a Close button calling `v.close`); a scrollable `.va-log` showing a hint (mentioning "hey BTM" if `wakeWordConfigured()`, plus an example command) when empty, else the messages styled per role; a `.va-confirm` card (when `v.pending`) showing the action summary with `Confirm(v.confirmPending)` / `Cancel(v.cancelPending)`; and a footer with the status label and a mic button (`v.micButton`) that shows a spinner while processing, otherwise the mic icon, with a level-driven box-shadow while listening. Always renders the `.va-fab` mic button that toggles the session (`v.open ? v.close() : v.start()`), scaled by `ringScale` while listening.

14. Voice Pipeline (client/src/voice/)

The voice assistant is a hands-free, multilingual conversational loop. The full audio path is: **wake word** (optional, on-device) or a mic tap → **mic capture with voice-activity detection** → **speech-to-text** (server) → **intent** (server "brain") → **action** (navigate / answer / confirm-then-mutate) → **text-to-speech** reply → re-open the mic. The four files below implement the capture, TTS, orchestration, and wake-word stages.

voice/recorder.ts

Microphone recorder with voice-activity detection (VAD) — the capture primitive for each conversational turn. It records via `MediaRecorder` (webm) while a `Web Audio AnalyserNode` watches the input level, and auto-stops after the speaker falls silent, so turns end without a tap. A hard time cap is the safety net.

Interface Recording — { `stop(): void`, `cancel(): void`, `done: Promise<Blob>` }. `stop` finalizes early (resolving `done` with the captured blob); `cancel` aborts (resolving with an empty blob); `done` resolves with the recorded `Blob` (empty if nothing usable).

Interface RecordOptions — `onLevel?(0..1)` (mic level for the UI meter), `silenceMs?` (quiet time after speech that ends the turn), `maxMs?` (hard cap), `minSpeechMs?` (minimum sound required before silence can end the turn), `speechThreshold?` (RMS level counted as speaking), `noSpeechMs?` (give-up window if nobody speaks).

Function canRecord() — returns whether `navigator.mediaDevices.getUserMedia` and `MediaRecorder` exist.

Function startRecording(opts = {}) → `Promise<Recording>`

- Defaults: `silenceMs=1400`, `maxMs=15000`, `minSpeechMs=350`, `speechThreshold=0.045`, `noSpeechMs=6000`.
- **Steps:**
 1. `getUserMedia({ audio: true })` for the mic stream.
 2. Creates a `MediaRecorder` (preferring audio/webm, falling back to default mime). Collects data chunks via `ondataavailable`.
 3. Sets up level detection: an `AudioContext` + `AnalyserNode` (`fftSize 512`) fed from a `MediaStreamSource`, with a `Uint8Array` time-domain buffer.
 4. Tracks `startedAt`, `speechAccum` (ms of detected speech), `lastLoudAt`, `cancelled`, and `animation/timeout` handles.
 5. `cleanup()` cancels the `rAF` loop and `max` timer, stops all stream tracks, and closes the `AudioContext`.
 6. Builds the `done` promise; `mr.onstop` runs `cleanup` and resolves either an empty blob (if cancelled) or a blob of the chunks.
 7. `finish()` stops the recorder if active (falling back to direct `cleanup/resolve` on error).
 8. `tick()` (per animation frame) reads time-domain data, computes RMS, reports `min(1, rms*3)` via `onLevel`. If `rms ≥ speechThreshold`, accumulates 16ms of speech and updates `lastLoudAt`. It calls `finish()` when the speaker has spoken (`speechAccum ≥ minSpeechMs`) and then been silent for `silenceMs`, or when nobody spoke at all within `noSpeechMs`.
 9. Starts the recorder, the `rAF` loop, and the `maxMs` safety timer.
- **Returns** { `stop: finish`, `cancel: () => { cancelled = true; finish() }`, `done` }.

- **Browser APIs:** getUserMedia, MediaRecorder, AudioContext/webkitAudioContext, AnalyserNode, requestAnimationFrame.

voice/tts.ts

Text-to-speech for the assistant's spoken replies, abstracting over a native Android plugin and the web Speech Synthesis API. `speak()` resolves when the utterance finishes so the loop can re-open the mic right after the assistant stops talking.

- **NativeTTS** — a Capacitor plugin proxy created via `registerPlugin<TextToSpeechPlugin>('TextToSpeech')` (the @capacitor-community/text-to-speech plugin, wired natively via `cap sync`; never imported into the web bundle). Interface: `speak({ text, lang?, rate?, pitch?, volume? })` and `stop()`.
- **isNative()** — `Capacitor.isNativePlatform()`.
- **ttsSupported()** — true if native or `window.speechSynthesis` exists.
- **Module enabled flag** with **setTtsEnabled(on)** (also stops speaking when turned off) and **isTtsEnabled()**.
- **stopSpeaking()** — on native calls `NativeTTS.stop()`; on web calls `window.speechSynthesis.cancel()`.
- **speak(text, lang = 'en-US') → Promise<void>** — no-op if disabled or empty. Stops any current speech. On native, awaits `NativeTTS.speak({...})` at rate/pitch/volume 1.0. On web, wraps `SpeechSynthesisUtterance` in a promise resolved on `onend/onerror`, plus a length-based safety timeout (`min(15000, 1200 + text.length*60)`ms) for browsers that never fire `onend`. Never rejects — TTS is best-effort.

voice/useVoiceAssistant.ts

The conversation engine and hands-free state machine (`idle` → `listening` → `processing` → `{navigate | answer | confirm | clarify}` → `speaking` → `re-listen`, falling back to `idle` on silence). It records a turn (recorder VAD), transcribes it, asks the server what to do, and acts — for data-changing actions it requires a spoken (or button) yes/no before executing against the normal task APIs, preserving permissions & notifications.

Types: `VoiceState; VoiceMsg ({ role: 'user' | 'ai', text }); PendingAction ({ kind, task_id?, summary?, body })`.

Regexes YES_RE / NO_RE — multilingual yes/no matchers covering English plus Hindi/Telugu/other transliterations (e.g. `haan, ji, karo, avunu, cheyyi` for yes; `nahi, mat, vddu, venda` for no) — used to resolve a pending confirmation locally.

Hook: useVoiceAssistant()

- **State:** `open, state, messages, level, pending, ttsOn` (initialized from `isTtsEnabled()`).
- **Refs (mirror state for the async loop, avoiding stale closures):** `sessionRef` (bump to invalidate an in-flight loop), `recRef` (active recording), `pendingRef`, `msgsRef`, `emptyStreakRef` (consecutive empty transcriptions), `openRef`.
- **push(m)** appends a message (keeping the last ~20). **setPend(p)** updates both the ref and state. **setTtsOn(on)** updates the module flag and local state.
- **transcribeBlob(blob) → Promise<string>** — builds `FormData` with the webm blob as `command.webm`, adds a Bearer auth header, POST

- `${API_BASE}/api/tasks/transcribe` (cache: 'no-store'), and returns the trimmed transcript (throws on non-OK).
- **sendCommand(transcript)** — POST `/assistant/command` with { transcript, history: last 8 messages } (the server "brain").
 - **execute(action)** — sets processing; dispatches by kind: `create_task` → POST `/tasks`; `update_status` → POST `/tasks/${task_id}/status`; else → PATCH `/tasks/${task_id}` (assign/priority/due). On success pushes a "✓" message, speaks "Done.", dispatches a `tasks-changed` CustomEvent, and navigates to `/tasks`. On error pushes/speaks the failure.
 - **say(text)** — sets speaking and awaits `speak(text)`.
 - **handleResponse(resp)** — pushes any `resp.say`, then branches on `resp.mode`: `confirm` (stores `resp.action` as pending, sets confirming, speaks the prompt), `navigate` (speaks, then navigates to `resp.navigate.url`), or `answer/clarify/default` (just speaks).
 - **runTurn() (the loop, memoized)**: captures `mySession`; `alive()` = session unchanged and panel open. While alive:
 1. **Listen** — `startRecording` (feeding level), await `rec.done`; a mic-access failure ends the loop with a spoken message.
 2. **Transcribe** — empty blob ends the loop (→ idle). Transcribes; on two consecutive empty results ends the loop, otherwise says "Sorry, I didn't catch that." and continues; a real transcript is pushed as a user message.
 3. **Pending yes/no** — if a confirmation is pending, tests the transcript against `YES_RE/NO_RE`: yes-only executes it; no-only cancels; ambiguous clears it and treats the text as a new command.
 4. **Command** — `sendCommand(text)` then `handleResponse`; loops back to re-open the mic.
 - **Public controls**:
 - **start()** — if `!canRecord()` shows a mic message; opens the panel, resets the empty streak, bumps the session, sets `openRef`, and runs a turn.
 - **stop()** — invalidates the session, cancels any recording, stops speaking, clears pending, sets idle.
 - **close()** — `stop()` then closes the panel.
 - **micButton()** — while listening, finalizes the current turn (`rec.stop()`); while speaking, barges in (`stopSpeaking()`); while idle, starts a fresh turn.
 - **confirmPending()** — clears pending, invalidates+cancels the current recording, executes the action, then resumes the loop.
 - **cancelPending()** — clears pending and pushes/speaks "Okay, cancelled."
 - **Cleanup effect**: on unmount, invalidates the session, cancels recording, stops speaking.
 - **Returns**: { open, state, messages, level, pending, ttsOn, start, stop, close, micButton, confirmPending, cancelPending, setTtsOn, setOpen }.
 - **Browser/APIs used**: `fetch` (transcribe), `api` (command/tasks), `CustomEvent('tasks-changed')`, `react-router navigate`.

voice/wakeword.ts

On-device wake-word ("hey BTM") detection using **openWakeWord** ONNX models run in the browser/WebView via `onnxruntime-web` (WASM). Free and unlimited (no cloud). It is optional and config-gated: with the `env` flag unset or model files missing it no-ops (falling back to the mic button), and any load/runtime error disables it silently so it can never break the app.

openWakeWord pipeline: audio → [melspectrogram] → mel frames → [embedding] → 96-d vectors → [wakeword] → score. The first two ONNX models are fixed/shared; the third is the custom "hey BTM" model.

Config (Vite env): `ENABLED (VITE_WAKEWORD_ENABLED === 'true')`, model paths `MEL_PATH/EMB_PATH/WW_PATH` (default under `/wakeword/`), `THRESHOLD` (default 0.5), and `ORT_WASM` (onnxruntime WASM base URL, default a pinned jsDelivr CDN, overridable for offline).

wakeWordConfigured() — returns `ENABLED`.

Streaming constants: `SR=16000`, `CHUNK=1280` (80ms hops), `MEL_BINS=32`, `EMB_WINDOW=76` mel frames/embedding, `WW_EMBS=16` embeddings/input, `MEL_HOP=160`, `MEL_WIN=640`, `MEL_FEED=1760` (samples fed to melspec for 8 fresh frames), `REFRACTORY_MS=1500`. The design feeds the melspec model only the newest 1760 samples for exactly 8 new mel frames, then takes one embedding from the newest 76 frames — an incremental pipeline that avoids recomputing 2s of audio each chunk.

Hook: `useWakeWord({ enabled, onWake })` — `onWakeRef` mirrors the callback. Effect (keyed on `enabled`): no-ops unless `enabled && ENABLED`; otherwise asynchronously `import('onnxruntime-web')`, sets `ort.env.wasm.wasmPaths = ORT_WASM`, creates the three `InferenceSessions` (wasm EP) in parallel, builds the detector, and starts the mic pump. Any error is logged as a warning (disabled silently). Cleanup cancels and stops the pump.

createDetector(ort, mel, emb, ww, onWake) — builds the incremental frame processor with rolling rings:

- `runMel()` — scales the newest samples to int16 range (`*32767`), runs the melspec model, appends the resulting mel frames (each normalized `/10 + 2`) to `melRing`, trimming it.
- `runEmb()` — takes the newest 76 mel frames, runs the embedding model, appends the first 96 values to `embRing`, trimming to 16.
- `score()` — once enough samples/frames/embeddings exist, flattens the 16 embeddings, runs the wakeword model, reads the score, and fires `onWake()` when `score ≥ THRESHOLD` and outside the refractory window.
- The returned function chains each incoming frame onto a serialized promise (frames must be processed in order without drops to preserve the 8-frame embedding stride), merging into the rolling raw buffer capped at `MEL_FEED`, then scoring.

startMicPump(onFrame) → `Promise<() => void>` — opens the mic at 16 kHz mono, wires a `MediaStreamSource` → deprecated `ScriptProcessorNode` (4096 buffer) → destination, and in `onaudioprocess` accumulates samples and emits exactly `CHUNK`-sized frames to `onFrame`. Returns a teardown function that disconnects the nodes, stops the tracks, and closes the context.

15. Utility Libraries (client/src/lib/)

lib/confirm.ts

A promise-based confirm-dialog bus replacing blocking `window.confirm()`. Any module can await `confirmDialog(...)`; `<ConfirmHost/>` renders the themed dialog and resolves the promise.

- **Interfaces:** ConfirmOptions (title?, message, confirmText?, cancelText?, danger?); PendingConfirm extends it with id and resolve(v: boolean).
- **Module state:** current (the active request), listeners, nextId. emit() notifies all listeners with current.
- **subscribeConfirm(1)** — registers a listener, immediately pushes current, and returns an unsubscribe function.
- **confirmDialog(opts)** → **Promise<boolean>** — supersedes any open dialog (resolving it false), stores a new PendingConfirm (auto-incremented id), emits, and resolves when the user chooses.
- **resolveConfirm(value)** — clears current, emits (closing the dialog), and resolves the stored promise with value.

lib/passwordStrength.ts

Lightweight, dependency-free password-strength scoring (0–4) with a common-password blocklist.

- **COMMON** — a Set of ~28 well-known weak passwords.
- **isCommonPassword(pw)** — lowercases and checks membership in COMMON.
- **Interface Strength** — { score, label, ok }.
- **passwordStrength(pw)** → **Strength** — empty → score 0, blank label, not ok. A common password is forced to score 0, label "Too common". Otherwise scores: +1 for ≥ 8 chars, +1 for ≥ 12, +1 for mixed lower+upper case, +1 for a digit, +1 for a symbol; capped at 4. Labels: ['Very weak', 'Weak', 'Fair', 'Good', 'Strong']. ok (acceptable to submit) requires ≥ 8 chars and score ≥ 2 (Fair or better).

lib/pcmStream.ts

Captures microphone audio and emits base64-encoded little-endian 16-bit PCM at 16 kHz — the format Sarvam's streaming STT WebSocket expects. Because MediaRecorder only produces webm/opus, this taps raw samples via the Web Audio API instead. (This is the streaming-STT capture primitive, distinct from recorder.ts's webm VAD recorder.)

- **Interface PcmStream** — { stop(): void }.
- **float32ToBase64Pcm16(input)** — converts Float32 [-1,1] samples to clamped Int16 ($s < 0 ? s * 0x8000 : s * 0x7fff$), then to bytes, then to a base64 string via btoa (browser-safe, no Buffer).
- **downsampleTo16k(input, inputRate)** — linear-ish averaging downsample from the AudioContext rate (usually 44.1/48 kHz) to 16 kHz; returns input unchanged if already 16 kHz.
- **startPcmStream(onFrame, onError)** → **Promise<PcmStream>** — requests the mic (channelCount 1, echo cancellation, noise suppression); on failure calls onError with a permission message and returns a no-op stop. Otherwise builds an AudioContext → MediaStreamSource → deprecated ScriptProcessorNode (4096 buffer, connected to destination so onaudioprocess fires) that downsamples each buffer and calls onFrame with the base64 PCM chunk. stop() disconnects the nodes, closes the context, and stops the mic tracks.

lib/toast.ts

A tiny global toast bus. Any module can call `toast.success/error/info` without threading a hook; `<ToastHost/>` subscribes and renders. Replaces native `alert()/confirm()` popups.

- **Types:** `ToastKind ('success' | 'error' | 'info');` `ToastItem ({ id, kind, text })`.
- **Module state:** `items, listeners, nextId`. `emit()` sends a snapshot to all listeners.
- **`subscribeToasts(1)`** — registers a listener, pushes the current items, returns an `unsubscribe`.
- **`dismissToast(id)`** — removes a toast and emits.
- **`add(kind, text, ms) (private)`** — appends a toast, emits, and (if `ms > 0`) auto-dismisses after `ms`; returns the `id`.
- **`toast`** — object with `success(text, ms=3200)`, `error(text, ms=5000)`, `info(text, ms=3500)`.

lib/useEscape.ts

A hook that invokes a callback on the Escape key, used by modals/drawers for keyboard dismissal (mobile uses the Android back handler instead).

- **`useEscape(onClose)`** — stores `onClose` in a ref (kept current via an effect so the listener always calls the latest callback) and, once on mount, installs a document keydown listener that fires `ref.current()` on Escape, removed on cleanup.

lib/wallpaper.ts

App background ("wallpaper") management — a per-device preference stored in `localStorage` and applied via the `--app-wallpaper` CSS variable (purely client-side, no backend field).

- **Interface `Wallpaper`** — `{ id, name, value, swatch }` (`value` is the CSS background applied to `<body>`; `swatch` is a small preview; `id 'default'` clears the override to fall back to `--bg`).
- **`WALLPAPERS`** — seven presets: Default, Sunset, Ocean, Mint, Lavender, Slate, Charcoal (each a linear gradient except Default).
- **`KEY`** — the `localStorage` key `'appWallpaper'`.
- **`getWallpaperId()`** — reads the stored `id` (default `'default'`).
- **`applyWallpaper(id)`** — resolves the wallpaper (falling back to the first entry), persists the `id`, and sets or removes the `--app-wallpaper` custom property on `document.documentElement`.

16. styles.css — Design System Summary

A single ~1100-line global stylesheet implementing the "Befach Task Manager" theme. There is no separate light/dark class-toggle theme system; theming is driven by CSS custom properties, with a dark-mode override only for the voice panel via `prefers-color-scheme: dark`, and heavy use of `color-mix()` for tints.

Design tokens (:root variables):

- Colors: `--bg` (#f8e5c5 warm tan), `--surface` (#fff), `--border` (#e7d6bc), `--text` (#1f1a16), `--muted` (#7a6f63), `--primary` (#c5560f deep orange), `--primary-dark` (#a8480c), `--accent` (#d4a017 gold), `--sidebar` (#201a16 espresso), `--sidebar-muted` (#b6a99d).
- Shape/elevation: `--radius` (12px), `--shadow`, `--shadow-lg`.
- `--app-wallpaper` — set at runtime by the wallpaper picker; body uses `background: var(--app-wallpaper, var(--bg))` with `background-attachment: fixed`.
- Per-component runtime variable `--kc` (KPI accent color) used by dashboard KPI cards.
- Base typography: Inter (with system fallbacks), 14px/1.5 body.

Main sections (in file order):

- **Base/reset** — box-sizing, full-height layout, links, buttons, headings, and unified input/textarea/select styling with an orange focus ring.
- **Buttons** — `.btn` base plus variants: `.btn-primary` (orange gradient), `.btn-ghost`, `.btn-sm`, `.btn-danger`, `.btn-danger-solid`, `.btn-done/.btn-done-soft/.btn-tick` (green completion actions), and disabled styling.
- **Voice/mic buttons** — `.btn-mic`, `.btn-mic-live` (pulsing red), `.mic-dot`, and the signature `.btn-mic-hero` (pill with an attention-pulse animation, respecting `prefers-reduced-motion`); keyframes `speak-attention`, `micpulse`, `bell-vibrate`.
- **Split & share** — `.split-part`, `.split-part-title`, `.split-part-who` sizing rules.
- **Layout** — `.app`, `.sidebar` (dark, sticky) with `.brand`, `.nav`, `.sidebar-user`, `.logout-btn`; `.main`, `.topbar`, `.content`; mobile `.nav-toggle`, `.back-btn`, `.sidebar-backdrop`.
- **Cards & grid utilities** — `.card`, `.card-pad`, `.card-head`, `.grid/.grid-2/.grid-3/.grid-stats`, `.row`, `.spread`, `.muted`, `.wrap`.
- **Stat cards** — `.stat-card`, `.stat-value`, `.stat-label`, `.stat-hint`.
- **Badges/avatar** — `.badge`, `.avatar`.
- **Tables** — base `table/th/td`, `.clickable`, `.day-group-row`, `.audit-detail`, and the responsive `.table-cards` pattern that reflows rows into stacked label:value cards below 640px (using `data-label::before`).
- **Bars** — `.bar-track/.bar-fill` progress bars.
- **Download report dropdown** — `.report-menu`, `.report-custom`.
- **Power BI-style dashboard** — a single-screen mosaic: `.pbi`, `.pbi-filter`, segmented range tabs (`.pbi-seg/.pbi-tab`), `.pbi-datepop`, and `.pbi-kpis/.kpi` KPI cards (accent-driven via `--kc`, with left color bar, icon tile, clickable/hover states, and an overdue blink).
- **Kanban** — `.board`, `.board-col` (with `.drop-over` drop target), `.board-col-head/-dot/-title/-count/-body/-empty`, `.board-card` (draggable, `.dragging`, hover lift), and card sub-parts. (Note the file defines `.board` twice — an early grid version and a later flex horizontal-scroll version that wins.)
- **Login** — `.pw-row/.pw-toggle` (show/hide password), peeking-panda login art, and `.pw-strength/.pw-strength-bar/.pw-seg/.pw-strength-label` (the password meter).
- **Voice assistant** — `.va-root`, `.va-fab` (with state variants and a `va-pulse` listening animation), `.va-panel`, `.va-head`, state dots `.va-dot--*`, `.va-log`, `.va-hint`, message bubbles `.va-msg--user/--ai`, `.va-confirm`, `.va-foot`, `.va-mic` — including a `@media (prefers-color-scheme: dark)` block that darkens the panel, bubbles, and mic.
- **Modal / drawer** — `.overlay + .drawer` (right-slide), `.modal-center + .modal`, plus `.modal-overlay/.modal-card/.modal-tabs/.modal-list/.modal-user` for the picker modal, and `.transcript/assistant` sections.

- **Profile modal** — .profile-modal, .profile-id, .profile-role, .profile-section, .profile-msg.ok/.err, .profile-logout, and the .wallpaper-grid/.wallpaper-swatch/.wallpaper-check picker (7-column, collapsing to 4 on small phones).
- **Empty states & spinner** — .empty, .empty-state*, .spinner (spin keyframe).
- **Notification mute toggle & rows** — .mute-toggle (.on/.off) and .notif-item.actionable (hover/focus states).
- **Custom sort menu** — .sortmenu* (a themed replacement for the native <select>, with a sortmenu-in animation).
- **Search UI** — .icon-btn, collapsible .search-box, .search-dialog*, and the .search-active applied-search chip.
- **Comments** — .comment/.comment .body.
- **Responsive breakpoints** — @media blocks at 1000px, 760px (tablets), 720px (phones: off-canvas sidebar, stacked toolbars, single-column boards/modals, roomier chat, touch targets), 640px (table→cards), 560px, 430px (small phones: 2-col KPIs). A dedicated 720px block also forces overflow-x: hidden app-wide.
- **UI/UX polish layer (bottom, numbered 0–8)** — additive global enhancements: (0) min-width:0 layout-safety to prevent horizontal overflow; (1) :focus-visible keyboard focus rings; (2) modal/overlay entrance motion (overlay-fade, modal-pop) gated by prefers-reduced-motion; (3) card shadow transitions; (4) **toasts** (.toast-host, .toast, .toast-icon, and .toast-success/-error/-info color variants, toast-in animation); (5) mobile ~44px touch targets; (6) employee dashboard KPI grid & list rows; (7) themed **confirm dialog** (.confirm-modal, .confirm-body, .confirm-icon/.danger, .confirm-title/-message/-actions, .btn-danger-solid); (8a) loading **skeletons** (.skeleton shimmer, skeleton-sweep); and (8) chat touch usability.